



VARIABLES

Aprende a guardar información en Python

1 ¿QUÉ ES UNA VARIABLE?

Una variable es una **caja** donde Python guarda información.

```
nombre = "Guille"
```



2 CÓMO CREAR UNA

```
edad = 25
```

```
precio = 19.99
```

```
activo = True
```

```
nombre = "Guille"
```

→ entero (int)

→ decimal (float)

→ booleano (bool)

→ texto (str)

3 REASIGNACIÓN

```
edad = 25
```

```
edad = 26
```

```
edad = 27
```

El valor cambia.
La variable sigue siendo la **misma**.

¿SABÍAS?

💡 Python es **dinámico**: no necesitas declarar el tipo de dato.
¡Solo asigna y listo! 😊

4 BUENAS PRÁCTICAS

✓ NOMBRES DESCRIPTIVOS

```
nombre_cliente  
total_pedido  
es_activo
```

✗ EVITAR

```
a  
b  
x1
```

★ TIP PRO

Usa nombres que expliquen qué guarda la variable.
Tu código será más fácil de **entender y mantener**.





VARIABLES

Todo lo que necesitas saber



5 NOMBRES DE VARIABLES

Reglas para elegir buenos nombres:

- ✓ Deben empezar con letra o guion bajo (_)
- ✓ Pueden contener letras, números y guion bajo
- ✓ No pueden usar espacios
- ✓ No pueden ser palabras reservadas (como if, for, True, etc.)
- ✓ Usa nombres que tengan **sentido**



¡Un buen nombre hace que tu código sea **fácil de entender** para ti y para otros!



NOMBRES CORRECTOS ✓

mi_edad
total_precio
usuario1
_temporal
nombreCompleto

NOMBRES INCORRECTOS ✗

1nombre (empieza con número)
total precio (tiene espacio)
mi-edad (usa guion medio)
for (palabra reservada)



6 TIPOS DE DATOS EN VARIABLES

Python detecta **automáticamente** el tipo de dato según el valor asignado.

TIPO	EJEMPLO	CÓDIGO	DESCRIPCIÓN
Entero (int)	25	edad = 25	Números sin decimales (positivos o negativos) 123
Decimal (float)	3.14	precio = 3.14	Números con decimales 🥧
Texto (str)	"Hola"	saludo = "Hola"	Cadenas de texto entre comillas simples o dobles Hola!
Booleano (bool)	True False	activo = True logueado = False	Solo pueden ser True o False ✓ ✗

7 ¡IMPORTANTE!



- Python es case-sensitive: **edad** ≠ **Edad**
- Las variables deben asignarse antes de usarse
- Puedes tener variables del mismo nombre en diferentes partes de tu programa (se pueden reutilizar)

```
# Buenas variable
mensaje = "¡Vamos!"
contador = 1
total = 0.0
activo = True
print(mensaje)
```




VARIABLES

Todo lo que necesitas saber

8 CONSTANTES (POR CONVENCIÓN)

Python no tiene constantes reales, pero usamos **MAYÚSCULAS** para indicar que un valor no debería cambiar.

```
PI = 3.1416
IVA = 0.21
NOMBRE_APP = "GuilleApp"
```

💡 No es obligatorio, pero es una buena práctica para que otros entiendan tu código fácilmente.

9 ASIGNACIÓN MÚLTIPLE

Puedes asignar varios valores en una sola línea.

MISMO VALOR

```
x = y = z = 0
```

↓ ↓ ↓
0 0 0

x, y y z tienen el mismo valor.

DIFERENTES VALORES

```
a, b, c = 10, 20, "Hola"
```

↓ ↓ ↓
10 20 Hola

Cada variable recibe su valor.

¡Menos líneas, mismo poder!

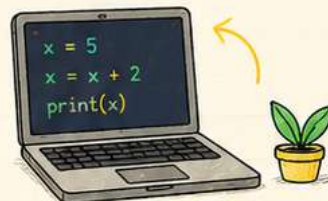
10 OPERACIONES CON VARIABLES

Podemos usar variables en operaciones matemáticas.

OPERACIÓN	EJEMPLO	CÓDIGO	RESULTADO
Suma (+)	5 + 3	a = 5 b = 3 resultado = a + b	8
Resta (-)	10 - 4	a = 10 b = 4 resultado = a - b	6
Multiplicación (*)	4 * 2	a = 4 b = 2 resultado = a * b	8
División (/)	10 / 2	a = 10 b = 2 resultado = a / b	5.0
Módulo (%)	10 % 3	a = 10 b = 3 resultado = a % b	1

TIP PRO

✓ Las variables hacen tu código flexible y fácil de modificar.



¡Cambia un valor y todo se actualiza!

EN RESUMEN

- ✓ Las variables guardan información.
- ✓ Pueden cambiar su valor.
- ✓ Tienen nombres que deben seguir reglas.
- ✓ Podemos hacer operaciones con ellas.

RECUERDA

Un buen programador usa buenos nombres y entiende sus variables.



VARIABLES

RESUMEN GENERAL

MAPA MENTAL



CHEAT SHEET RÁPIDA

```
# Crear variables
nombre = "Guille" # texto (str)
edad = 25          # entero (int)
precio = 15.5      # decimal (float)
activo = True      # booleano (bool)

# Usar variables
print(nombre)
print(edad + 1)

# Ver tipo de dato
type(edad)
type(precio)

# Asignación múltiple
a = b = c = 0

# Asignación múltiple con valores
x, y, z = 10, 20, 30

# Reasignación
contador = 0
contador = contador + 1
```

ERRORES COMUNES

- ❌ `1nombre = 10`
→ No puede empezar con número.
- ❌ `total precio = 50`
→ No se permiten espacios.
- ❌ `for = 5`
→ `for` es una palabra reservada.
- ❌ `Edad = 20` y `edad = 20`
→ Python distingue mayúsculas y minúsculas.
- ❌ `print(mensaje)`
→ Usar una variable antes de crearla da error (`NameError`).

MINI QUIZ

¿Cuál de las siguientes variable es un nombre válido?

- ☐ A `1edad = 20`
- ☒ B `edad = 20`
- ☐ C `total precio = 50`
- ☐ D `for = 10`

Respuesta: B

TIP PRO

Usa nombres **descriptivos** que expliquen qué guarda la variable.

- ✓ `nombre_cliente`
- ✓ `total_pedido`
- ✓ `es_activo`
- ❌ `a`
- ❌ `x1`
- ❌ `temp`

DESAFÍO PARA TI

Crea las siguientes variables y muestra sus valores con `print()`:

- ✓ `nombre`
- ✓ `edad`
- ✓ `altura`
- ✓ `estudiante`
- ✓ `ciudad`

Ejemplo de salida:

```
Nombre: Guille
Edad: 25
Altura: 1.75
Estudiante: True
Ciudad: Buenos Aires
```



PARA LLEVAR

Las variables guardan información.



Pueden cambiar de valor.



Deben tener nombres válidos.



Se usan en operaciones y en todo tu código.





TIPOS DE DATOS

Los tipos básicos en Python



En Python, cada dato que guardamos tiene un tipo. Conocerlos nos ayuda a escribir mejores programas.

LOS 5 TIPOS BÁSICOS

TIPO	EJEMPLO	DESCRIPCIÓN	USO PRINCIPAL	ÍCONO
<code>int</code> (entero)	25	Números enteros positivos, negativos o cero.	Contar elementos, edades, posiciones, índices, etc.	123
<code>float</code> (decimal)	3.14	Números con parte decimal (con punto).	Medidas, precios, promedios, cálculos con decimales, etc.	
<code>str</code> (texto)	"Hola"	Cadenas de texto entre comillas simples o dobles.	Nombres, mensajes, direcciones, textos en general.	Hola!
<code>bool</code> (booleano)	True False	Valores lógicos que solo pueden ser True o False.	Condiciones, decisiones lógicas, validaciones, comparaciones.	☒ ☐
<code>None</code> (NoneType)	None	Representa la ausencia de valor o un valor vacío.	Cuando aún no hay dato, o una función no devuelve nada.	

RECUERDA

- ★ Cada valor que guardas en una variable pertenece a un tipo.
- ★ Python detecta el tipo de dato automáticamente.
- ★ Podemos conocer el tipo con la función `type()`.

EJEMPLO RÁPIDO

```
edad = 25
precio = 19.99
nombre = "Guille"
activo = True
nada = None

type(edad)
type(precio)
type(nombre)
type(activo)
type(nada)
```

TIP PRO



Entender los tipos de datos es la base para todo: operaciones, condicionales, funciones y mucho más.





TIPOS DE DATOS

Cómo saber el tipo y convertir datos

1 ¿CÓMO SABER EL TIPO?

Usamos la función `type()` para conocer el tipo de dato de una variable.

```

edad = 25
type(edad)      # int

precio = 19.99
type(precio)    # float

nombre = "Guille"
type(nombre)    # str

activo = True
type(activo)    # bool

nada = None
type(nada)      # NoneType

```



2 CONVERSIONES BÁSICAS

Podemos convertir un dato de un tipo a otro usando funciones.

FUNCIÓN	QUÉ HACE	EJEMPLO	RESULTADO
<code>int()</code>	Convierte a entero.	<code>int(3.14)</code>	3
<code>float()</code>	Convierte a decimal.	<code>float("5.5")</code>	5.5
<code>str()</code>	Convierte a texto.	<code>str(100)</code>	"100"
<code>bool()</code>	Convierte a booleano.	<code>bool(1)</code> <code>bool(0)</code>	True False



Importante: No todas las conversiones son posibles. Si el dato no se puede convertir, Python genera un **error**.



3 ¿QUÉ CONVERSIONES FUNCIONAN?

SÍ FUNCIONAN ✓

```

int(3.9)      → 3
int("42")     → 42
float("3.14") → 3.14
str(25)       → "25"
bool(1)       → True
bool("")      → False

```

NO FUNCIONAN ✗

```

int("Hola")   → Error (texto no numérico)
float("Hola")  → Error (texto no numérico)
int(3.14.5)    → Error (número inválido)
str(True)      → Si funciona ("True")
bool("0")      → True (texto no vacío)

```

★ RECUERDA

`bool()` considera:

- 0, "", [], {}, None son **False**
- Cualquier otro valor es **True**



4 TYPE() DEVUELVE...

La función `type()` devuelve el tipo exacto del dato.



TIP EXTRA ★

Conocer el tipo de dato es clave para:

- ✓ evitar errores
- ✓ validar información
- ✓ realizar operaciones correctas
- ✓ escribir código más eficiente



¡Entender los tipos te hace mejor programador!

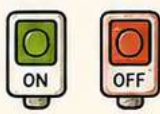




TIPOS DE DATOS

Casos prácticos en el día a día

1 ¿DÓNDE USAMOS CADA TIPO?

int (entero)	float (decimal)	str (texto)	bool (booleano)	None (NoneType)
Contar cosas enteras. Ejemplo: cantidad = 10	Medidas, precios, promedios. Ejemplo: precio = 19.90	Nombres, mensajes, direcciones. Ejemplo: nombre = "Ana"	Decisiones, estados verdadero o falso. Ejemplo: activo = True	Cuando no hay dato disponible. Ejemplo: resultado = None
				

2 ¿QUÉ OPERACIONES PODEMOS HACER?

int	float	str	bool	None
+ - * // % comparaciones Ejemplo: 10 + 5 = 15 10 // 3 = 3 10 % 3 = 1	+ - * / % comparaciones Ejemplo: 3.5 + 2.1 = 5.6 10.0 / 4 = 2.5 5.5 % 2 = 1.5	+ (concatenación) * (repetición) Ejemplo: "Hola" + " Mundo" = "Hola Mundo" "Py" * 3 = "PyPyPy"	and or not comparaciones Ejemplo: True and False = False True or False = True not True = False	No se pueden hacer operaciones matemáticas.
				

3 VALORES TRUTHY Y FALSY

Se consideran True (truthy):

- ✓ Cualquier número distinto de 0
- ✓ Cualquier string no vacío "hola"
- ✓ Listas, tuplas, sets, dicts no vacíos
- ✓ True

Se consideran False (falsy):

- ✗ 0, 0.0
- ✗ "" (string vacío)
- ✗ [] {} set()
- ✗ None
- ✗ False

Python usa estos valores para las condiciones en if, while, etc.



4 EL TIPO NoneType

None representa la ausencia de valor.

Se usa cuando aún no tenemos un dato o una función no devuelve nada.

```
resultado = None
print(resultado)
# Salida: None
```

Es útil para inicializar variables o manejar datos faltantes.

5 ERRORES COMUNES

- ✗ Sumar texto con números
"10" + 5 → Error
- ✗ Usar comillas en números
num = "25" # es str, no int
- ✗ No convertir antes de operar
"5" * 2 → "55" (texto repetido)
- ✗ Confundir = con ==
if edad = 18: → Error



¡Ojo con estos detalles!

EJEMPLO INTEGRADO

```
edad = 25          # int
altura = 1.75      # float
nombre = "Guille"  # str
estudiante = True  # bool
curso = None       # NoneType

print(nombre + " tiene " + str(edad) + " años")
print("Altura:", altura, "m")
print("¿Estudia?:", estudiante)
print("Curso asignado:", curso)
```

Cada tipo tiene su uso y propósito específico.

TIP PRO



Elegir el tipo correcto hace tu código más claro, eficiente y fácil de mantener.





TIPOS DE DATOS

RESUMEN GENERAL

1 CHEAT SHEET RÁPIDA

TIPO	EJEMPLO	CREACIÓN	TYPE() DEVUELVE
int	42	x = 42	<class 'int'>
float	3.14	x = 3.14	<class 'float'>
str	"Hola"	x = "Hola"	<class 'str'>
bool	True	x = True	<class 'bool'>
None	None	x = None	<class 'NoneType'>

★ Usa `type(variable)` para conocer el tipo de dato.

2 MINI QUIZ

¿Cuál tipo de dato es correcto para cada valor?

- 1 25 → _____ (int / str / float)
- 2 "Python" → _____ (str / int / bool)
- 3 3.75 → _____ (float / int / str)
- 4 False → _____ (bool / int / str)
- 5 None → _____ (NoneType / str / int)

Respuestas: 1) int 2) str 3) float 4) bool 5) NoneType

3 ERRORES COMUNES

- ❌ Confundir `int` con `float`
10 / 2 → 5.0 (float) no es 5 (int)
- ❌ Usar comillas en números
"25" es `str`, no `int`
- ❌ Pensar que `True` es 1 y `False` es 0
Son bool, no int (aunque `True == 1` es `True`)
- ❌ No convertir antes de operar
"10" + 5 → Error
- ❌ Usar `None` sin verificar
Si una variable es `None`, no se puede operar con ella.

4 TIP PRO

Entender los tipos te ahorra muchos errores.
Antes de operar, verifica que los datos sean del tipo esperado.

```
edad = input("Edad: ") # str
edad = int(edad)       # ahora es int
edad = edad + 1
print("El próximo año tendrás:", edad)
```

Convierte, valida y luego opera.

5 DESAFÍO PARA TI

Crea variables de diferentes tipos y muestra su tipo con `type()`.

- ✓ edad (int)
- ✓ altura (float)
- ✓ nombre (str)
- ✓ estudiante (bool)
- ✓ resultado (None)

Ejemplo de salida:

```
edad es: <class 'int'>
altura es: <class 'float'>
nombre es: <class 'str'>
estudiante es: <class 'bool'>
resultado es: <class 'NoneType'>
```

6 PARA LLEVAR

- ✓ Cada valor que guardamos tiene un tipo.
- ✓ Los tipos básicos son: `int`, `float`, `str`, `bool` y `None`.
- ✓ Usamos funciones como `type()` para conocer el tipo.
- ✓ Podemos convertir datos entre tipos con `int()`, `float()`, `str()` y `bool()`.
- ✓ Elegir el tipo correcto hace tu código más claro, seguro y eficiente.

EN RESUMEN

Los tipos de datos son la base de todo en Python.
Conócelos, úsalos y conviértelos cuando sea necesario.





CASTING

CONVERSIÓN DE TIPOS



¿QUÉ ES CASTING?

El casting (conversión de tipos) es **cambiar un dato de un tipo a otro** para poder usarlo de la forma que necesitamos.

¿POR QUÉ EXISTE?

- ✓ Los datos llegan en diferentes formatos (texto, números, etc.).
- ✓ Necesitamos convertirlos para realizar operaciones.
- ✓ Nos permite mostrar, calcular y validar correctamente.



TIPOS DE CONVERSIÓN

1. CONVERSIÓN IMPLÍCITA

Python la hace **automáticamente** en algunos casos.

Ejemplo:

```
a = 5          # int
b = 2.5        # float
c = a + b      # Python convierte automáticamente a float
print(c)       # 7.5
```

→ Suele pasar entre `int` y `float`.

2. CONVERSIÓN EXPLÍCITA

La hacemos nosotros usando funciones de casting.

Ejemplo:

```
texto = "25"    # str
numero = int(texto) # convertimos
print(numero)   # 25
```

→ Es la forma más común en Python.

FUNCIONES DE CASTING PRINCIPALES

int()

Convierte a número entero.

123

`int("10") → 10`

float()

Convierte a número decimal.

3.14

`float("3.5") → 3.5`

str()

Convierte a texto (cadena).

"Hola"

`str(100) → "100"`

bool()

Convierte a booleano (True/False).



`bool(1) → True`



IDEA CLAVE

El casting nos da poder y flexibilidad para trabajar con los datos correctamente en cualquier situación.



Volumen 3 • Casting



Designed & Summarized by GuilleGiu



CASTING

CONVERSIÓN DE TIPOS



TODAS LAS CONVERSIONES PRINCIPALES

Estas son las conversiones más comunes que podemos hacer en Python.

FUNCIÓN	DESCRIPCIÓN	ENTRADA (EJEMPLO)	CÓDIGO	SALIDA	TIPO RESULTANTE
<code>int()</code>	Convierte a número entero.	"25" (str) 3.99 (float) - True (bool)	<pre>int("25") int(3.99) int(True)</pre>	25 3 1	int
<code>float()</code>	Convierte a número decimal.	"3.14" (str) 25 (int) - True (bool)	<pre>float("3.14") float(25) float(True)</pre>	3.14 25.0 1.0	float
<code>str()</code>	Convierte a texto (cadena).	100 (int) 3.5 (float) - True (bool)	<pre>str(100) str(3.5) str(True)</pre>	"100" "3.5" "True"	str
<code>bool()</code>	Convierte a booleano (True/False).	1 (int) 0 (int) "" (str vacío) "Hola" (str)	<pre>bool(1) bool(0) bool("") bool("Hola")</pre>	True False False True	bool



CONVERSIONES VÁLIDAS

Python puede convertir automáticamente entre muchos tipos cuando el valor lo permite.

Ejemplos:

```
int("10")    → 10
float("2.5")  → 2.5
str(200)      → "200"
bool(1)       → True
bool(0)       → False
float(10)     → 10.0
int(3.99)     → 3
```



CONVERSIONES INVÁLIDAS

Si el valor no se puede interpretar como el tipo al que queremos convertir, Python genera un error (ValueError).

Ejemplos:

```
int("Hola")
float("abc")
int("12.5.3")
float("10,5") # coma no válida
bool("maybe") # no acepta texto así
```



RECUERDA

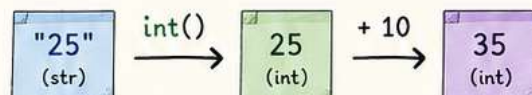


El casting **no** cambia el valor original.
Solo crea una nueva representación del dato en otro tipo.



EJEMPLO RÁPIDO

```
texto = "25"          # str
numero = int(texto)    # convertimos a int
suma = numero + 10     # ahora podemos sumar
print(suma)            # 35
```



Sin el casting, Python no puede sumar texto con números.





CASTING

CONVERSIÓN DE TIPOS



CASOS PRÁCTICOS

Veamos ejemplos reales de cómo usamos el casting en el día a día.

1. input() siempre devuelve texto (str)

```
edad = input("¿Cuál es tu edad?" ) # "25"
edad = int(edad) # 25
print(edad + 5) # 30
```

→ Convertimos el texto ingresado a int para poder sumar.



2. Trabajando con decimales

```
precio_str = "15.90"
precio = float(precio_str) # 15.90
descuento = precio * 0.10 # 1.59
print(descuento) # 1.59
```

→ Convertimos el texto a float para realizar cálculos con decimales.



3. Mostrar números en mensajes

```
puntos = 250
mensaje = "Tienes " + str(puntos) + " puntos"
print(mensaje) # Tienes 250 puntos
```

→ Convertimos int a str para poder unirlo con texto y mostrarlo.



4. Convertir a booleano (True / False)

```
activo = bool(1) # True
vacio = bool("") # False
texto = bool("Python") # True
print(activo, vacio, texto) # True False True
```

→ Útil para validar condiciones y tomar decisiones.



TRUTHY Y FALSY

En Python, algunos valores se consideran True (truthy) y otros False (falsy) cuando los convertimos con bool().

Se consideran True (truthy)

- Números distintos de 0 → bool(10) # True
- Cadenas no vacías → bool("Hola") # True
- Listas, tuplas, sets no vacíos → bool([1,2]) # True
- Diccionarios no vacíos → bool({"a":1}) # True
- Cualquier objeto → bool(object()) # True



Se consideran False (falsy)

- 0 (cero) → bool(0) # False
- 0.0 (cero decimal) → bool(0.0) # False
- Cadena vacía "" → bool("") # False
- Listas, tuplas, sets vacíos → bool([]) # False
- Diccionarios vacíos → bool({}) # False
- None → bool(None) # False



IMPORTANTE

bool("False") es True, porque la cadena "False" no está vacía.



ERRORES COMUNES

1. Olvidar convertir antes de operar

```
a = "10"
b = "5"
print(a + b)
# '105' (concatena)
# No suma

print(int(a) + int(b))
# 15
```

2. Intentar convertir texto no numérico a número

```
int("Hola")
# ValueError
# no se puede convertir

Verificar antes
.isdigit() o try/except
```

3. Pensar que bool("False") es False

```
bool("False")
# True
# porque no está vacía

Usar comparación:
texto.lower() == "false"
```

4. Perder decimales al usar int()

```
int(3.99)
# 3
# se truncan

Usar float()
si necesitas decimales
```



TIP PRO

Piensa antes de convertir:

¿Qué tipo tiene mi dato ahora? ¿Qué tipo necesito? ¿La conversión tiene sentido?

Menos errores, más código limpio.



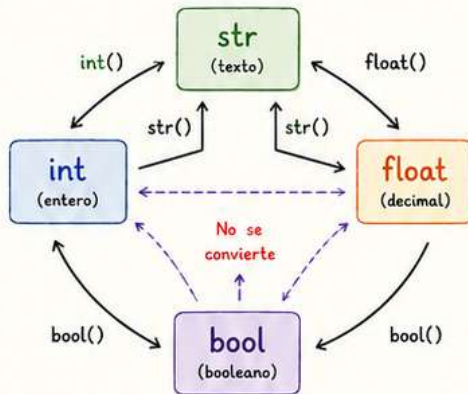


CASTING

CONVERSIÓN DE TIPOS

1. MAPA DE CONVERSIONES

Estos son los caminos más comunes entre tipos usando casting explícito.



💡 None no se puede convertir a otros tipos.

2. CHEAT SHEET RÁPIDO

FUNCIÓN	¿QUÉ HACE?	EJEMPLO	RESULTADO
int()	Convierte a número entero.	<code>int("10")</code> <code>int(3.9)</code> <code>int(True)</code>	10 3 1
float()	Convierte a número decimal.	<code>float("2.5")</code> <code>float(10)</code> <code>float(True)</code>	2.5 10.0 1.0
str()	Convierte a texto (cadena).	<code>str(100)</code> <code>str(3.14)</code> <code>str(True)</code>	"100" "3.14" "True"
bool()	Convierte a booleano (True/False).	<code>bool(1)</code> <code>bool(0)</code> <code>bool("")</code> <code>bool("Hola")</code>	True False False True

★ Usa `type(valor)` para verificar el tipo antes y después.

3. MINI QUIZ

¿Qué devuelve el casting en cada caso?

- `int("42")` → `_____` (int / str / float)
- `float("3.75")` → `_____` (int / float / str)
- `str(99)` → `_____` ("99" / 99 / True)
- `bool([])` → `_____` (True / False)
- `bool("False")` → `_____` (True / False)

Respuestas: 1) 42 2) 3.75 3) "99" 4) False 5) True

4. DESAFÍO RÁPIDO

Escribe las conversiones necesarias para que el código funcione correctamente.

- `texto = "123"`
`numero = _____(texto)` # debe ser int
`print(numero + 10)` # 133
- `precio = "9.99"`
`iva = _____(0.21)` # debe ser float
`print(float(precio) + iva)` # 10.20
- `activo = "1"`
`es_activo = _____(activo)` # debe ser bool
`if es_activo:`
`print("Activo")`

5. PATRONES COMUNES

- ✓ `input()` siempre devuelve `str`. Convierte antes de operar.
- ✓ Convierte a `float` cuando trabajes con dinero o medidas.
- ✓ Usa `bool()` para validar condiciones y entradas del usuario.
- ✓ Convierte a `str` para mostrar valores en mensajes o combinarlos.

EJEMPLO REAL

```
edad_str = input("Edad: ")
edad = int(edad_str)
altura_str = input("Altura: ")
altura = float(altura_str)
print("Gracias!")
print("Tienes", edad, "años")
print("Mides", altura, "metros")
```

6. MENSAJES DE ERROR (ValueError)

Ocurre cuando el valor no puede convertirse al tipo solicitado.

```
int("Hola")
# ValueError: invalid literal for int() with base 10: 'Hola'

float("abc")
# ValueError: could not convert string to float: 'abc'

int(3.14.5)
# SyntaxError: invalid syntax
```

💡 Solución: Asegúrate de que el valor tenga el formato correcto antes de convertirlo.

RESUMEN GENERAL



Casting = convertir un dato de un tipo a otro.



Funciones clave: `int()`, `float()`, `str()`, `bool()`.



Verifica el tipo con `type()`.



No todos los valores se pueden convertir.



Menos errores, más código limpio y eficiente.



ENTRADA Y SALIDA



1. PRINT()



La función `print()` se usa para **mostrar información** en la pantalla (la consola).

Es una de las funciones más básicas y más usadas en Python.

1. ¿QUÉ HACE PRINT()?

Imprime lo que le pasemos entre paréntesis.
Puede ser texto, números, variables
o una combinación de ellos.



```
print("Hola, mundo!") # texto
print(123)             # número
nombre = "Guille"
print(nombre)          # variable
```

SALIDA:

Hola, mundo!
123
Guille

2. MOSTRAR TEXTO

Usa comillas simples (') o dobles (") para mostrar texto.

```
print('Hola Python')
print("Aprender es poder")
```

SALIDA:

Hola Python
Aprender es poder

¡Esto es un texto!

3. MOSTRAR VARIABLES Y NÚMEROS

Puedes mostrar variables, números o ambas sin necesidad de comillas.

```
edad = 25
altura = 1.75
print(edad)
print(altura)
print("Edad:", edad, " | Altura:", altura)
```

SALIDA:

25
1.75
Edad: 25 | Altura: 1.75

4. MOSTRAR VARIOS VALORES

Podemos mostrar varios valores separados por comas. Por defecto, Python agrega un espacio entre ellos.

```
nombre = "Guille"
lenguaje = "Python"
nivel = "Principiante"
print(nombre, lenguaje, nivel)
```

SALIDA:

Guille Python Principiante



¿SABÍAS?

Lo mismo que la línea de la izquierda, pero de forma más personalizada:

```
print("Nombre:", nombre, "- Lenguaje:", lenguaje, "- Nivel:", nivel)
```

SALIDA:

Nombre: Guille - Lenguaje: Python - Nivel: Principiante

5. CARACTERES ESPECIALES

Podemos usar secuencias especiales para dar formato a nuestro texto.

SECUENCIA	SIGNIFICADO	EJEMPLO	SALIDA
<code>\n</code>	Salto de línea	<code>print("Hola\nMundo")</code>	Hola Mundo
<code>\t</code>	Tabulación (espacio)	<code>print("Columna1\tColumna2")</code>	Columna1 Columna2
<code>\"</code>	Comilla doble dentro de texto	<code>print("Él dijo: \"Hola!\")</code>	Él dijo: "Hola!"
<code>\\</code>	Barra invertida	<code>print("Ruta: C:\\Python\\proyectos")</code>	Ruta: C:\Python\proyectos



Estos caracteres hacen que nuestros mensajes se vean más claros y ordenados.





ENTRADA Y SALIDA

2. INPUT()



La función `input()` se usa para recibir datos del usuario a través del teclado.
Siempre devuelve un valor de tipo `str` (texto).

1. ¿QUÉ HACE INPUT()?

Detiene el programa y espera a que el usuario escriba algo y presione Enter.

```
nombre = input("¿Cómo te llamas? ")  
print(nombre)
```

EJECUCIÓN:

¿Cómo te llamas? Guille
Guille ← Lo que escribió el usuario

★ Todo lo que escriba el usuario se guarda como texto (`str`).

2. SIEMPRE DEVUELVE STR

Aunque el usuario escriba números, `input()` los guarda como texto.

```
edad = input("Edad: ")  
print(edad)  
print(type(edad))
```

EJECUCIÓN:

Edad: 25 ← Se escribió 25
25 ← Se muestra 25
<class 'str'> ← Pero es texto (`str`)

💡 Para hacer operaciones matemáticas, debemos convertir el valor a `int` o `float`.



ERROR COMÚN

```
a = input("Número 1: ") # '10'  
b = input("Número 2: ") # '5'  
c = a + b # '10' + '5' = '105'  
print(c) # Resultado: '105'
```

Se concatenan los textos en lugar de sumar.

→ Solución: convertir a `int` o `float`

```
a = int(input("Número 1: "))  
b = int(input("Número 2: "))  
c = a + b  
print(c) # Resultado: 15
```

3. EJEMPLO CON CONVERSIÓN

Convertimos la entrada para poder trabajar con números.

```
edad = int(input("Edad: "))  
altura = float(input("Altura (m): "))  
print("En 10 años tendrás", edad + 10, "años.")  
print("Tu altura es", altura, "metros.")
```

EJECUCIÓN:

Edad: 20
Altura (m): 1.75
En 10 años tendrás 30 años.
Tu altura es 1.75 metros.

Usamos `int()` y `float()` para convertir el texto a números.

4. OTRAS FORMAS ÚTILES

Podemos dar un mensaje, o dejar el `input` sin mensaje.

- Con mensaje:
ciudad = `input("¿En qué ciudad vives? ")`
- Sin mensaje:
dato = `input()` # El usuario escribe sin ver nada
- Con mensaje + salto de línea:
nombre = `input("Nombre:\n")`
`\n` hace un salto de línea antes del `input`
- Usar `input()` dentro de una variable:
color = `input("Color favorito: ").lower()`
`.lower()` para convertir a minúsculas

★ TIP PRO

`input()` es perfecto para crear programas interactivos:

- ✓ Pedir datos al usuario
- ✓ Validar respuestas
- ✓ Personalizar mensajes
- ✓ Tomar decisiones





ENTRADA Y SALIDA

3. PRINT() + INPUT() + CASTING



Combinando `print()`, `input()` y `casting` podemos crear programas que interactúan con el usuario y realizan cálculos.

1. EJEMPLO COMPLETO

```
nombre = input("¿Cómo te llamas? ") # str
edad = int(input("¿Cuántos años tienes? ")) # int
altura = float(input("¿Altura (m)? ")) # float

print("\n¡Hola", nombre + "!")
print("Tienes", edad, "años.")
print("Mides", altura, "metros.")
print("En 10 años tendrás", edad + 10, "años.")
```

SALIDA EJEMPLO:

¿Cómo te llamas? Guille
 ¿Cuántos años tienes? 25
 ¿Altura (m)? 1.75
 ¡Hola Guille!
 Tienes 25 años.
 Mides 1.75 metros.
 En 10 años tendrás 35 años.

2. F-STRINGS (FORMATO MODERNO)

La forma más recomendada para mostrar mensajes combinando texto y variables.

```
nombre = input("Nombre: ")
edad = int(input("Edad: "))

print(f"\n¡Hola {nombre}!")
print(f"Tienes {edad} años.")
print(f"En 10 años tendrás {edad + 10} años.")
```

SALIDA EJEMPLO:

Nombre: Guille
 Edad: 25
 ¡Hola Guille!
 Tienes 25 años.
 En 10 años tendrás 35 años.

Más legible
y fácil de
mantener



3. PARÁMETROS ÚTILES DE PRINT()

sep= (separador)

Define qué se coloca entre los valores.

```
print("Python", "es", "genial")
print("Python", "es", "genial", sep=" - ")
print(10, 20, 30, sep=" | ")
```

SALIDA:

Python es genial
 Python - es - genial
 10 | 20 | 30

Por defecto
sep=" " (espacio)

end= (final)

Define qué se imprime al final (por defecto es salto de línea).

```
print("Hola", end=" ")
print("mundo!")
print("Python", end=" - ")
print("Es calidad")
```

SALIDA:

Hola mundo!
 Python - Es calidad

Por defecto end="\n" (salto de línea)

Ejemplo combinado

Usando `sep` y `end` juntos.

```
print("A", "B", "C", sep="*", end=" -> ")
print("Fin")
```

SALIDA:

A*B*C -> Fin



4. CONCATENACIÓN vs F-STRINGS

Concatenación (forma clásica)

```
nombre = "Guille"
edad = 25
mensaje = "Hola " + nombre + ", tienes " + str(edad) + " años."
print(mensaje)
```

SALIDA:

Hola Guille, tienes 25 años.



F-Strings (forma moderna)

```
nombre = "Guille"
edad = 25
mensaje = f"Hola {nombre}, tienes {edad} años."
print(mensaje)
```

SALIDA:

Hola Guille, tienes 25 años.

¡Más
limpio y
legible!

5. FLUJO BÁSICO DE UN PROGRAMA INTERACTIVO



1. Pedir datos
`input()`
El usuario escribe algo.



2. Procesar datos
(casting + lógica)
Convertimos y realizamos cálculos.



3. Mostrar resultados
`print()`
Mostramos la información al usuario.

RECUERDA

`input()` siempre devuelve `str`. Si necesitas trabajar con números, conviértelos antes de hacer operaciones.

¡Convierte, calcula y muestra!





ENTRADA Y SALIDA

Volumen 4



4. RESUMEN GENERAL

1. PRINT() - MOSTRAR

- ✓ Muestra información en la consola.
- ✓ Puede mostrar texto, números, variables o combinaciones.
- ✓ Usa comas para separar valores.
- ✓ Soporta caracteres especiales como `\n`, `\t`, etc.

```
print("Hola")
x = 10
print("Valor:", x)
```

SALIDA:
Hola
Valor: 10

2. INPUT() - RECIBIR

- ✓ Recibe datos del usuario por teclado.
- ✓ Siempre devuelve tipo `str` (texto).
- ✓ Ideal para programas interactivos.

```
nombre = input("Nombre: ")
edad = input("Edad: ")
print(nombre)
print(edad)
```

SALIDA EJEMPLO:
Nombre: Guille
Edad: 25
Guille
25

Aunque escribas un número en `input()`, Python lo guarda como `str`.

3. CASTING - CONVERTIR

- ✓ Para trabajar con números, debemos convertir el resultado de `input()`.
- ✓ Usamos `int()` o `float()`.

`str` → `int`

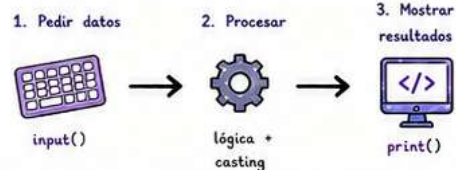
```
edad = int(input("Edad: "))
altura = float(input("Altura: "))
print(edad + 10)
print(altura + 2)
```

SALIDA EJEMPLO:
Edad: 20
Altura: 1.75
30
3.5

CHEAT SHEET RÁPIDO

FUNCIÓN	¿QUÉ HACE?	EJEMPLO	DEVUELVE
<code>print()</code>	Muestra información por pantalla.	<pre>print("Hola") print(10)</pre>	No devuelve nada (muestra)
<code>input()</code>	Recibe información del usuario.	<code>dato = input("Dato: ")</code>	<code>str</code> (texto)
<code>int()</code>	Convierte a número entero.	<code>int("25")</code>	<code>int</code>
<code>float()</code>	Convierte a número decimal.	<code>float("3.14")</code>	<code>float</code>
<code>str()</code>	Convierte a texto.	<code>str(100)</code>	<code>str</code>

FLUJO DE UN PROGRAMA INTERACTIVO



TIP PRO

Usa f-strings para mensajes más claros y fáciles de leer.

```
nombre = "Guille"
edad = 25
print(f"Hola {nombre}, tienes {edad} años.")
```



ERRORES COMUNES

- ✗ Olvidar convertir `input()` a `int` o `float` y tratar de sumar.

```
edad = input("Edad: ")
edad + 5 # ❌ Error
```

Solución:

```
edad = int(input("Edad: "))
edad + 5 # ✅ Correcto
```

- ✗ Pensar que `input()` devuelve un número.

```
type(input("Edad: ")) # <class 'str'>
```

- ✗ Usar `+` para unir texto y número sin convertir.

```
"Tienes " + 25 + " años"
```

Solución:

```
"Tienes " + str(25) + " años" # ✅
```

- ✗ Olvidar comillas en el texto.

```
print(Hola mundo) # ❌ Error
```

```
print("Hola mundo") # ✅
```



DESAFÍO PRÁCTICO

Crea un programa que:

1. Pida al usuario su nombre.
2. Pida su edad.
3. Muestre un mensaje personalizado indicando su nombre y cuántos años tendrá en 5 años.

Ejemplo de salida esperada:
Nombre: Guille
Edad: 25
Hola Guille, en 5 años tendrás 30 años.

```
nombre = input("Nombre: ")
edad = int(input("Edad: "))
futuro = edad + 5
print("Hola", nombre + ", en 5 años tendrás", futuro, "años.")
# (o usa f-strings 😊)
```





OPERADORES

1. OPERADORES ARITMÉTICOS



Los operadores aritméticos se usan para realizar cálculos matemáticos con números.

Pueden usarse con `int` (enteros) y `float` (decimales).

¿CUÁLES SON?

OPERADOR	NOMBRE	DESCRIPCIÓN	EJEMPLO	RESULTADO
+	Suma	Suma dos valores.	<code>10 + 5</code>	15
-	Resta	Resta el segundo valor al primero.	<code>10 - 5</code>	5
*	Multiplicación	Multiplica dos valores.	<code>10 * 5</code>	50
/	División	Divide el primero por el segundo (resultado decimal).	<code>10 / 4</code>	2.5
%	Módulo	Devuelve el resto de la división entera.	<code>10 % 3</code>	1
**	Potencia	Eleva el primero a la potencia del segundo.	<code>2 ** 3</code>	8
//	División entera	Divide y devuelve solo la parte entera.	<code>10 // 3</code>	3

EJEMPLOS EN CÓDIGO

```
a = 10
b = 3

# Suma
print(a + b)    # 13

# Resta
print(a - b)    # 7

# Multiplicación
print(a * b)    # 30

# División
print(a / b)    # 3.3333333333
```

```
# Módulo (resto de la división)
print(a % b)    # 1

# Potencia
print(2 ** 3)   # 8

# División entera
print(a // b)   # 3
```

Usa `//` cuando quieras el resultado sin decimales (parte entera).



TIP PRO

Recuerda la jerarquía (de mayor a menor prioridad):

`( )` → `**` → `*` / `/` `//` `%` → `+` `-`

Los paréntesis siempre se evalúan primero.





OPERADORES

2. OPERADORES DE COMPARACIÓN



Los operadores de comparación se usan para comparar dos valores. Siempre devuelven un resultado de tipo **bool**: **True** o **False**

¿CUÁLES SON?

OPERADOR	NOMBRE	DESCRIPCIÓN	EJEMPLO	RESULTADO
==	Igual a	Verifica si dos valores son iguales.	5 == 5	True
!=	Diferente de	Verifica si dos valores son distintos.	5 != 3	True
>	Mayor que	Verifica si el valor de la izquierda es mayor que el de la derecha.	10 > 5	True
<	Menor que	Verifica si el valor de la izquierda es menor que el de la derecha.	3 < 8	True
>=	Mayor o igual que	Verifica si el valor de la izquierda es mayor o igual que el de la derecha.	5 >= 5	True
<=	Menor o igual que	Verifica si el valor de la izquierda es menor o igual que el de la derecha.	3 <= 3	True

EJEMPLOS EN CÓDIGO

```
a = 10
b = 20

print(a == b)    # False
print(a != b)    # True
print(a > b)     # False
print(a < b)     # True
print(a >= 10)   # True
print(b <= 20)   # True
```

SALIDA:

```
False
True
False
True
True
True
```

Los operadores de comparación son muy usados en los condicionales (if), que veremos en el próximo volumen.



ERRORES COMÚNES

✗ Usar **=** en lugar de **==** para comparar.

```
if edad = 18:    # Error ✗
```

Solución:

```
if edad == 18:   # Correcto ✓
```

✗ Comparar valores de distinto tipo.

```
"5" == 5         # False ✗
```

Solución: convierte el tipo con **int()** o **str()** según corresponda.

MINI QUIZ

Elige la opción correcta.

- ¿Qué resultado devuelve **7 > 3**?
a) False **b) True** c) 3
- ¿Qué resultado devuelve **5 == "5"**?
a) True **b) False** c) Error
- ¿Qué operador usamos para "mayor o igual que"?
a) > **b) >=** c) =>

Respuestas: 1) b 2) b 3) b



TIP PRO

Podés encadenar comparaciones para mayor claridad:

```
18 <= edad < 65    # True si edad está entre 18 y 64
```





OPERADORES

3. OPERADORES LÓGICOS



Los operadores lógicos se usan para combinar condiciones.

Siempre devuelven un resultado de tipo **bool**: **True** o **False**

¿CUÁLES SON?

OPERADOR	NOMBRE	DESCRIPCIÓN	EJEMPLO	RESULTADO
and	Y lógico (AND)	Devuelve True solo si ambas condiciones son True.	$5 > 3$ and $2 < 4$	True
or	O lógico (OR)	Devuelve True si al menos una de las condiciones es True.	$5 > 10$ or $2 < 4$	True
not	No lógico (NOT)	Invierte el valor de la condición. Si es True → False Si es False → True.	not $(5 > 10)$	True

EJEMPLOS EN CÓDIGO

```
a = 5
b = 10
c = 15
```

```
# AND lógico
print(a < b and b < c) # True
print(a > b and b < c) # False
```

```
# OR lógico
print(a > b or b < c) # True
print(a > b or b > c) # False
```

```
# NOT lógico
print(not (a > b)) # True
print(not (a > c)) # False
```

EJEMPLOS EN LA VIDA REAL

• AND lógico (ambas condiciones)

Puedo salir de casa si está sin lluvia
Y tengo tiempo.



• OR lógico (al menos una)

Llevo paraguas si llueve O si hay
probabilidad de lluvia.



• NOT lógico (invierte)

No entraré si no estoy autorizado.

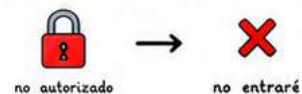


TABLA DE VERDAD

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True



NOTAS IMPORTANTES

- ✓ Las condiciones entre paréntesis se evalúan primero.
- ✓ True y False en Python siempre comienzan con mayúscula.
- ✓ Los operadores lógicos son clave en los condicionales (if).

★ TIP PRO

Podés combinar varios operadores lógicos:

```
(a > 0 and b > 0) or c == 0
```



Usá paréntesis para que tu código sea más claro.



DESAFÍO PRÁCTICO

Escribe una expresión que sea True si:

- x es mayor que 10 Y menor que 20. (Usá and)
- y es menor que 5 O igual a 10. (Usá or)
- z NO es igual a 0. (Usá not)

```
x = 12
y = 10
z = 2
```

```
# Tu expresión aquí
```





OPERADORES

Volumen 5



4. RESUMEN GENERAL

1. ARITMÉTICOS

Realizan cálculos.

- +** Suma
- Resta
- *** Multiplicación
- /** División
- %** Módulo (resto)
- **** Potencia
- //** División entera

Ejemplo rápido:

```
10 + 5 = 15
10 / 4 = 2.5
10 % 3 = 1
2 ** 3 = 8
10 // 3 = 3
```

2. COMPARACIÓN

Comparan valores.

- ==** Igual a
- !=** Diferente de
- >** Mayor que
- <** Menor que
- >=** Mayor o igual que
- <=** Menor o igual que

Ejemplo rápido:

```
5 == 5 → True
7 != 3 → True
10 > 4 → True
2 <= 2 → True
3 < 1 → False
```

3. LÓGICOS

Combinan condiciones.

- and** Y lógico
True solo si ambas son True.
- or** O lógico
True si al menos una es True.
- not** No lógico
Invierte la condición.

Ejemplo rápido:

```
(5 > 3) and (2 < 4) → True
(5 > 10) or (2 < 4) → True
not (5 > 10) → True
```

4. PRIORIDAD (de mayor a menor)

- 1) Paréntesis ()
- 2) ** Potencia
- 3) * / // %
- 4) + -
- 5) Comparación
== != > < >= <=
- 6) not
- 7) and
- 8) or

Paréntesis primero, siempre.



ERRORES COMUNES

- ✗ Usar **=** en lugar de **==** para comparar.

```
if edad = 18: # Error ✗
```

Solución:

```
if edad == 18: # Correcto ✓
```

- ✗ Olvidar que **and** / **or** usan lógica booleana.

```
5 > 3 and 2 < 4 # True (no 1)
True and False # False
```

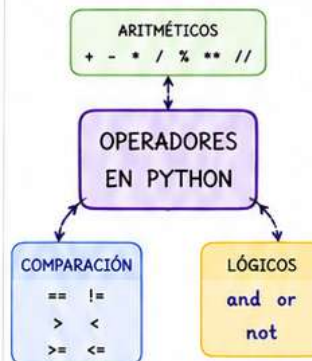
- ✗ No usar paréntesis en expresiones complejas.

```
# Sin paréntesis (puede dar resultado inesperado)
a > b and c > d or e > f
# Con paréntesis (más claro)
(a > b and c > d) or (e > f)
```

- ✗ Usar **not** dos veces sin querer.

```
not not True # True (se cancela)
```

MAPA MENTAL



★ Los operadores son la base para tomar decisiones y repetir acciones en nuestros programas.

CHEAT SHEET RÁPIDO

Aritméticos

+ Suma	Ej: 5 + 2 = 7
- Resta	Ej: 5 - 2 = 3
* Multiplicación	Ej: 5 * 2 = 10
/ División	Ej: 5 / 2 = 2.5
% Módulo	Ej: 5 % 2 = 1
** Potencia	Ej: 2 ** 3 = 8
// División entera	Ej: 5 // 2 = 2

Comparación

== Igual a	} Devuelven True o False
!= Diferente de	
> Mayor que	
< Menor que	
>= Mayor o igual que	
<= Menor o igual que	

Lógicos

and Y lógico	} Devuelven True o False
or O lógico	
not No lógico (niega)	

MINI QUIZ

Elige la opción correcta.

1. ¿Qué resultado da 10 % 4?
a) 2 b) 4 c) 6
2. ¿Qué devuelve 7 > 3 and 2 < 1?
a) True b) False c) 7
3. ¿Qué operador usamos para "no es igual"?
a) = b) != c) <>
4. ¿Qué devuelve not (3 > 5)?
a) True b) False c) 3

Respuestas: 1) a 2) b 3) b 4) a



DESAFÍO PRÁCTICO

Escribe una expresión booleana para cada caso.

1. Edad entre 18 y 30 años (inclusive).
(usa: edad)
2. El número es par.
(usa: n)
3. Está lloviendo o hace viento, pero no ambos.
(usa: lluvia, viento)

TIP PRO

Cuando tengas dudas, usa paréntesis y descompón la expresión en pasos más simples.

```
# Ejemplo complejo
resultado = (a > b and c < d) or not (e == f)

# Paso 1
x = a > b and c < d # True o False

# Paso 2
y = not (e == f) # True o False

# Paso 3
resultado = x or y # Más claro ✓
```





CONDICIONALES

1. IF (INTRODUCCIÓN)



Los condicionales nos permiten tomar decisiones en nuestro programa.
El `if` ejecuta un bloque de código solo si la condición es `True`.

¿QUÉ ES UN CONDICIONAL?

Un condicional evalúa una condición y decide si ejecuta o no un bloque de código.

Si la condición es `True`,
el código se ejecuta.

Si es `False`, se salta ese bloque.



SI



Permitir
acceso



NO



No permitir
acceso



SINTAXIS BÁSICA

`if` condición:

Bloque de código

(se ejecuta si la condición es `True`)

IMPORTANTE



En Python, la indentación (sangría) es **OBLIGATORIA** para definir qué instrucciones pertenecen al bloque del `if`.

OPERADORES DE COMPARACIÓN

Operador	Significado	Ejemplo	Resultado
<code>==</code>	igual a	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	diferente de	<code>5 != 3</code>	<code>True</code>
<code>></code>	mayor que	<code>10 > 4</code>	<code>True</code>
<code><</code>	menor que	<code>3 < 7</code>	<code>True</code>
<code>>=</code>	mayor o igual que	<code>5 >= 5</code>	<code>True</code>
<code><=</code>	menor o igual que	<code>2 <= 1</code>	<code>False</code>

EJEMPLOS RÁPIDOS

`x = 10`

`if x > 5:`

`print("x es mayor que 5")`

SALIDA:

x es mayor que 5

`edad = 16`

`if edad >= 18:`

`print("Eres mayor de edad")`

SALIDA:

(No imprime nada)
porque la condición es `False`

EJEMPLO COMPLETO

`edad = 20`

`if edad >= 18:`

`print("Puedes entrar al evento 🎉")`

Este código solo se ejecuta si la condición es `True`

`print("Fin del programa")` # Se ejecuta siempre

PASO A PASO

- 1 Se evalúa la condición: `20 >= 18` → `True`
- 2 Como es `True`, se ejecuta el bloque indentado.
- 3 Después, se ejecuta el resto del código.

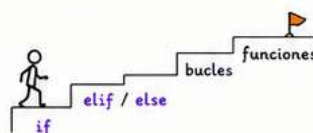
SALIDA:

Puedes entrar al evento 🎉
Fin del programa



TIP PRO

Empieza siempre probando condiciones simples.
Comprender el `if` básico es la base para estructuras más avanzadas como `elif`, `else` y bucles.



¡Paso a paso
llegas lejos! 🚀





CONDICIONALES

2. IF - ELSE (DOS POSIBILIDADES)



Cuando queremos que nuestro programa tome una de dos decisiones posibles, usamos **if** junto con **else**.

Si la condición es **True** → bloque **if**

Si la condición es **False** → bloque **else**

SINTAXIS

if condición:

bloque de código si es **True**

else:

bloque de código si es **False**



RECUERDA:

Ambos bloques (**if** y **else**) son excluyentes, solo uno se ejecuta.

EJEMPLO BÁSICO

```
edad = 18
if edad >= 18:
    print("Puedes entrar, eres mayor de edad ✅")
else:
    print("No puedes entrar, eres menor de edad ❌")
```

SALIDA 1:

Puedes entrar, eres mayor de edad ✅

```
edad = 15
if edad >= 18:
    print("Puedes entrar, eres mayor de edad ✅")
else:
    print("No puedes entrar, eres menor de edad ❌")
```

SALIDA 2:

No puedes entrar, eres menor de edad ❌

EJEMPLOS EN LA VIDA REAL

SITUACIÓN	CONDICIÓN	IF (TRUE)	ELSE (FALSE)	EJEMPLO EN CÓDIGO
Mayor de edad	edad >= 18	Puede entrar	No puede entrar	<pre>if edad >= 18: print("Puedes entrar") else: print("No puedes entrar")</pre>
Stock disponible	stock > 0	Producto disponible	Producto agotado	<pre>if stock > 0: print("¡Disponible!") else: print("Agotado")</pre>
Contraseña correcta	contraseña == clave	Acceso concedido	Acceso denegado	<pre>if contraseña == clave: print("Acceso concedido") else: print("Acceso denegado")</pre>
¿Llueve o no?	llueve == True	Llevar paraguas	No llevar paraguas	<pre>if llueve: print("Lleva paraguas ☔") else: print("No lleva paraguas ☀️")</pre>



ERRORES COMÚNES

❌ Olvidar los dos puntos (:) después de **if** o **else**

```
if edad >= 18 # Error ❌
```

❌ Indentación incorrecta (sangría)

```
if edad >= 18:
    print("Oops") # Mal indentado ❌
```

❌ Usar **=** en lugar de **==** para comparar

```
if edad = 18: # Error ❌
```

Solución: usar **==**

```
if edad == 18: # Correcto ✅
```

MINI QUIZ

Elige la opción correcta.

1. ¿Qué imprime el siguiente código si **x = 10**?

```
if x > 5:
    print("A")
else:
    print("B")
```

a) A b) B c) Error

2. ¿Qué imprime si **x = 3**?

a) A b) B c) Nada

3. ¿Qué se ejecuta si **x = 7**?

a) Ambos bloques b) Solo el **if**
c) Solo el **else**

Respuestas: 1) a 2) b 3) b



TIP PRO

Pensá siempre:

¿Qué condición quiero evaluar?
¿Qué hago si es **True**?
¿Qué hago si es **False**?

Escribir las dos respuestas te ayuda a organizar mejor tu código.





CONDICIONALES

3. ELIF (MÚLTIPLES CONDICIONES)

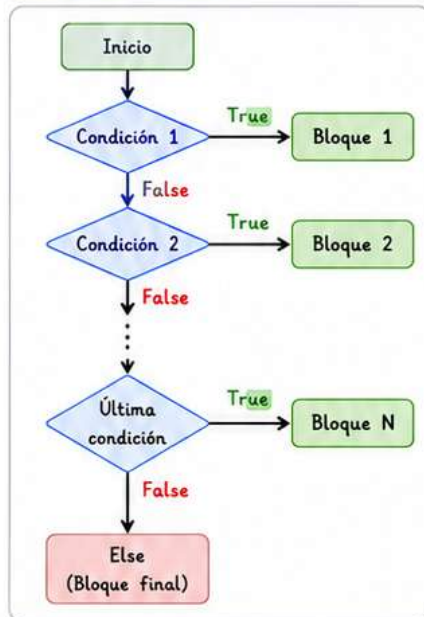


Cuando tenemos más de dos posibilidades, usamos `elif` (abreviatura de "else if").

Se evalúa cada condición en orden hasta encontrar la primera `True`.



¿CÓMO FUNCIONA?



SINTAXIS

```

if condicion_1:
    # bloque si condicion_1 es True
elif condicion_2:
    # bloque si condicion_2 es True
elif condicion_3:
    # bloque si condicion_3 es True
...
else:
    # bloque si todas las condiciones son False
  
```

EJEMPLO BÁSICO

```

nota = 8
if nota >= 9:
    print("Excelente 🏆")
elif nota >= 7:
    print("Aprobado 👍")
elif nota >= 4:
    print("Suficiente 😊")
else:
    print("Desaprobado ❌")
  
```

SALIDA POSIBLE:

Aprobado 👍

Se ejecuta el bloque de la primera condición que sea `True` y luego se salta el resto. ★

EJEMPLOS EN LA VIDA REAL

SITUACIÓN	CONDICIONES
Nivel en un juego	Puntos >= 1000 → Nivel experto elif Puntos >= 500 → Nivel intermedio elif Puntos >= 100 → Nivel principiante else → Sin nivel
Tamaño de pizza	if tamaño == "grande" elif tamaño == "mediana" elif tamaño == "chica" else → Tamaño no válido
Temperatura	if temp > 30 → Hace calor elif temp >= 15 → Clima agradable elif temp >= 0 → Hace frío else → Hace mucho frío ❄️

EJEMPLO COMPLETO

```

temp = 12
if temp > 30:
    print("Hace calor ☀️")
elif temp >= 15:
    print("Clima agradable 😊")
elif temp >= 0:
    print("Hace frío 🌨️")
else:
    print("Hace mucho frío ❄️")
  
```

SALIDA:

Hace frío 🌨️



ERRORES COMÚNES

- ❌ Olvidar el `:` al final de `if`, `elif` o `else`
- ❌ Indentación incorrecta (Python usa sangría)
- ❌ Poner `else` en medio de un `elif`
- ❌ Escribir `=` en lugar de `==` para comparar

MINI QUIZ

¿Qué se imprimirá?

```

x = 7
if x > 10:
    print("A")
print("A")
elif x = 5:
    print("C")
else:
    print("D")
  
```

- a) A b) B c) C d) D



★ TIP PRO

Usa `elif` cuando tengas varias condiciones que son excluyentes entre sí. Es más claro y eficiente que usar muchos `if`.





CONDICIONALES

4. RESUMEN GENERAL

Los condicionales nos permiten tomar decisiones y ejecutar diferentes bloques de código según se cumplan las condiciones.

Clave: **comparar** → **decidir** → **ejecutar**



CHEAT SHEET RÁPIDO

Estructura	Cuándo usarla	Ejemplo
<code>if</code>	Una sola condición	<pre>if edad >= 18: print("Mayor de edad")</pre>
<code>if - else</code>	Dos posibilidades	<pre>if stock > 0: print("Disponible") else: print("Agotado")</pre>
<code>if - elif - else</code>	Tres o más posibilidades	<pre>if nota >= 9: print("Excelente") elif nota >= 7: print("Aprobado") else: print("Desaprobado")</pre>

ESQUEMA GENERAL

```
if condición_1:
    # bloque si es True
elif condición_2:
    # bloque si es True
elif condición_3:
    # bloque si es True
...
else:
    # bloque si todas son False
```

Se ejecuta si la 1ª es True

Se evalúan en orden

Solo si todas fueron False



Python ejecuta solo un bloque: el primero que sea True.



ERRORES COMUNES

✗ Usar `=` en lugar de `==`

```
if edad = 18: # Error
```

Solución: `if edad == 18:` ✓

✗ Olvidar los dos puntos :

```
if edad >= 18 # Error
```

Solución: `if edad >= 18:` ✓

✗ Indentación incorrecta

```
if edad >= 18:
    print("Mayor de edad")
else:
    print("Menor de edad") # Error
```

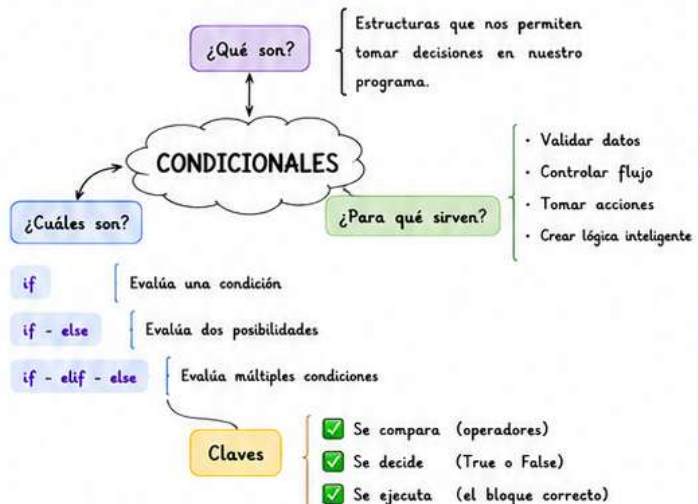
Solución: usar la misma sangría en cada bloque. ✓

✗ Escribir `else` o `elif` sin un `if` previo

```
else:
    print("Hola") # Error
```

Solución: siempre debe haber un `if` antes. ✓

MAPA MENTAL



MINI QUIZ

Elege la ejecuta "oil" la condición es True?

1. a) else b) if c) elif d) ninguna

2. ¿Qué palabra usamos para evaluar varias condiciones?

a) switch b) case c) elif d) choose

3. ¿Qué se ejecuta si todas las condiciones son False?

a) if b) elif c) else d) None

Respuestas: 1) b 2) c 3) c



DESAFÍO PRÁCTICO

Escribe un programa que clasifique una persona según su edad.

- ✓ Si es menor de 12 → "Niño"
- ✓ Si tiene entre 12 y 17 → "Adolescente"
- ✓ Si tiene entre 18 y 59 → "Adulto"
- ✓ Si tiene 60 o más → "Adulto mayor"

Muestra el resultado en pantalla.



TIP PRO

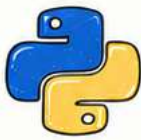


Usa comparaciones claras y ordena tus condiciones de mayor a menor importancia.

Así tu código será más legible y fácil de mantener.

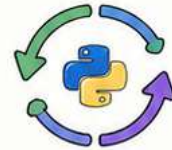
Piensa como el programa, estructura como un humano.





BUCLES (LOOPS)

1. INTRODUCCIÓN A LOS BUCLES



Los bucles (loops) nos permiten repetir un bloque de código varias veces sin tener que escribirlo una y otra vez.

Son esenciales para automatizar tareas y procesar datos.

Clave: **repetir** → **ahorrar tiempo** y escribir **código más limpio**.

¿POR QUÉ USAR BUCLES?

AHORRAN TIEMPO



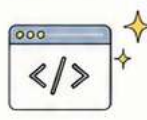
Evitan repetir muchas líneas de código.

MENOS ERRORES



Menos código repetido = menos posibilidades de equivocarse.

CÓDIGO LIMPIO



Más legible, organizado y fácil de mantener.

FLEXIBILIDAD

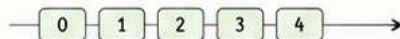


Puedes repetir algo 0, 1 o muchas veces según necesites.

TIPOS DE BUCLES EN PYTHON

for (para)

Se usa para repetir un bloque de código un número conocido de veces.
Ideal para recorrer secuencias (listas, rangos, strings, etc.).



```
for i in range(5):  
    print(i)
```

SALIDA:

0
1
2
3
4

while (mientras)

Se usa para repetir un bloque de código mientras una condición sea verdadera.
Ideal cuando no sabemos cuántas veces se repetirá.

```
contador = 0  
while contador < 5:  
    print(contador)  
    contador += 1
```

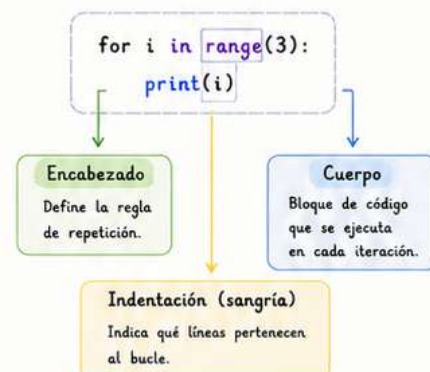
SALIDA:

0
1
2
3
4

COMPARACIÓN RÁPIDA

Característica	for	while
Cuándo usar	Cuando sabes cuántas veces se repetirá	Cuando no sabes cuántas veces se repetirá
Control	Automático (Python se encarga del conteo)	Manual (tú controlas la condición)
Ejemplo típico	Recorrer una lista o un rango	Esperar entrada del usuario, validar datos, menús, etc.
Riesgo	Bajo (hay un fin definido)	Alto si la condición nunca cambia (bucle infinito)

PARTES DE UN BUCLE



EJEMPLO DE LA VIDA REAL



Cuando vas al supermercado, repites la acción de "agregar cada producto" hasta terminar la lista.

Eso es un **bucle** en la vida real. ¡Python hace lo mismo!

★ TIP PRO

Siempre revisa la condición de salida en un **while** para evitar bucles infinitos.

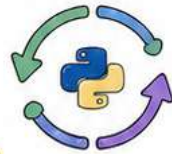
Usa **for** cuando puedas: es más seguro y claro.





BUCLES (LOOPS)

Volumen 7



El bucle `for` se usa para recorrer una secuencia (lista, tupla, string, rango, etc.) y ejecutar un bloque de código en cada elemento.

Ideal cuando **sabes cuántas veces** se repetirá.

SINTAXIS BÁSICA

```
for variable in secuencia:  
    # bloque de código
```

variable → toma cada valor de la secuencia

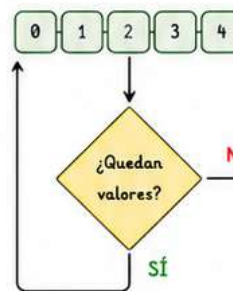
in → significa "en"

secuencia → puede ser lista, string, range(), etc.

¿CÓMO FUNCIONA?

La variable toma el primer valor de la secuencia

La variable toma el siguiente valor y se repite



Se ejecuta el bloque de código

EJEMPLOS PRÁCTICOS

1. Recorrer un rango

```
for i in range(5):  
    print(i)
```

SALIDA:

0
1
2
3
4

range(5) genera números desde 0 hasta 4. (El 5 no se incluye)

2. Recorrer una lista

```
frutas = ["manzana", "pera", "uva"]  
  
for fruta in frutas:  
    print(fruta)
```

SALIDA:

manzana
pera
uva

3. Recorrer un string

```
nombre = "Python"  
  
for letra in nombre:  
    print(letra)
```

SALIDA:

P
y
t
h
o
n

Un string es una secuencia de caracteres.

FUNCIÓN RANGE()

`range(inicio, fin, paso)`

Genera una secuencia de números.

- `range(5)` → 0, 1, 2, 3, 4
- `range(2, 6)` → 2, 3, 4, 5
- `range(1, 10, 2)` → 1, 3, 5, 7, 9
- `range(10, 0, -2)` → 10, 8, 6, 4, 2

```
for i in range(2, 6, 2):  
    print(i)
```

SALIDA:

2
4



NOTA

El paso puede ser positivo (hacia adelante) o negativo (hacia atrás).



ERRORES COMÚNES

- ✗ Olvidar los dos puntos (:)
- ✗ Mala indentación en el bloque
- ✗ Usar `=` en vez de `in`
- ✗ Modificar la lista mientras se itera

```
for i in range(5) # sin ":"  
    print(i)
```

MINI QUIZ



¿Qué imprime el siguiente código?

```
for i in range(1, 6, 2):  
    print(i)
```

- a) 1 2 3 4 5 b) 1 3 5
c) 0 2 4 d) 2 4 6

Respuesta: b) 1 3 5

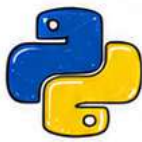
TIP PRO



Usa `for` cuando conozcas de antemano la cantidad de elementos a recorrer.

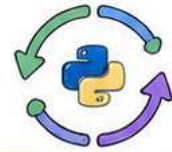
Ejemplo: recorrer todos los productos de una lista, todas las letras de una palabra, todos los días de la semana, etc.





BUCLES (LOOPS)

3. BUCLE WHILE (MIENTRAS)



El bucle **while** se usa cuando no sabemos cuántas veces se repetirá el bloque.

Se ejecuta **mientras** la condición sea **True**.

Clave: la condición debe cambiar en algún momento, o el bucle será **infinito**.

SINTAXIS BÁSICA

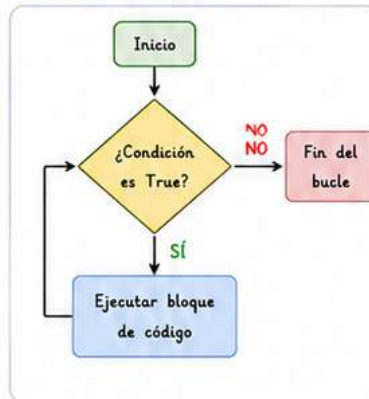
```
while condicion:  
    # bloque de código
```

condicion → expresión que se evalúa

True → se ejecuta el bloque

False → se sale del bucle

¿CÓMO FUNCIONA?



EJEMPLO BÁSICO

```
contador = 0  
while contador < 5:  
    print(contador)  
    contador += 1
```

SALIDA:

0
1
2
3
4

La condición cambia en cada iteración, por eso el bucle termina.

EJEMPLOS EN LA VIDA REAL

1. Intentos de contraseña

El usuario intenta hasta ingresar la contraseña correcta.

```
contrasena = "1234"  
intento = ""  
while intento != contrasena:  
    intento = input("Clave: ")  
    print("Acceso concedido ✅")
```

2. Contar regresivo

Desde 5 hasta 1.

```
n = 5  
while n > 0:  
    print(n)  
    n -= 1  
    print("¡Despegue! 🚀")
```

3. Acumular una suma

Sumar números hasta que el usuario escriba 0.

```
total = 0  
numero = int(input("Número: "))  
while numero != 0:  
    total += numero  
    numero = int(input("Número: "))  
    print("Total:", total)
```

WHILE vs FOR

Característica	while	for
Uso	No sabemos cuántas veces se repetirá.	Sabemos cuántas veces se repetirá.
Control	Manual (cambiamos la condición).	Automático (Python lo hace).
Ejemplo	Leer datos hasta encontrar un valor específico.	Recorrer una lista o rango conocido.
Riesgo	Bucle infinito si la condición nunca cambia.	No hay riesgo de infinito.

BUCLE INFINITO

⚠️ Ocurre cuando la condición siempre es **True**.

```
while True:  
    print("Esto nunca se detiene")
```



Para detenerlo (en la mayoría de entornos):

Presiona **Ctrl** + **C** en la consola.



ERRORES COMÚNES

- ❌ Olvidar cambiar la condición (bucle infinito).
- ❌ Usar **=** en lugar de **==** en la condición.
- ❌ Mala indentación en el bloque.

```
contador = 0  
while contador < 5:  
    print(contador) # mala indentación  
    contador += 1
```

MINI QUIZ

¿Qué imprime el siguiente código?

```
i = 3  
while i > 0:  
    print(i)  
    i -= 1
```

- a) 3 2 1 b) 1 2 3
c) 3 2 1 0 d) 0 1 2

Respuesta: a) 3 2 1

TIP PRO

Usa **for** siempre que sepas cuántas veces debes repetir.

Usa **while** cuando dependas de una condición que puede cambiar.

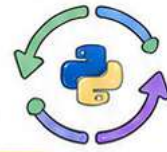
Piensa: **for** = recorrido conocido
mientras que **while** = condición que controla el bucle.





BUCLES (LOOPS)

Volumen 7



4. BUENAS PRÁCTICAS Y CASOS ESPECIALES



Los bucles son muy poderosos, pero usarlos bien marca la diferencia.
Lee siempre tu condición y asegúrate de que el bucle **tenga un fin**.

CONTROL DE FLUJO EN BUCLES

Palabra clave	Uso	Ejemplo
break	Termina el bucle antes de tiempo.	<pre>for i in range(10): if i == 5: break # se detiene en 5</pre>
continue	Salta a la siguiente iteración.	<pre>for i in range(5): if i == 2: continue print(i) # imprime 0, 1, 3, 4</pre>
else	Se ejecuta cuando el bucle termina normalmente (sin break).	<pre>for i in range(3): print(i) else: print("Fin del bucle")</pre>

BUCLE ANIDADO

Un bucle dentro de otro.

```
for i in range(3):  
    for j in range(2):  
        print(f"i={i}, j={j}")
```

SALIDA:

```
i=0, j=0  
i=0, j=1  
i=1, j=0  
i=1, j=1  
i=2, j=0  
i=2, j=1
```

Útil para trabajar con matrices, tablas, patrones, juegos, etc.



CONSEJOS IMPORTANTES

- ✓ Asegúrate de que la **condición cambie** en cada iteración.
- ✓ Evita bucles **infinitos**.
- ✓ Usa nombres de variables claros y significativos.
- ✓ Piensa antes de escribir: ¿cuántas veces quiero repetir?
- ✓ Prueba con ejemplos pequeños primero.



CASOS ÚTILES EN LA VIDA REAL

Situación	Cómo usar bucles
Procesar una lista de tareas	→ Recorrer la lista con for y tratar cada tarea.
Calcular el total de una compra	→ Usar un bucle para sumar cada producto.
Esperar un evento (contador)	→ Usar while para contar el tiempo hasta llegar.
Juegos y niveles	→ Repetir acciones, enemigos, movimientos, etc.

RESUMEN GENERAL



★ CHEAT SHEET RÁPIDO

FOR

for variable in secuencia:
bloque de código

Ejemplo:

```
for i in range(3):  
    print(i)
```

WHILE

while condición:
bloque de código

Ejemplo:

```
x = 0  
while x < 3:  
    print(x)  
    x += 1
```

⚠ ERRORES COMÚNES

- ✗ Olvidar cambiar la condición en **while**.
- ✗ Usar **=** en lugar de **==** en la condición.
- ✗ Indentación incorrecta.
- ✗ No pensar en cómo termina el bucle.

```
while x < 5  
    print(x)  
x == 1
```

→ Falta indentación ✗

MINI QUIZ

¿Qué imprime el siguiente código?

```
i = 0  
while i < 3:  
    print(i)  
    i += 1
```

- a) 1 2 3 b) 0 1 2
c) 0 1 2 3 d) 1 2

Respuesta: **b) 0 1 2**



DESAFÍO

Escribe un programa que imprima la siguiente cuenta regresiva:

```
5  
4  
3  
2  
1
```

¡Despegue! 🚀

TIP PRO



Los bucles son la base de muchos programas. Domínalos y podrás automatizar casi todo.

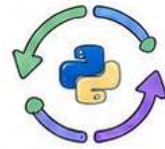
Práctica diaria = progreso real 🌟





BUCLES (LOOPS)

Volumen 7



5. CASOS AVANZADOS Y PATRONES ÚTILES



Cuando entiendes bien los bucles, puedes resolver problemas más complejos de forma elegante y eficiente.

Aquí van patrones y técnicas que te serán muy útiles.



1. Acumulador (patrón clásico)

Usar un bucle para acumular un valor en cada iteración.

```
total = 0
for n in range(1, 6):
    total += n
print(total) # 15
```

Muy útil para sumas, promedios, contadores, etc.

2. Búsqueda en una lista

Buscar un elemento y salir del bucle cuando se encuentra.

```
numeros = [4, 7, 1, 9, 3]
buscado = 7
encontrado = False

for n in numeros:
    if n == buscado:
        encontrado = True
        break

print(encontrado) # True
```

Usa break para salir en cuanto encuentres lo que buscas.

3. Contador condicional

Contar cuántos elementos cumplen cierta condición.

```
pares = 0
for n in range(1, 11):
    if n % 2 == 0:
        pares += 1
print(pares) # 5
```

Ideal para estadísticas y análisis de datos.

4. BUCLES ANIDADOS (NESTED LOOPS)

Un bucle dentro de otro. Permiten trabajar con matrices, tablas, combinaciones, patrones, etc.

Ejemplo: imprimir una cuadrícula 3x4

```
for i in range(3): # filas
    for j in range(4): # columnas
        print(f"({i},{j})", end=" ")
    print()
```

SALIDA:

```
(0,0) (0,1) (0,2) (0,3)
(1,0) (1,1) (1,2) (1,3)
(2,0) (2,1) (2,2) (2,3)
```

5. IMPRIMIR PATRONES

Los bucles son perfectos para dibujar patrones en la consola.

Ejemplo: triángulo rectángulo

```
for i in range(1, 6):
    print("*" * i)
```

SALIDA:

```
*
**
***
****
*****
```

¡Practica patrones! Son muy entretenidos y útiles.



6. FUNCIONES ÚTILES CON BUCLES

Función	Descripción	Ejemplo
<code>range(inicio, fin, paso)</code>	Genera una secuencia de números.	<code>range(1, 10, 2)</code> → 1, 3, 5, 7, 9
<code>len(lista)</code>	Devuelve la cantidad de elementos.	<code>len([1,2,3])</code> → 3
<code>enumerate(iterable)</code>	Devuelve índice y valor en cada iteración.	<code>for i, v in enumerate([10,20]):</code> <code>print(i, v) # 0 10 / 1 20</code>
<code>zip(lista1, lista2)</code>	Combina elementos de dos listas.	<code>for a, b in zip([1,2],["a","b"]):</code> <code>print(a, b) # 1 a / 2 b</code>

7. COMBINAR FOR Y WHILE

A veces un bucle controla la cantidad y el otro la condición.

Ejemplo: repetir hasta acierto

```
secreto = 7
intentos = 0
while True:
    n = int(input("Adivina: "))
    intentos += 1
    if n == secreto:
        print(f"Correcto en {intentos} intentos!")
        break
```

¿Qué hace?

- ✓ El while controla el ciclo.
- ✓ El if verifica y termina el bucle.
- ✓ Cuenta los intentos.



DESAFÍOS PRÁCTICOS

Intenta resolver estos ejercicios:

- Sumar todos los números del 1 al 100.
- Contar cuántos números impares hay del 1 al 50.
- Pedir al usuario 5 números y mostrar el promedio.
- Imprimir la tabla de multiplicar del 1 al 10 del número que elija el usuario.



MINI QUIZ

¿Qué imprime este código?

```
suma = 0
for i in range(1, 4):
    suma += i
print(suma)
```

- a) 3 b) 6
c) 9 d) 10

Respuesta: b) 6

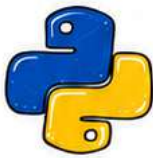


TIP PRO

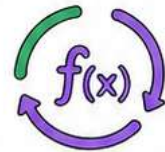
- ✓ Piensa en el objetivo antes de elegir el bucle.
- ✓ Usa for cuando conozcas el número de veces.
- ✓ Usa while cuando dependas de una condición.
- ✓ Siempre asegúrate de que el bucle termine. Practica, practica y practica.

Los bucles son la base para automatizar todo en Python. ¡Domínalos!





FUNCIONES



1. INTRODUCCIÓN A LAS FUNCIONES



Una función es un bloque de código que realiza una **tarea específica**.

Permite organizar mejor tu programa, **reutilizar código** y evitar repeticiones innecesarias.

Piensa en una función como una **receta**: le das algunos ingredientes (datos) y obtienes un resultado.

¿POR QUÉ USAR FUNCIONES?

ORGANIZACIÓN



Mantiene el código ordenado y fácil de entender.

REUTILIZACIÓN



Escribes el código una vez y lo usas muchas veces.

MENOS ERRORES



Al centralizar el código, es más fácil encontrar y corregir errores.

Ahorro de tiempo



Te permite enfocarte en la lógica principal del programa.

SINTAXIS BÁSICA

```
def nombre_funcion(parametros):  
    # bloque de código  
    return resultado # opcional
```

- **def**: palabra clave para definir una función.
- **nombre_funcion**: el nombre que le das.
- **parametros**: datos que recibe (opcional).
- **bloque de código**: instrucciones que se ejecutan.
- **return**: devuelve un valor (opcional).

EJEMPLO SIMPLE

```
def saludar():  
    print("¡Hola, mundo!")
```

Llamada a la función:

saludar()

Salida:

¡Hola, mundo!



NOTA

Una función no se ejecuta hasta que la llamas. Solo la defines.

PARTES DE UNA FUNCIÓN

Definición (def)	Indica que vas a crear una función.
Nombre	Debe ser descriptivo y seguir las reglas de nombres de Python.
Parámetros	Son los datos que la función puede recibir.
Bloque	Código que se ejecuta cuando se llama la función (siempre indentado).
Return	Devuelve un valor al lugar desde donde fue llamada (opcional).

EJEMPLO CON RETORNO

```
def sumar(a, b):  
    resultado = a + b  
    return resultado  
  
# Llamada a la función  
suma_total = sumar(3, 5)  
print(suma_total)
```

Salida:

8

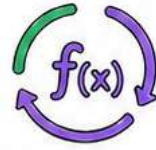
La función retorna el resultado de la suma y lo guardamos en la variable **suma_total**.





FUNCIONES

Volumen 8



2. PARÁMETROS Y ARGUMENTOS



Los **parámetros** son las variables que se definen en la función.

Los **argumentos** son los valores que le pasas cuando la llamas.

PARÁMETROS POSICIONALES

El orden importa.

```
def presentar(nombre, edad):  
    print(f"Me llamo {nombre} y tengo {edad} años.")
```

Llamada a la función

```
presentar("Ana", 25)
```

↑ ↑
argumento 1 argumento 2

Salida:

Me llamo Ana
y tengo 25 años.

PARÁMETROS POR NOMBRE (KEYWORD ARGUMENTS)

Puedes indicar el nombre del parámetro al llamar.

```
def presentar(nombre, edad):  
    print(f"Me llamo {nombre} y tengo {edad} años.")
```

Llamada usando nombres

```
presentar(edad=30, nombre="Luis")
```

Salida:

Me llamo Luis
y tengo 30 años.

El orden ya no importa cuando usas nombres.
¡Pero debe coincidir el nombre del parámetro!



PARÁMETROS CON VALOR POR DEFECTO

Si no pasas un argumento, se usa el valor por defecto.

```
def saludar(nombre, saludo="Hola"):  
    print(f"{saludo}, {nombre}!")
```

Llamadas

```
saludar("Marta")
```

usa 'Hola' por defecto

```
saludar("Pedro", "Buen día") # sobrescribe el saludo
```

Salida:

Hola, Marta!
Buen día, Pedro!



IDEA CLAVE

Los valores por defecto deben ir siempre al final de la lista de parámetros.

```
def correcto(a, b, c=10): ✓  
def incorrecto(a=10, b): ✗
```

PARÁMETROS VARIABLES: *args

Permiten recibir un número variable de argumentos posicionales. Se reciben como una tupla.

```
def sumar_todo(*numeros):  
    total = 0  
    for n in numeros:  
        total += n  
    return total
```

Llamadas

```
sumar_todo(1, 2, 3)      # 3 argumentos
```

```
sumar_todo(10, 20)      # 2 argumentos
```

```
sumar_todo(7, 8, 9, 10, 11) # 5 argumentos
```

Salida:

6

30

45

*args puede tener cualquier nombre (*valores, *datos, *items, etc.), pero por convención se usa *args.



PARÁMETROS VARIABLES POR NOMBRE: **kwargs

Permiten recibir un número variable de argumentos por nombre. Se reciben como un diccionario.

```
def mostrar_info(**datos):  
    for clave, valor in datos.items():  
        print(f"{clave}: {valor}")
```

Llamada

```
mostrar_info(nombre="Ana", edad=25, ciudad="Córdoba")
```

Salida:

nombre: Ana
edad: 25
ciudad: Córdoba



kwargs puede tener cualquier nombre (kw, **opciones, **datos, etc.).

RESUMEN RÁPIDO

Tipo	Símbolo	Qué recibe	Ejemplo
Posicional	(nada)	Valores en orden	func(1, 2)
Por nombre	clave=valor	Valores con nombre	func(a=1, b=2)
Con defecto	= valor	Usa defecto si no se pasa	func(a=1, b=2)
Variable posicional	*args	Cualquier cantidad (tupla)	func(1, 2, 3, 4)
Variable por nombre	**kwargs	Cualquier cantidad (dict)	func(a=1, b=2)

MINI QUIZ



¿Cuál será la salida del siguiente código?

```
def saludar(nombre, saludo="Hola"):  
    print(f"{saludo}, {nombre}!")  
saludar("Lucía")  
saludar(saludo="Buen día", nombre="Juan")
```

- a) Hola, Lucía! / Buen día, Juan!
- b) Lucía, Hola! / Juan, Buen día!
- c) Error en la segunda llamada
- d) Buen día, Lucía! / Hola, Juan!

Respuesta:

a)

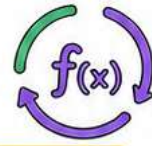




FUNCIONES

Volumen 8

3. RETURN Y VALORES DE RETORNO



El **return** es lo que hace útil a una función: devuelve un **valor al lugar** donde fue llamada. Si no usas **return**, la función devuelve **None** (nada).

RETURN EXPLÍCITO

Usamos **return** para devolver un valor específico.

```
def cuadrado(x):  
    resultado = x * x  
    return resultado
```

Llamada a la función

```
valor = cuadrado(5)  
print(valor)
```

Salida:

25

La función calcula y devuelve el resultado.

RETURN IMPLÍCITO

Si no hay **return**, Python devuelve **None**.

```
def saludar(nombre):  
    print(f"Hola, {nombre}!")  
    # no hay return
```

```
respuesta = saludar("Ana")  
print(respuesta)
```

Salida:

Hola, Ana!
None

La función realiza una acción, pero no devuelve un valor.

RETURN MÚLTIPLE

Puedes devolver varios valores separados por comas (como una tupla).

```
def operaciones(a, b):  
    suma = a + b  
    resta = a - b  
    return suma, resta
```

```
s, r = operaciones(10, 4)
```

```
print(s) # 14  
print(r) # 6
```

Salida:

14
6

Python agrupa los valores y los devuelve como una tupla.

¿CÓMO FUNCIONA?



REGLAS IMPORTANTES

- ✓ Cuando se ejecuta **return**, la función se detiene inmediatamente.
- ✓ Puedes tener varios **return**, pero solo se ejecutará uno.
- ✓ El código después de un **return** no se ejecuta.
- ✓ Si no hay **return**, Python devuelve **None** automáticamente.

```
def ejemplo(x):  
    return x  
    print("Esto nunca se ejecuta")
```

Después de **return**, la función termina. ¡Nada más se ejecuta!

MÁS EJEMPLOS

- 1. Verificar número par

```
def es_par(n):  
    return n % 2 == 0  
  
print(es_par(4)) # True  
print(es_par(7)) # False
```

Devuelve True o False. ✓

- 2. Calcular promedio

```
def promedio(a, b, c):  
    suma = a + b + c  
    return suma / 3  
  
print(promedio(3, 6, 9))
```

Salida:

6.0

Devuelve un número decimal. ★

- 3. Índice y valor máximo

```
def maximo(lista):  
    max_val = max(lista)  
    idx = lista.index(max_val)  
    return max_val, idx  
  
valor, pos = maximo([2, 9, 4, 7])  
print(valor, pos) # 9 1
```

Salida:

9 1

Devuelve dos valores: el máximo y su posición.

ERRORES COMUNES

- ✗ Esperar un valor y no usar **return**.

```
def sumar(a, b):  
    c = a + b  
    # falta return c  
    print(sumar(2, 3)) # None
```

- ✗ Usar **return** sin valor.

```
def nada():  
    return  
    print(nada()) # None
```

- ✗ Código después de **return**.

```
def ejemplo():  
    return 1  
    print("Nunca se ejecuta")  
    print(ejemplo()) # 1
```

TIP PRO

- ✓ Piensa si tu función debe devolver algo o solo realizar una acción.
- ✓ Usa nombres descriptivos para que sea claro qué devuelve.
- ✓ Prueba siempre tus funciones con diferentes entradas.

Funciones pequeñas y bien pensadas = programas más limpios y reutilizables. 🧠

MINI QUIZ

¿Qué imprime el siguiente código?

```
def dobles(n):  
    return n * 2  
x = dobles(7)  
print(x)
```

- a) 7 b) 14
c) None d) Error

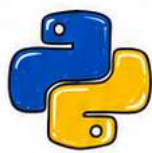
Respuesta: b) 14

EN RESUMEN

- Las funciones organizan y reutilizan código.
- Los **parámetros** son las entradas.
- El **return** devuelve el resultado.
- Sin **return** → se devuelve **None**.
- Pueden devolver uno o varios valores.

¡Domina las funciones y Python será tu mejor herramienta! 🧑‍💻





FUNCIONES

Volumen 8



4. ALCANCE (SCOPE) Y VARIABLES



El alcance (scope) determina dónde puedes usar una variable.

Python sigue la regla **LEGB**:

Local → **Enclosing** → **Global** → **Built-in**

1. VARIABLES LOCALES

Se crean dentro de la función y solo existen mientras la función se ejecuta.

```
def funcion():  
    x = 10 # local  
    print("Dentro:", x)  
  
funcion()  
# print(x) # Error: x no existe aquí
```

Salida:

Dentro: 10

Fuera no
existe x

Las variables locales protegen los datos de afuera.

2. VARIABLES GLOBALES

Se crean fuera de la función y pueden usarse desde cualquier parte del programa.

```
x = 5 # global  
  
def mostrar():  
    print("El valor de x es:", x)  
  
mostrar()  
print(x)
```

Salida:

El valor de
x es: 5
5

Las funciones pueden leer variables globales, pero no pueden modificarlas (por defecto).

3. MODIFICAR GLOBALES: global

Para modificar una variable global dentro de una función, usamos la palabra clave **global**.

```
contador = 0 # global  
  
def incrementar():  
    global contador # indica que usaremos la global  
    contador += 1  
    print("Contador dentro:", contador)  
  
incrementar()  
print("Contador fuera:", contador)
```

Salida:

Contador dentro: 1
Contador fuera: 1

Sin **global**, Python crearía una variable local con el mismo nombre y la global no cambiaría.

4. VARIABLES EN FUNCIONES ANIDADAS: nonlocal

Sirve para modificar una variable en la función externa más cercana (no global).

```
def externa():  
    mensaje = "Hola" # enclosing  
  
    def interna():  
        nonlocal mensaje # apunta a la variable  
        de la función externa  
        mensaje = "¡Hola desde interna!"  
        print(mensaje)  
  
    interna()  
    print(mensaje)  
  
externa()
```

Salida:

¡Hola desde
interna!
¡Hola desde
interna!

nonlocal busca en el scope externo; si no encuentra, usa **global** (si existe) o da error.

5. FUNCIONES INCORPORADAS (BUILT-IN)

Python trae muchas funciones listas para usar.

print()



Muestra información
por pantalla.

len()



Devuelve la cantidad
de elementos.

type()



Devuelve el tipo
de dato.

sum()



Suma los elementos
de una colección.

max() / min()



Devuelve el valor
máximo o mínimo.

sorted()



Devuelve una lista
ordenada.



Úsalas para escribir código más corto, claro y eficiente.

EJEMPLO COMPLETO



```
total_llamadas = 0 # global  
  
def registrar_llamada(duración):  
    global total_llamadas  
    total_llamadas += 1  
    print(f"Llamada de {duración} min registrada.")  
  
registrar_llamada(15)  
registrar_llamada(8)  
print(f"Total llamadas:", total_llamadas)
```

Salida:

Llamada de
15 min registrada.
Llamada de
8 min registrada.
Total llamadas: 2

MINI QUIZ



¿Qué imprimirá el siguiente código?

```
x = 3  
def f():  
    x = x + 2  
    print(x)  
  
f()
```

a) 5 b) 3
c) Error d) 6

Respuesta: c) Error

TIP PRO



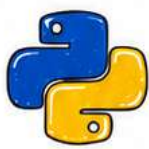
- ✓ Usa funciones para tareas repetitivas.
- ✓ Usa nombres claros y descriptivos.
- ✓ Evita funciones muy largas.
- ✓ Piensa en el scope de tus variables.
- ✓ Prueba tus funciones con distintos datos.

Funciones pequeñas, nombres claros
y buen uso del scope = código profesional



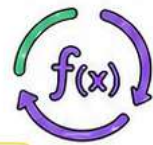
Volumen 8

• Funciones



FUNCIONES

Volumen 8



5. MEJORES PRÁCTICAS Y CONSEJOS



Escribe funciones **claras**, **pequeñas** y con un solo propósito.

Nombres descriptivos, buen formato y pruebas te harán un mejor programador.

1. NOMBRES RECOMENDADOS

Usa nombres que indiquen qué hace la función.

```
✓ def calcular_promedio(lista):
  def obtener_datos_cliente(id):
  def es_numero_primo(n):
```

```
✗ def f(x):
  def funcion1():
```

Buen nombre
= código
fácil de leer
y mantener.

2. FUNCIONES PEQUEÑAS

Cada función debe hacer UNA sola cosa.

✗ Mala idea
Hace muchas cosas

```
leer datos
validar
calcular
mostrar resultados
```

✓ Buena idea
Funciones pequeñas
y enfocadas

```
leer_datos()
validar_datos()
calcular()
mostrar_resultados()
```

Funciones pequeñas son más fáciles de probar,
reutilizar y corregir.

3. USA DOCSTRINGS

Explica qué hace tu función.

```
def area_rectangulo(base, altura):
    """Calcula el área de un rectángulo.
    Parámetros:
        base (float): la base del rectángulo
        altura (float): la altura del rectángulo
    Retorna:
        float: el área calculada
    """
    return base * altura
```

Los docstrings
ayudan a ti y a
otros a entender
el propósito de
la función.

4. FORMATO Y ESTILO (PEP 8 BÁSICO)

- ✓ Usa snake_case para nombres de funciones y variables.
- ✓ Usa 4 espacios para sangría.
- ✓ Deja una línea en blanco entre funciones.
- ✓ Máximo 79-99 caracteres por línea.
- ✓ Escribe espacios alrededor de operadores (+ - * / ==).

```
def suma(a, b):
    resultado = a + b
    return resultado
```

Código limpio
= código
profesional

5. PRUEBA TUS FUNCIONES

Siempre prueba tus funciones con distintos datos.

Ejemplo: función que divide

```
def dividir(a, b):
    if b == 0:
        return None
    return a / b
```

Pruebas recomendadas

Casos normales: dividir(10, 2)
Cero: dividir(5, 0)
Negativos: dividir(-10, 2)
Decimales: dividir(7.5, 2.5)

Salida esperada

5.0 → correcto
None → evita error
-5.0 → correcto
3.0 → correcto

Probar es la
mejor forma de
encontrar errores
antes de que lo
hagan otros.

6. ERRORES COMUNES

- ✗ No retornar nada (devuelve None sin querer)

```
def saludar():
    print("Hola") # No retorna nada
```
- ✗ Usar variables globales cuando no es necesario

```
total = 0 # global innecesaria
```
- ✗ Funciones muy largas y complicadas
Divide en funciones más pequeñas.

7. CUÁNDO USAR FUNCIONES

Situación	Usa función cuando...
Repetición	Vas a usar el mismo código varias veces.
Organización	Tu programa crece y necesita orden.
Legibilidad	Quieres que el código sea fácil de entender.
Mantenimiento	Necesitas cambiar algo en un solo lugar.

8. RESUMEN RÁPIDO

- Las funciones organizan y reutilizan código.
- Pueden recibir datos (parámetros).
- Pueden devolver resultados (return).
- El scope define dónde se usan las variables.
- Escribe funciones pequeñas y enfocadas.
- Usa nombres descriptivos y docstrings.
- Prueba tus funciones siempre.

¡Domina las funciones y llevarás tus programas al siguiente nivel!

MINI QUIZ

¿Qué imprime el siguiente código?

```
def doble(n):
    return n * 2

print(doble(7))
```

a) 5 b) 14
c) 9 d) None

Respuesta: b) 14

TIP PRO

- ✓ Piensa en el problema → divide en pasos.
- ✓ Crea una función por cada paso importante.
- ✓ Prueba cada función por separado.
- ✓ Combina todo en tu programa principal.

¡Pequeños bloques de código hoy,
grandes proyectos mañana!

EJEMPLO INTEGRADOR

```
def es_par(n):
    return n % 2 == 0

def sumar_pares(lista):
    total = 0
    for n in lista:
        if es_par(n):
            total += n
    return total

print(sumar_pares([1, 2, 3, 4, 5, 6])) # 12
```

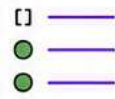
Funciones que
trabajan juntas
= programas
potentes.





LISTAS

Volumen 9



1. INTRODUCCIÓN A LAS LISTAS



Una lista es una colección **ordenada** y **mutable** de elementos.
Puede contener elementos de cualquier tipo y se accede por su **índice**.
Se define usando **corchetes** `[ ]`.

¿POR QUÉ USAR LISTAS?

- ✓ Permiten almacenar muchos valores en una sola variable.
- ✓ Son ordenadas: cada elemento tiene una posición (índice).
- ✓ Son mutables: podemos agregar, eliminar o modificar elementos.
- ✓ Muy usadas en programación para manejar colecciones.

EJEMPLO BÁSICO

```
# Creamos una lista
frutas = ["manzana", "banana", "uva"]

# Accedemos a elementos por índice
print(frutas[0]) # manzana
print(frutas[1]) # banana
print(frutas[2]) # uva
```

★ RECUERDA

El primer elemento tiene índice 0, no 1.

Índice: 0 1 2
frutas: manzana banana uva

CARACTERÍSTICAS

- Ordenadas**
Los elementos mantienen su posición.
- Mutables**
Se pueden cambiar, agregar o eliminar elementos.
- Permiten duplicados**
Pueden existir valores repetidos.
- Pueden ser heterogéneas**
Almacenan distintos tipos de datos.

CREAR LISTAS

1. Lista vacía

```
lista_vacia = []
print(lista_vacia) # []
```

2. Lista con valores

```
colores = ["rojo", "azul", "verde"]
print(colores) # ['rojo', 'azul', 'verde']
```

3. Lista con tipos diferentes

```
datos = [10, "hola", 3.14, True]
print(datos) # [10, 'hola', 3.14, True]
```

ACCESO A ELEMENTOS Y SLICING

Acceder por índice

```
nums = [5, 10, 15, 20]
print(nums[0]) # 5 (primero)
print(nums[2]) # 15 (tercero)
print(nums[-1]) # 20 (último)
print(nums[:2]) # 5 (penúltimo)
```

Slicing (porciones)

```
nums = [5, 10, 15, 20, 25]
print(nums[1:3]) # [10, 15] (desde 1 hasta 3-1)
print(nums[:3]) # [5, 10, 15] (desde inicio hasta 2)
print(nums[2:]) # [15, 20, 25] (desde 2 hasta el final)
print(nums[::-2]) # [5, 15, 25] (paso de 2)
```

SINTAXIS GENERAL

`lista[inicio:fin:paso]`

- **inicio**: desde dónde empieza (incluido).
- **fin**: hasta dónde llega (no incluido).
- **paso**: de cuánto en cuánto (por defecto 1).

MÉTODOS BÁSICOS (PARTE 1)

append(x)

Agrega un elemento al final.



```
frutas = ["manzana"]
frutas.append("banana")
print(frutas)
# ['manzana', 'banana']
```

insert(i, x)

Inserta un elemento en la posición i.



```
frutas = ["manzana", "uva"]
frutas.insert(1, "banana")
print(frutas)
# ['manzana', 'banana', 'uva']
```

extend(lista)

Extiende la lista con otra lista.



```
a = [1, 2]
b = [3, 4]
a.extend(b)
print(a)
# [1, 2, 3, 4]
```

remove(x)

Elimina la primera aparición de x.



```
nums = [1, 2, 2, 3]
nums.remove(2)
print(nums)
# [1, 2, 3]
```

pop(i)

Elimina y devuelve el elemento en índice i (último si no se indica).



```
nums = [10, 20, 30]
valor = nums.pop(1)
print(valor) # 20
print(nums) # [10, 30]
```

clear()

Elimina todos los elementos.



```
nums = [1, 2, 3]
nums.clear()
print(nums)
# []
```



TIP IMPORTANTE

Las listas son mutables. Si asignas una lista a otra variable, ambas apuntan al mismo objeto.

```
a = [1, 2, 3]
b = a
a.append(4)
print(b) # [1, 2, 3, 4]
```

MINI QUIZ

¿Qué imprime?

```
lista = [0, 1, 2, 3, 4]
print(lista[1:4])
```

a) [1, 2, 3, 4]
b) [0, 1, 2]
c) [1, 2, 3]
d) [2, 3, 4]

Respuesta: c) [1, 2, 3]



EN RESUMEN

- ✓ Una lista guarda muchos valores en orden.
- ✓ Se accede por índices (empezando en 0).
- ✓ Es mutable: podemos cambiarla.
- ✓ Tiene métodos útiles para agregar, eliminar y más.
- ✓ El slicing permite obtener porciones fácilmente.

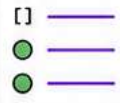
¡Las listas son una herramienta fundamental en Python! 🐍





LISTAS

Volumen 9



2. MÉTODOS BÁSICOS (PARTE 2)



Python tiene **métodos** listos para trabajar con listas sin escribir bucles manualmente. Usarlos te ahorra **tiempo** y hace tu código más **legible**.

1. index(x)



Devuelve el índice de la primera aparición de x.

```
nums = [5, 10, 15, 10]
i = nums.index(10)
print(i) # 1
```

Salida:

1

★ Si x no existe, da error (ValueError).

2. count(x)

123

Cuenta cuántas veces aparece x en la lista.

```
nums = [1, 2, 2, 3, 2]
c = nums.count(2)
print(c) # 3
```

Salida:

3

3. sort()



Ordena la lista de forma ascendente (por defecto).

```
nums = [3, 1, 4, 1, 5]
nums.sort()
print(nums)
```

Salida:

[1, 1, 3, 4, 5]

★ Modifica la lista original.

4. reverse()



Invierte el orden de los elementos.

```
nums = [1, 2, 3, 4]
nums.reverse()
print(nums)
```

Salida:

[4, 3, 2, 1]

★ Modifica la lista original.

5. copy()



Devuelve una copia superficial de la lista.

```
a = [1, 2, 3]
b = a.copy()
print(b)
```

Salida:

[1, 2, 3]

★ Útil para no modificar la lista original.

6. clear()



Elimina todos los elementos de la lista.

```
nums = [1, 2, 3]
nums.clear()
print(nums)
```

Salida:

[]

★ La lista queda vacía.

7. pop(i)



Elimina y devuelve el elemento en el índice i (por defecto, el último).

```
nums = [10, 20, 30]
x = nums.pop(1)
print(x) # 20
print(nums) # [10, 30]
```

Salida:

20

[10, 30]

★ Modifica la lista original.

8. remove(x)



Elimina la primera aparición de x en la lista.

```
nums = [4, 5, 4, 6]
nums.remove(4)
print(nums)
```

Salida:

[5, 4, 6]

★ Si x no existe, da error.

sorted() VS sort()

sorted() devuelve una **NUEVA** lista ordenada sin modificar la original.

```
a = [3, 1, 2]
b = sorted(a)
print(a) # [3, 1, 2] (sin cambios)
print(b) # [1, 2, 3] (nueva lista)
```

Devuelve nueva lista

VS

sort() ordena la lista **ORIGINAL** en el lugar.

```
a = [3, 1, 2]
a.sort()
print(a) # [1, 2, 3] (lista modificada)
```

Modifica la misma lista

COMBINACIONES ÚTILES

- ➕ Agregar al inicio:
`nums.insert(0, nuevo)`
- ✂ Eliminar duplicados (manteniendo orden):
`seen = set()`
`unicos = []`
`for x in nums:`
 `if x not in seen:`
 `seen.add(x)`
 `unicos.append(x)`
- 🔗 Concatenar listas:
`a = [1, 2]`
`b = [3, 4]`
`c = a + b # [1, 2, 3, 4]`

MINI QUIZ



¿Qué imprime el siguiente código?

```
nums = [7, 8, 7, 9]
print(nums.count(7))
nums.remove(7)
nums.append(10)
nums.sort()
print(nums)
```

- a) 1 [7, 8, 9, 10]
- b) 2 [7, 8, 9, 10]
- c) 2 [8, 9, 10]
- d) 1 [8, 9, 10]

Respuesta: **b) 2 [7, 8, 9, 10]**

ERRORES COMUNES

- ✗ Usar **sort()** esperando que devuelva una lista (devuelve None).
`resultado = nums.sort() # Malo`
- ✗ Modificar una lista mientras iteras sobre ella.
`for x in nums:`
 `if == --`
 `nums.remove(x) # Puede fallar`
- ✗ Usar **index(x)** cuando puede haber varios x (devuelve el primero solo). Usa un bucle o list comprehension para todos los índices.





LISTAS

Volumen 9

3. MÉTODOS AVANZADOS Y ÚTILES



Estos métodos te ayudan a resolver tareas comunes de forma más **eficiente**.
Úsalos en lugar de escribir **bucles** cuando sea posible.

1. extend(iterable)



Agrega todos los elementos de un iterable al final de la lista.

```
a = [1, 2]
a.extend([3, 4, 5])
print(a)
```

Salida:
[1, 2, 3, 4, 5]

★ Modifica la lista original.

2. in y not in



Verifica si un elemento está (in) o no está (not in) en la lista.

```
frutas = ["manzana", "pera", "uva"]
print("pera" in frutas) # True
print("banana" not in frutas) # True
```

Salida:
True
True

★ Devuelve True o False.

3. min() y max()



Devuelven el valor mínimo y máximo de la lista (solo con números).

```
nums = [7, 2, 9, 1, 5]
print(min(nums)) # 1
print(max(nums)) # 9
```

Salida:
1
9

★ La lista no se modifica.

4. sum()



Devuelve la suma de todos los números de la lista.

```
nums = [2, 4, 6, 8]
print(sum(nums)) # 20
```

Salida:
20

★ Solo funciona con listas numéricas.

5. any() y all()



any(): True si al menos un elemento es verdadero.

all(): True si todos los elementos son verdaderos.

```
datos = [True, False, True]
print(any(datos)) # True
print(all(datos)) # False
```

Salida:
True False

★ Útiles con listas booleanas.

6. enumerate()



Devuelve pares (índice, valor) de la lista.

```
colores = ["rojo", "verde", "azul"]
for i, color in enumerate(colores):
    print(i, color)
```

Salida:
0 rojo
1 verde
2 azul

★ Muy útil en bucles.

7. LIST COMPREHENSIONS (COMPRENSIONES DE LISTAS)



Forma corta y clara de crear listas usando una sola línea de código.

BÁSICA

```
cuadrados = [x**2 for x in range(5)]
print(cuadrados)
```

Salida:
[0, 1, 4, 9, 16]

CON CONDICIÓN

```
pares = [x for x in range(10)
         if x % 2 == 0]
print(pares)
```

Salida:
[0, 2, 4, 6, 8]

CON CONDICIÓN Y EXPRESIÓN

```
result = ["par" if x % 2 == 0 else "impar"
          for x in range(1, 6)]
print(result)
```

Salida:
['impar', 'par', 'impar', 'par', 'impar']

8. LISTAS ANIDADAS

Una lista puede contener otras listas.

```
matriz = [[1, 2], [3, 4], [5, 6]]
print(matriz[0]) # [1, 2]
print(matriz[1][0]) # 3
print(matriz[2][1]) # 6
```

Salida:
[1, 2]
3
6

★ Accede por fila y columna.

EJEMPLOS PRÁCTICOS



1. Contar ocurrencias

```
letras = ['a', 'b', 'a', 'c', 'a']
print(letras.count('a'))
```

Salida:
3

Cuenta cuántas veces aparece un elemento.

2. Eliminar por valor

```
nums = [1, 2, 3, 2, 4]
nums.remove(2)
print(nums)
```

Salida:
[1, 3, 2, 4]

Elimina la primera aparición del valor indicado.

3. Copia vs Referencia

```
a = [1, 2, 3]
b = a # referencia
c = a.copy() # copia real
b.append(4)
c.append(5)
print(a) # [1, 2, 3, 4]
print(b) # [1, 2, 3, 4]
print(c) # [1, 2, 3, 5]
```

Usa .copy() para copiar y no modificar la original.

TIP PRO



- ✓ Prefiere métodos de listas sobre bucles manuales.
- ✓ Usa list comprehensions para código corto y eficiente.
- ✓ Recuerda: los índices empiezan en 0.
- ✓ Prueba tus listas con distintos datos.

MINI QUIZ



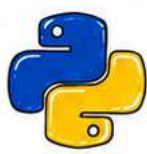
¿Qué mostrará el siguiente código?

```
lst = [10, 20, 30]
lst.extend([40, 50])
lst.pop(1)
print(lst)
```

b) [10, 20, 40, 50]
c) [10, 40, 50, 50]
d) [10, 40, 20]

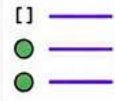
Respuesta: a) [10, 30, 40, 50]





LISTAS

Volumen 9



4. SLICING Y COMPREHENSIONS



Slicing (rebanado) permite obtener partes de una lista.

Las comprensiones de listas permiten crear listas de forma rápida y legible.

1. SLICING (REBANADO)

Sintaxis general

`lista[inicio:fin:paso]`

- **inicio**: índice donde empieza (incluido).
- **fin**: índice donde termina (no incluido).
- **paso**: salto entre elementos (por defecto 1).

Ejemplo base

```
nums = [0, 1, 2, 3, 4, 5, 6]
0     1     2     3     4     5     6
```

Expresión	Resultado	Explicación
<code>nums[2:5]</code>	<code>[2, 3, 4]</code>	Índices 2 hasta 4
<code>nums[:4]</code>	<code>[0, 1, 2, 3]</code>	Desde el inicio hasta índice 3
<code>nums[3:]</code>	<code>[3, 4, 5, 6]</code>	Desde índice 3 hasta el final
<code>nums[:]</code>	<code>[0, 1, 2, 3, 4, 5, 6]</code>	Copia completa de la lista
<code>nums[::-1]</code>	<code>[6, 5, 4, 3, 2, 1, 0]</code>	Invertir la lista
<code>nums[::2]</code>	<code>[0, 2, 4, 6]</code>	Cada 2 elementos
<code>nums[5:1:-2]</code>	<code>[5, 3]</code>	Desde 5 hasta 2 (no incl.), paso -2

★ Recuerda: el fin NO se incluye. El paso negativo recorre en sentido inverso.

2. LIST COMPREHENSIONS (COMPRENSIONES DE LISTAS)

Permiten crear listas nuevas a partir de otras listas, aplicando operaciones y/o filtros en una sola línea.

BÁSICA

Sintaxis:

`[expresión for item in iterable]`

Ejemplo:

```
cuadrados = [x**2 for x in range(6)]
print(cuadrados)
```

Salida:

```
[0, 1, 4, 9, 16, 25]
```

★ Aplica una expresión a cada elemento.

CON CONDICIÓN (FILTRO)

Sintaxis:

`[expresión for item in iterable if condición]`

Ejemplo:

```
pares = [x for x in range(10) if x % 2 == 0]
print(pares)
```

Salida:

```
[0, 2, 4, 6, 8]
```

★ Incluye solo los elementos que cumplen la condición.

CONDICIÓN + EXPRESIÓN

Sintaxis:

`[expr1 if condición else expr2 for item in iterable]`

Ejemplo:

```
etiquetas = ["par" if x % 2 == 0 else "impar" for x in range(5)]
print(etiquetas)
```

Salida:

```
['par', 'impar', 'par', 'impar', 'par']
```

★ Elige entre dos valores según se cumpla la condición.

3. OTROS MÉTODOS ÚTILES

Método	¿Qué hace?	Ejemplo	Salida
<code>join()</code>	Une elementos de una lista de strings con un separador.	<code>"".join(['a', 'b', 'c'])</code>	<code>"a-b-c"</code>
<code>len()</code>	Devuelve la cantidad de elementos.	<code>len([1, 2, 3, 4])</code>	<code>4</code>
<code>sorted()</code>	Devuelve una nueva lista ordenada.	<code>sorted([3, 1, 2])</code>	<code>[1, 2, 3]</code>
<code>reversed()</code>	Devuelve un iterador en orden inverso.	<code>list(reversed([1, 2, 3]))</code>	<code>[3, 2, 1]</code>
<code>index(x)</code>	Devuelve el índice de la primera aparición de x.	<code>[10, 20, 30].index(20)</code>	<code>1</code>
<code>in / not in</code>	Verifica existencia.	<code>5 in [1, 2, 3, 5]</code>	<code>True</code>
<code>extend()</code>	Agrega los elementos de otra lista al final.	<code>a=[1, 2]; a.extend([3, 4])</code>	<code>[1, 2, 3, 4]</code>

4. COMPRENSIÓN ANIDADA (2D)

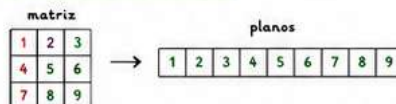
Útil para listas de listas (matrices).

Ejemplo:

```
matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
planos = [num for fila in matriz for num in fila]
print(planos)
```

Salida:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```



★ Lee fila por fila y agrega cada elemento a la nueva lista.

TIP PRO



- ✓ Usa slicing para evitar bucles cuando sea posible.
- ✓ Las comprensiones son más rápidas y legibles.
- ✓ Elige nombres descriptivos para tus listas.
- ✓ Prueba con ejemplos pequeños antes de datos grandes.



MINI QUIZ



¿Qué imprime el siguiente código?

```
nums = [10, 20, 30]
res = [nums[i] for i in range(1, 6, 2)]
print(res)
```

a) [1, 2, 3, 4, 5] b) [1, 3, 5]
c) [10, 30, 50] d) [20, 40]

Respuesta: b) [1, 3, 5]

ERRORES COMUNES

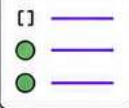
- ✗ Olvidar que el fin del slicing NO se incluye.
- ✗ Usar índices fuera de rango.
- ✗ Modificar una lista mientras se itera sobre ella.
- ✗ Confundir paso negativo en slicing.





LISTAS

Volumen 9



5. RECORRIDOS Y OPERACIONES COMUNES



Las listas se recorren fácilmente con bucles y permiten realizar muchas operaciones útiles. Dominar esto te hará más eficiente y te dará flexibilidad en tu código.

1. RECORRER UNA LISTA

Usa un bucle for para acceder a cada elemento.

```
frutas = ["manzana", "banana", "uva"]
for fruta in frutas:
    print(fruta) # imprime cada elemento
print("- - -")
for i in range(len(frutas)):
    print(i, frutas[i]) # imprime índice y valor
```

Salida:	Índice	elemento
manzana	0	manzana
banana	1	banana
uva	2	uva
- - -	-	uva



Recuerda:
El índice empieza en 0 y llega hasta len(lista)-1

2. MODIFICAR ELEMENTOS

Puedes cambiar el valor de un elemento usando su índice.

```
nums = [10, 20, 30, 40]
nums[1] = 99 # cambia 20 por 99
nums[3] = nums[3] + 10 # suma al valor
print(nums)
```

Salida:
[10, 99, 30, 50]



Importante:
Asegúrate de que el índice exista antes de modificar.

3. ELIMINAR ELEMENTOS

Existen varias formas de eliminar elementos.

- remove(x):** elimina la primera aparición del valor x.
`nums = [1, 2, 3, 2, 4]`
`nums.remove(2)` # [1, 3, 2, 4]
`x = nums.pop(0)` # x = 1 -> [3, 2, 4]
`del nums[2]` # [3, 2]
`nums.clear()` # []
- pop(i):** elimina y devuelve el elemento en el índice i.
- del lista[i]:** elimina el elemento en el índice i. (no devuelve).
- clear():** elimina todos los elementos.

Salida:
[]

⚠ remove() elimina por valor. pop() y del usan índice.

4. OPERACIONES ENTRE LISTAS

Puedes combinar, repetir y verificar elementos.

- Concatenar (+):** une dos listas.
- Repetición (*):** repite la lista n veces.
- in / not in:** verifica existencia.
- len():** devuelve la cantidad de elementos.

```
a = [1, 2, 3]
b = [4, 5]
c = a + b # [1, 2, 3, 4, 5]
d = a * 3 # [1, 2, 3, 1, 2, 3, 1, 2, 3]
print(3 in a) # True
print(10 not in b) # True
print(len(c)) # 5
```

Salida:

True
True
5

5. ORDENAR LISTAS

Ordena una lista usando sort() o sorted().

```
sort() - ordena en el lugar
sorted() - devuelve nueva lista

nums = [5, 2, 9, 1]
nums.sort() # ascendente
print(nums) # [1, 2, 5, 9]
nums.sort(reverse=True)
print(nums) # [9, 5, 2, 1]

sorted() - devuelve nueva lista
nums = [5, 2, 9, 1]
nueva = sorted(nums)
print(nueva) # [1, 2, 5, 9]
nueva = sorted(nums, reverse=True)
print(nueva) # [9, 5, 2, 1]
```

★ sort() modifica la lista original. sorted() no la modifica.

6. LIST COMPREHENSIONS (REPASO)

Crear listas nuevas de forma compacta.

```
cuadrados = [x**2 for x in range(6)]
pares = [x for x in range(10) if x % 2 == 0]
mayores_5 = [n for n in [3, 6, 2, 7, 1] if n > 5]
print(cuadrados)
print(pares)
print(mayores_5)
```

Salida:
[0, 1, 4, 9, 16, 25]
[0, 2, 4, 6, 8]
[6, 7]

Formato básico:

[expresión for item in iterable if condición]

7. PROBLEMAS COMUNES Y CÓMO EVITARLOS

Problema	¿Qué pasa?	Ejemplo incorrecto	Solución
Índice fuera de rango	Usar un índice que no exista.	<code>nums = [1, 2, 3]</code> <code>print(nums[2])</code> ❌	Usa índices válidos: 0 a len(lista)-1 <code>print(nums[2])</code> ✅
Modificar mientras se recorre	Puede saltar elementos.	<code>for x in nums:</code> <code>if x % 2 == 0:</code> <code>nums.remove(x)</code> ❌	Recorre una copia o usa comprensión <code>for x in nums[:]:</code> # copia <code>if x % 2 == 0:</code> <code>nums.remove(x)</code> ✅
Confundir + y ==	Asignar en lugar de comparar.	<code>if x = 5:</code> ❌ (SyntaxError)	Usa == para comparar: <code>if x == 5:</code> ✅
Olvidar que las listas son mutables	Cambios en una lista afectan a todas las referencias.	<code>a = [1, 2, 3]</code> <code>b = a</code> <code>b.append(4)</code> # a también cambia	Usa copia si no quieres modificar original: <code>b = a.copy()</code> # a b = +[:] <code>b.append(4)</code> # a NO cambia ✅

CHEAT SHEET RÁPIDO

len(lista)	→ cantidad
min(lista)	→ mínimo
max(lista)	→ máximo
sum(lista)	→ suma (números)
sorted(lista)	→ nueva lista ordenada
lista.sort()	→ ordena en el lugar
lista.count(x)	→ cuenta ocurrencias
lista.index(x)	→ índice de x (primera aparición)

MINI QUIZ

¿Qué imprime el siguiente código?

```
nums = [1, 2, 3]
nums.append(4)
nums[1] = 10
print(nums)

a) [1, 2, 3, 4]    b) [1, 10, 3, 4]
c) [10, 2, 3, 4]    d) [1, 2, 3]
```

Respuesta: b) [1, 10, 3, 4]

TIP PRO

- ✓ Piensa antes de modificar una lista.
- ✓ Usa funciones y métodos para simplificar.
- ✓ Lee los índices con atención.
- ✓ Prueba con ejemplos pequeños antes de datos grandes.
- ✓ Las listas son poderosas: ¡úsalas bien!

EJEMPLO PRÁCTICO

Contar palabras en una frase:

```
texto = "Python es genial y Python es útil"
palabras = texto.split()
unicas = list(set(palabras))
print(unicas)
```

Salida: ['Python', 'es', 'genial', 'y', 'útil']

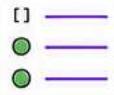




LISTAS

6. RESUMEN GENERAL

Volumen 9



Las listas son una herramienta **poderosa** y **flexible** en Python.

Domina las **métodos**, **comprensiones** y buenas prácticas para escribir código más **limpio** y **eficiente**.

ELEMENTOS CLAVE

- ✓ Ordenadas: mantienen el orden de inserción.
- ✓ Mutables: podemos cambiar, agregar y eliminar elementos.
- ✓ Indexadas: accedemos por índice (empieza en 0).
- ✓ Pueden almacenar distintos tipos de datos.
- ✓ Muy usadas en bucles, funciones y estructuras de datos.

OPERACIONES MÁS USADAS

Operación	Ejemplo
Acceder	lista[2]
Slicing	lista[1:4]
Agregar al final	lista.append(x)
Insertar en índice	lista.insert(i, x)
Eliminar por índice	lista.pop(i)
Eliminar por valor	lista.remove(x)
Ordenar	lista.sort()
Invertir	lista.reverse()
Longitud	len(lista)
Comprobar existencia	x in lista / x not in lista
Recorrer	for x in lista:

COMPRESIÓN DE LISTAS

Crea listas nuevas de forma compacta.

Sintaxis básica:

```
[expresión for item in iterable  
if condición]
```

Ejemplos:

```
cuadrados = [x**2 for x in range(6)]  
pares = [x for x in range(10) if x & 2 == 0]  
mayores_5 = [n for n in [3, 6, 2, 7, 1] if n > 5]  
con_vocal = [p for p in ["sol", "luz", "mar"]  
if 'a' in p]
```

EJEMPLOS RÁPIDOS DE USO DIARIO

1. Contar elementos

```
nombres = ["Ana", "Luis", "Sol"]  
print(len(nombres)) # 3
```

2. Filtrar elementos

```
nums = [1, 2, 3, 4, 5, 6]  
mayores_3 = [n for n in nums if n > 3]  
print(mayores_3) # [4, 5, 6]
```

3. Transformar elementos

```
palabras = ["Python", "es", "genial"]  
mayus = [p.upper() for p in palabras]  
print(mayus) # ['PYTHON', 'ES', 'GENIAL']
```

4. Sumar valores

```
valores = [10, 20, 30]  
total = sum(valores)  
print(total) # 60
```

5. Buscar elemento

```
datos = ["rojo", "azul", "verde"]  
print("azul" in datos) # True  
print("amarillo" not in datos) # True
```

6. Copiar lista

```
original = [1, 2, 3]  
copia = original.copy()  
copia.append(4)  
print(original) # [1, 2, 3]  
print(copia) # [1, 2, 3, 4]
```

BUENAS PRÁCTICAS

- ✓ Usa nombres descriptivos para tus listas.
- ✓ Prefiere comprensiones para código corto y eficiente.
- ✓ Evita modificar una lista mientras iteras sobre ella. (usa copias o listas nuevas).
- ✓ Verifica siempre si un índice existe antes de acceder.
- ✓ Usa `any()` o `all()` para lógica booleana.
- ✓ No uses métodos que cambian en vez de independientes (ej. `nums.sort()` cambia la lista original).
- ✓ Usa métodos apropiados (`append`, `pop`, etc.) en lugar de reescribir o reemplazar.

ERRORES COMUNES

- ✗ Usar índices fuera de rango.
`IndexError`
- ✗ Olvidar que `pop(i)` elimina y devuelve el mismo tiempo.
- ✗ Confundir `remove(x)` (por valor) con `pop(i)` (por índice).
- ✗ Modificar la lista mientras se recorre.
- ✗ Usar `=` para copiar (ambas variables apuntan a la misma lista).

COMPLEJIDAD (TIEMPO)

Operación	Complejidad
Acceder por índice	O(1)
Agregar al final (<code>append</code>)	O(1) amortizado
Insertar en índice	O(n)
Eliminar por índice (<code>del</code>)	O(n)
Eliminar por valor (<code>remove</code>)	O(n)
Buscar elemento (<code>in</code>)	O(n)
Recorrer lista	O(n)
Recorrer lista (<code>loop</code> , <code>len</code>)	O(n)

n = cantidad de elementos

RESUMEN EXPRESS

```
Declarar      -> lista = []  
Acceder       -> lista[i]  
Slicing       -> lista[inicio:fin:paso]  
Agregar al final -> lista.append(x)  
Insertar      -> lista.insert(i, x)  
Eliminar por índice -> lista.pop(i)  
Eliminar por valor -> lista.remove(x)  
Ordenar       -> lista.sort()  
Invertir      -> lista.reverse()  
Longitud      -> len(lista)  
Comprobar existencia -> x in lista / x not in lista  
                                     (exp for x in iterable if condición)
```

MINI QUIZ

¿Qué imprime el siguiente código?

```
nums = [2, 4, 6, 8, 10]  
nums.remove(6)  
nums.insert(1, 9)  
print(nums[1:4])
```

a) [2, 4, 8] b) [9, 8, 10]
c) [4, 6, 8] d) [9, 6, 8]

Respuesta: b) [9, 8, 10]

RETO RÁPIDO

Crea una lista con los cuadrados de los números del 1 al 10 usando comprensión de listas.

Ejemplo de salida:

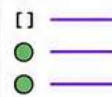
```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```





TUPLAS

Volumen 10



1. ¿QUÉ SON LAS TUPLAS?



Las tuplas (tuple) son colecciones **ordenadas** e **inmutables** de elementos.
Se usan para almacenar datos que **no deben cambiar**.

CONCEPTO CLAVE



- ✓ Ordenadas: mantienen el orden de inserción.
- ✓ Inmutables: una vez creadas, no se pueden modificar (no se pueden agregar, eliminar ni cambiar elementos).
- ✓ Permiten elementos duplicados.
- ✓ Más ligeras que las listas.
- ✓ Se definen usando paréntesis () en lugar de corchetes [].

¿CUÁNDO USAR TUPLAS?



- Cuando los datos no deben cambiar.
- Para devolver múltiples valores desde una función.
- Como claves en diccionarios (las listas no se pueden usar como claves).
- Para representar datos relacionados (coordenadas, fechas, configuraciones, etc.).

Ejemplo real

Coordenadas GPS (latitud, longitud):
ubicacion = (-34.6037, -58.3816)



1.1 CREAR TUPLAS

1. Tupla vacía



```
t1 = ()
print(t1)
```

Salida:
()

2. Tupla con elementos

```
t2 = (1, 2, 3, 4)
print(t2)
```

Salida:
(1, 2, 3, 4)

3. Diferentes tipos

```
t3 = ("Hola", 10, 3.14, True)
print(t3)
```

Salida:
('Hola', 10, 3.14, True)

4. Tupla de un solo elemento

```
t4 = (5,) # ¡Importante la coma!
print(t4)
type(t4)
```

Salida:
(5,) <class 'tuple'>

★ Si creas una tupla con un solo elemento, debes colocar una **coma**. De lo contrario, Python no la reconoce como tupla.

1.2 ACCEDER A ELEMENTOS

Por índice (empieza en 0)

```
datos = ("a", "b", "c", "d")
print(datos[0]) # a
print(datos[1]) # b
print(datos[3]) # d
print(datos[-1]) # d (último)
print(datos[-2]) # c (penúltimo)
```

Índices positivos

0 1 2 3

'a'	'b'	'c'	'd'
-----	-----	-----	-----

Índices negativos

-4 -3 -2 -1

Recuerda

El primer elemento
tiene índice 0.

Los índices negativos
cuentan desde el final.

1.3 VERIFICAR EXISTENCIA

```
frutas = ("manzana", "banana", "uva")
print("banana" in frutas) # True
print("pera" in frutas) # False
```

Útil para:

- ✓ Validaciones
- ✓ Búsquedas rápidas
- ✓ Condiciones



1.4 LONGITUD DE UNA TUPLA

```
colores = ("rojo", "verde", "azul")
print(len(colores)) # 3
```



len() -> devuelve la cantidad
de elementos.

1.5 CONVERTIR A TUPLA

```
lista = [1, 2, 3]
tupla = tuple(lista)
print(tupla)
type(tupla)
```

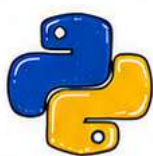


Salida:
(1, 2, 3) <class 'tuple'>



Las tuplas son ideales para datos **fijos y seguros**.
En las próximas páginas veremos cómo recorrerlas, usar métodos útiles y más ejemplos prácticos.

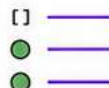




TUPLAS

2. OPERACIONES BÁSICAS CON TUPLAS

Volumen 10



Las tuplas son **inmutables**, pero podemos realizar muchas operaciones **útiles** con ellas: **acceso**, **slicing**, **concatenación**, **repetición** y más.

2.1 ACCEDER POR ÍNDICE

Usamos corchetes `[]` y el índice.

```
países = ("Argentina", "Brasil",  
          "Chile", "Perú")  
  
print(países[0]) # Argentina  
print(países[2]) # Chile
```

Salida:

```
Argentina  
Chile
```

★ Índices negativos

```
países[-1] # último → Perú  
países[-2] # penúltimo → Chile
```

2.2 SLICING (REBANADO)

Obtenemos partes de una tupla.

```
letras = ("a", "b", "c", "d", "e")  
  
print(letras[1:4]) # b, c, d  
print(letras[:3]) # a, b, c  
print(letras[2:]) # c, d, e  
print(letras[-3:-1]) # c, d
```

Salida:

```
('b', 'c', 'd')  
('a', 'b', 'c')  
('c', 'd', 'e')  
('c', 'd')
```

★ El slicing devuelve siempre una **NUEVA** tupla.

2.3 CONCATENACIÓN (+)

Unimos dos o más tuplas.

```
a = (1, 2)  
b = (3, 4)  
c = a + b  
d = a + (5, 6)  
print(c) # (1, 2, 3, 4)  
print(d) # (1, 2, 5, 6)
```

Salida:

```
(1, 2, 3, 4)  
(1, 2, 5, 6)
```

★ Se crea una nueva tupla con todos los elementos.

2.4 REPETICIÓN (*)

Repite los elementos varias veces.

```
t = ("ha", "ha")  
print(t * 3) # ('ha', 'ha',  
              'ha', 'ha', 'ha', 'ha')  
  
nums = (1, 2, 3)  
print(nums * 2) # (1, 2, 3, 1, 2, 3)
```

Salida:

```
('ha', 'ha', 'ha', 'ha', 'ha', 'ha')  
(1, 2, 3, 1, 2, 3)
```

★ Muy útil para patrones o tests.

2.5 PERTENENCIA (in / not in)

Verificamos si un valor está en la tupla.

```
colores = ("rojo", "verde", "azul")  
print("verde" in colores) # True  
print("amarillo" in colores) # False  
print("rojo" not in colores) # False
```

Salida:

```
True  
False  
False
```

2.6 LONGITUD (len)

Obtenemos la cantidad de elementos.

```
frutas = ("manzana", "banana", "uva")  
print(len(frutas)) # 3  
  
nums = (10, 20, 30, 40, 50)  
print(len(nums)) # 5
```

Salida:

```
3  
5
```

2.7 CONTAR ELEMENTOS (count)

Cuenta cuántas veces aparece un valor.

```
datos = (1, 2, 2, 3, 2, 4)  
print(datos.count(2)) # 3  
print(datos.count(5)) # 0
```

Salida:

```
3  
0
```

RECUERDA

Las tuplas no tienen métodos como `append()`, `remove()`, `pop()`, etc. No puedes modificar ni eliminar elementos.



2.8 DESEMPAQUETADO (UNPACKING)

Asignamos cada elemento a una variable.

BÁSICO

```
punto = (10, 20)  
x, y = punto  
print(x) # 10  
print(y) # 20
```

Salida:

```
10  
20
```

CON MÁS VARIABLES

```
info = ("Ana", 25, "AR")  
nombre, edad, país = info  
print(nombre) # Ana  
print(edad) # 25  
print(país) # AR
```

Salida:

```
Ana  
25  
AR
```

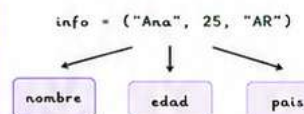
CON * (RESTO DE ELEMENTOS)

```
nums = (1, 2, 3, 4, 5, 6)  
a, b, *resto = nums  
print(a) # 1  
print(b) # 2  
print(resto) # [3, 4, 5, 6]
```

Salida:

```
1  
2  
[3, 4, 5, 6]
```

Ejemplo visual



TIP PRO

El desempaquetado es súper útil para trabajar con valores claros y legibles.



2.9 CONVERTIR A LISTA Y VICEVERSA

Podemos convertir entre listas y tuplas.

DE TUPLA A LISTA

```
tupla = (1, 2, 3)  
lista = list(tupla)  
print(lista) # [1, 2, 3]
```

Salida:

```
[1, 2, 3]
```

DE LISTA A TUPLA

```
lista = ["a", "b", "c"]  
tupla = tuple(lista)  
print(tupla) # ('a', 'b', 'c')
```

Salida:

```
('a', 'b', 'c')
```

2.10 EJEMPLOS PRÁCTICOS

1. Guardar coordenadas

```
coords = (-34.6037, -58.3816)  
lat, lon = coords  
print(lat) # -34.6037  
print(lon) # -58.3816
```

2. Días de la semana

```
dias = ("Lun", "Mar", "Mié",  
        "Jue", "Vie", "Sáb", "Dom")  
print(dias[0]) # Lun  
print(dias[-1]) # Dom  
print(dias[1:5])  
("Mar", "Mié", "Jue", "Vie")
```

3. Repetir patrón

```
patrón = ("*", "-")  
línea = patrón * 5  
print(línea)  
('*', '-', '*', '-',  
 '*', '-', '*', '-',  
 '*', '-')
```

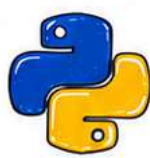


En la próxima página veremos métodos y funciones útiles con tuplas.



Volumen 10 • Tuplas

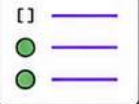




TUPLAS

3. MÉTODOS Y FUNCIONES ÚTILES

Volumen 10



Las tuplas tienen algunos **métodos** y muchas **funciones** incorporadas que facilitan trabajar con ellas.

3.1 MÉTODOS DE TUPLAS

Las tuplas tienen solo 2 métodos incorporados.

Método	¿Qué hace?	Ejemplo	Resultado
<code>count(x)</code>	Cuenta cuántas veces aparece <code>x</code> en la tupla.	<code>t = (1, 2, 2, 3, 2)</code> <code>t.count(2)</code>	3
<code>index(x)</code>	Devuelve el índice de la primera aparición de <code>x</code> .	<code>t = (10, 20, 30, 20)</code> <code>t.index(20)</code>	1

★ `index(x)` lanza un error si `x` no está en la tupla.
`ValueError: tuple.index(x): x not in tuple`

3.2 FUNCIONES INCORPORADAS

- `len(tupla)` → devuelve la cantidad de elementos.
- `max(tupla)` → devuelve el mayor valor.
- `min(tupla)` → devuelve el menor valor.
- `sum(tupla)` → suma los elementos (si son numéricos).
- `sorted(tupla)` → devuelve una lista ordenada.
- `tuple(iterable)` → convierte un iterable en tupla.

Ejemplos

```
t = (4, 1, 7, 2)
len(t)          # 4
min(t)          # 1
max(t)          # 7
sum(t)          # 14
sorted(t)       # [1, 2, 4, 7]
```

3.3 DESEMPAQUETADO (UNPACKING)

Permite asignar cada valor de la tupla a variables individuales.

DESEMPAQUETADO BÁSICO

```
persona = ("Ana", 25, "AR")
nombre, edad, pais = persona
print(nombre) # Ana
print(edad)   # 25
print(pais)   # AR
```

Salida:
Ana
25
AR

CON * (RESTO DE ELEMENTOS)

```
datos = (1, 2, 3, 4, 5)
a, b, *resto = datos
print(a)    # 1
print(b)    # 2
print(resto) # [3, 4, 5]
```

IGNORAR VALORES

```
punto = (10, 20)
x, _ = punto # ignoramos y
print(x)     # 10
```

Salida:
10

★ El guion bajo (`_`) es una convención para decir: "no me importa este valor".

3.4 SLICING EN TUPLAS

Funciona igual que en listas.

```
t = (0, 1, 2, 3, 4, 5)
t[1:4]    # (1, 2, 3)
t[:3]     # (0, 1, 2)
t[::2]    # (0, 2, 4)
t[::-1]   # (5, 4, 3, 2, 1, 0)
```

Salida:
(1, 2, 3) (0, 2, 4)
(0, 1, 2) (5, 4, 3, 2, 1, 0)

3.5 COMPARACIÓN DE TUPLAS

Las tuplas se comparan elemento por elemento.

```
a = (1, 2, 3)
b = (1, 2, 4)
a == b # False
a != b # True
a < b  # True  (3 < 4)
a <= b # True
b > a  # True
```

★ Si los primeros elementos son iguales, compara el siguiente, y así sucesivamente.

3.6 PERTENENCIA (in / not in)

Verifica si un valor está en la tupla.

```
t = ("rojo", "verde", "azul")
"verde" in t # True
"amarillo" in t # False
1 in t      # False
```

★ Búsqueda lineal: recorre la tupla hasta encontrar el valor.

3.7 EJEMPLOS PRÁCTICOS

1. Contar y buscar

```
t = (5, 2, 5, 8, 2, 5)
print(t.count(5)) # 3
print(t.index(8)) # 3
```

2. Estadísticas rápidas

```
notas = (7, 9, 6, 8)
print(len(notas)) # 4
print(max(notas)) # 9
print(min(notas)) # 6
print(sum(notas)) # 30
print(sum(notas)/len(notas)) # 7.5
```

3. Coord. con desemp. y slicing

```
coord = (34.0522, -118.2437, "LA")
lat, lon, ciudad = coord
print(lat) # 34.0522
print(coord[:2]) # (34.0522, -118.2437)
```

TIP PRO

- ✓ Usa tuplas para datos que no deben cambiar.
- ✓ Son ideales para claves de diccionarios.
- ✓ Más rápidas y ligeras que las listas.



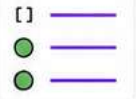
RECUERDA: Las tuplas **NO** permiten **asignación**, **append()**, **remove()**, **pop()**, etc.
Si necesitas modificar datos, convierte a lista: `nueva_lista = list(mi_tupla)`





TUPLAS

Volumen 10



4. RESUMEN GENERAL Y BUENAS PRÁCTICAS



Las tuplas son colecciones **ordenadas** e **inmutables**. Son rápidas, seguras y perfectas para datos que **no deben cambiar**.

4.1 VENTAJAS Y DESVENTAJAS

VENTAJAS ✓	DESVENTAJAS ✗
✓ Inmutables: más seguridad.	✗ No se pueden modificar (ni agregar, ni eliminar).
✓ Más rápidas que las listas.	
✓ Menos memoria que las listas.	
✓ Pueden usarse como claves en diccionarios.	✗ No tienen métodos como <code>append()</code> , <code>remove()</code> , <code>pop()</code> .
✓ Ideales para datos fijos.	✗ Menos flexibles que las listas.

4.2 LISTAS vs TUPLAS

	LISTA (lists)	TUPLA (tuple)
Mutabilidad	Mutable	Inmutable
Métodos	Muchos métodos	Solo <code>count()</code> e <code>index()</code>
Rendimiento	Más lenta	Más rápida
Memoria	Usa más memoria	Usa menos memoria
Claves en dict	No se puede	Sí se puede
Uso típico	Datos que cambian	Datos que no cambian

4.3 PATRONES COMUNES DE USO

1. Retornar múltiples valores

Función que devuelve varios resultados.

```
def calcular(a, b):  
    suma = a + b  
    producto = a * b  
    return suma, producto
```

```
s, p = calcular(3, 4)  
print(s, p) # 7 12
```

★ Las funciones devuelven tuplas automáticamente.

2. Coordenadas (x, y)

Muy usado en gráficos, juegos, mapas, etc.

```
punto = (10, 20)  
jugador = (5, 8)  
meta = (100, 50)
```

```
print(punto[0]) # x  
print(punto[1]) # y
```

★ Más claro que usar listas para datos fijos.

3. Datos de configuración

Guardar configuraciones que no deben cambiar.

```
CONFIG = (  
    "localhost", 3306,  
    "admin", "clave123"  
)
```

```
host, puerto, usuario, clave = CONFIG
```

★ Ideal para constantes y configuraciones.

4. Claves en diccionarios

Tuplas = claves compuestas.

```
coordenadas_colores = {  
    (0, 0): "negro",  
    (1, 0): "rojo",  
    (0, 1): "verde"  
}
```

```
print(coordenadas_colores[(1, 0)])  
# rojo
```

★ Las listas **NO** pueden ser claves de diccionarios.

4.4 BUENAS PRÁCTICAS

- ✓ Usa tuplas para datos que no deben cambiar.
- ✓ Prefiere tuplas para devolver múltiples valores.
- ✓ Si tienes 1 solo elemento, no olvides la coma: `(1,)`
- ✓ Usa nombres descriptivos para que tu código sea claro.
- ✓ Aprovecha que son más rápidas para datos grandes e iteraciones frecuentes.



Regla de oro: si los datos cambian, usa **lista**. Si no cambian, usa **tupla**.

4.5 ERRORES COMUNES

✗ Olvidar la coma en tupla de un solo elemento.

```
t = (1) # NO es tupla  
print(type(t)) # <class 'int'>  
Correcto:  
t = (1,) # Sí es tupla
```

✗ Intentar modificar una tupla.

```
t = (1, 2, 3)  
t[0] = 10 # TypeError
```

✗ Usar métodos de lista en tuplas.

```
t = (1, 2, 3)  
t.append(4) # AttributeError
```

✗ Confundir slicing con índices.

```
t = (1, 2, 3, 4)  
print(t[1:3]) # (2, 3) ✓  
print(t[1, 3]) # ✗ Error
```

4.6 RESUMEN RÁPIDO

Crear	Acceder	Funciones	Métodos	★ Características
<pre>t = () t = (1, 2, 3) t = ("a", 10, True)</pre>	<pre>t[0] # primer elemento t[-1] # último elemento t[1:3] # slicing</pre>	<pre>len(t) max(t) / min(t) sum(t) # si es numérica</pre>	<pre>t.count(x) # cantidad t.index(x) # primer índice</pre>	Ordenadas Inmutables Más rápidas Menos memoria



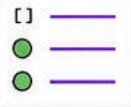
DOMINA LAS TUPLAS y escribe código más **limpio**, **seguro** y **profesional**.





DICCIONARIOS

Volumen 11



1. ¿QUÉ SON LOS DICCIONARIOS?



Los diccionarios (dict) son colecciones de pares **clave-valor**.

Cada clave es **única** e **inmutable** (no puede cambiar).

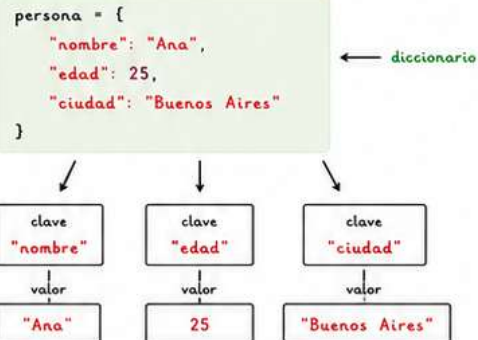
Se usan para almacenar y acceder a datos **relacionados** de forma rápida y clara.

CONCEPTO CLAVE



- ✓ Almacenan datos en pares clave: valor.
- ✓ Las claves deben ser únicas e inmutables:
str, int, float, bool o tuple.
- ✓ Los valores pueden ser de cualquier tipo.
- ✓ Son mutables: podemos agregar, modificar o eliminar elementos.
- ✓ Muy usados para representar datos del mundo real (objetos, registros, configuraciones).

EJEMPLO VISUAL



1.1 CREAR DICCIONARIOS

1. Diccionario vacío

```
d1 = {}  
print(d1)
```

Salida:
{}

2. Con pares clave-valor

```
d2 = {"a": 1, "b": 2, "c": 3}  
print(d2)
```

Salida:
{'a': 1, 'b': 2, 'c': 3}

3. Usando dict()

```
d3 = dict(nombre="Luis",  
          edad=30, pais="AR")  
print(d3)
```

Salida:
{'nombre': 'Luis',
 'edad': 30, 'pais': 'AR'}

4. Con diferentes tipos de valores

```
d4 = {"activo": True, "nota": 9.5,  
     "tags": ["python", "diccionarios"]}  
print(d4)
```

Salida:
{'activo': True, 'nota': 9.5,
 'tags': ['python', 'diccionarios']}

★ Podemos tener valores de cualquier tipo: números, textos, listas, otros diccionarios, etc.

1.2 REGLAS IMPORTANTES



- Las claves deben ser únicas.
Si repetimos una clave, el último valor sobrescribe al anterior.

```
d = {"x": 1, "x": 2}  
print(d) # {'x': 2}
```

- Las claves deben ser inmutables.
(No podemos usar listas como clave)

```
d = {[1, 2]: "lista"} # ❌ Error
```

- Los valores pueden repetirse.

```
d = {"a": 1, "b": 1, "c": 1} # ✅ OK
```

1.3 ¿CUÁNDO USAR DICCIONARIOS?



- ✓ Cuando necesitamos buscar información por una clave (nombre, id, código, etc).
- ✓ Cuando los datos tienen relación entre sí (objeto, persona, producto).
- ✓ Para configuraciones (parámetros, preferencias, ajustes).
- ✓ Para contar frecuencias o agrupar datos.

Ejemplo real

Datos de un producto en una tienda:

```
producto = {  
    "id": 101,  
    "nombre": "Teclado Mecánico",  
    "precio": 85000,  
    "stock": 12  
}
```



1.4 ACCEDER A VALORES



Usamos la clave entre corchetes []:

```
persona = {"nombre": "Ana", "edad": 25}  
print(persona["nombre"]) # Ana  
print(persona["edad"]) # 25
```

Si la clave no existe, dará error **KeyError**.

```
print(persona["telefono"]) # ❌ KeyError
```

Podemos usar `.get()` para evitar errores.

```
print(persona.get("telefono")) # None  
print(persona.get("telefono", "N/D")) # N/D
```

RESUMEN RÁPIDO



¿Qué es?	Colección de pares clave-valor.
Claves	Únicas e inmutables (str, int, float, bool, tuple).
Valores	Pueden ser de cualquier tipo.
Mutabilidad	Se pueden agregar, modificar y eliminar elementos.
Uso principal	Acceso rápido por clave y organización de datos relacionados.

SINTAXIS BÁSICA



```
d = {} # vacío  
d = {"k1": v1, "k2": v2} # con pares  
d["clave"] # acceder  
d["clave"] = valor # modificar / agregar  
del d["clave"] # eliminar  
d.get("clave", valor_def) # obtener seguro  
len(d) # cantidad de pares
```





DICCIONARIOS

Volumen 11

2. TRABAJAR CON DICCIONARIOS



Los diccionarios tienen muchos **métodos** y operaciones **útiles**.
¡Conozcámoslos!

2.1 ACCEDER A VALORES

Usando corchetes []

```
persona = {"nombre": "Ana", "edad": 25, "pais": "AR"}

print(persona["nombre"]) # Ana
print(persona["edad"]) # 25
```

★ NOTA

Si la clave no existe,
KeyError: 'clave'.
Usa `.get()` para evitarlo.

USANDO `.get()`

```
print(persona.get("pais")) # AR
print(persona.get("ciudad")) # None
print(persona.get("ciudad", "No existe")) # No existe
```

2.2 MODIFICAR Y AGREGAR ELEMENTOS

Modificar valor existente

```
persona = {"nombre": "Ana", "edad": 25}
persona["edad"] = 26
print(persona) # {'nombre': 'Ana', 'edad': 26}
```

Agregar nuevo par clave-valor

```
persona["correo"] = "ana@mail.com"
print(persona)
# {'nombre': 'Ana', 'edad': 26, 'correo': 'ana@mail.com'}
```

2.3 ELIMINAR ELEMENTOS

```
d = {"a": 1, "b": 2, "c": 3}

del d["b"] # Elimina clave 'b'
print(d) # {'a': 1, 'c': 3}

d.pop("c") # Elimina y devuelve el valor
print(d) # {'a': 1}

d.pop("x", "No existe") # No genera error
print(d) # {'a': 1}

d.clear() # Vacía el diccionario
print(d) # {}
```

2.4 VERIFICAR CLAVES Y VALORES

```
d = {"a": 1, "b": 2, "c": 3}

print("a" in d) # True
print("x" in d) # False
print("a" not in d) # False

print(1 in d.values()) # True
print(5 in d.values()) # False
```

2.5 OBTENER CLAVES, VALORES

```
d = {"a": 1, "b": 2, "c": 3}

print(d.keys()) # dict_keys(['a', 'b', 'c'])
print(list(d.keys())) # ['a', 'b', 'c']

print(d.values()) # dict_values([1, 2, 3])
print(list(d.values())) # [1, 2, 3]

print(d.items()) # dict_items([('a', 1), ('b', 2), ('c', 3)])
print(list(d.items())) # [('a', 1), ('b', 2), ('c', 3)]
```

2.6 COPIAR DICCIONARIOS

Copia superficial

```
d1 = {"a": 1, "b": 2}
d2 = d1.copy()
d2["a"] = 100
print(d1) # {'a': 1, 'b': 2}
print(d2) # {'a': 100, 'b': 2}
```

Con `dict()`

```
d3 = dict(d1)
d3["b"] = 200
print(d1) # {'a': 1, 'b': 2}
print(d3) # {'a': 1, 'b': 200}
```

2.7 RECORRER DICCIONARIOS

Solo claves (por defecto)

```
d = {"a": 1, "b": 2, "c": 3}
for clave in d:
    print(clave)
# a
# b
# c
```

Claves y valores

```
for clave, valor in d.items():
    print(clave, "->", valor)
# a -> 1
# b -> 2
# c -> 3
```

Solo valores

```
for valor in d.values():
    print(valor)
# 1
# 2
# 3
```

2.8 EJEMPLOS PRÁCTICOS

1. Contar palabras en un texto

```
texto = "python es genial y python es útil"
palabras = {}

for p in texto.split():
    palabras[p] = palabras.get(p, 0) + 1

print(palabras)
# {'python': 2, 'es': 2, 'genial': 1, 'y': 1, 'útil': 1}
```

2. Inventario simple

```
inventario = {"lapiz": 10, "goma": 5}

inventario["lapiz"] -= 2 # se usan 2 lápices
inventario["cuaderno"] = 3 # nuevo item

print(inventario)
# {'lapiz': 8, 'goma': 5, 'cuaderno': 3}
```

3. Configuración de usuario

```
config = {"tema": "oscuro", "idioma": "es",
          "notificaciones": True}

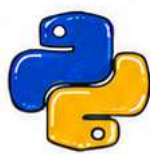
config["idioma"] = "en"
config["sonido"] = False

print(config)
# {'tema': 'oscuro', 'idioma': 'en',
#   'notificaciones': True, 'sonido': False}
```



TIP PRO: Usa `.get()` con valor por defecto para evitar errores y hacer tu código más robusto.





DICCIONARIOS

Volumen 11

3. MÉTODOS ÚTILES DE DICCIONARIOS



Los diccionarios tienen varios **métodos** que facilitan muchas tareas comunes.
¡Conócelos y úsalos a tu favor!

3.1 MÉTODOS MÁS USADOS

Método	¿Qué hace?	Ejemplo	Resultado
<code>keys()</code>	Devuelve una vista de todas las claves.	<code>d = {'a': 1, 'b': 2}</code> <code>d.keys()</code>	<code>dict_keys(['a', 'b'])</code>
<code>values()</code>	Devuelve una vista de todos los valores.	<code>d.values()</code>	<code>dict_values([1, 2])</code>
<code>items()</code>	Devuelve pares (clave, valor) como tuplas.	<code>d.items()</code>	<code>dict_items([('a', 1), ('b', 2)])</code>
<code>update()</code>	Actualiza el diccionario con pares de otro diccionario.	<code>d.update({'c': 3})</code>	<code>{'a': 1, 'b': 2, 'c': 3}</code>
<code>pop(clave)</code>	Elimina la clave y devuelve su valor.	<code>d.pop('a')</code>	<code>1</code> # y d -> <code>{'b': 2}</code>
<code>popitem()</code>	Elimina y devuelve el último par insertado.	<code>d.popitem()</code>	<code>('b', 2)</code> # y d -> <code>{'a': 1}</code>
<code>clear()</code>	Elimina todos los elementos del diccionario.	<code>d.clear()</code>	<code>{}</code>
<code>copy()</code>	Devuelve una copia superficial del diccionario.	<code>d2 = d.copy()</code>	<code>{'a': 1, 'b': 2}</code>
<code>setdefault(clave, valor)</code>	Devuelve el valor de la clave. Si no existe, la crea con valor.	<code>d.setdefault('x', 10)</code> <code>d.setdefault('a', 100)</code>	<code>10</code> # agrega 'x': 10 y luego devuelve 1 (no cambia)

EJEMPLO COMPLETO

```
d = {'a': 1, 'b': 2}
print(d.keys()) # dict_keys(['a', 'b'])
print(list(d.values())) # [1, 2]
print(list(d.items())) # [('a', 1), ('b', 2)]

d.update({'c': 3})
print(d) # {'a': 1, 'b': 2, 'c': 3}

d.setdefault('d', 4)
print(d) # {'a': 1, 'b': 2, 'c': 3, 'd': 4}

print(d.pop('b')) # 2
print(d) # {'a': 1, 'c': 3, 'd': 4}

d.popitem() # ('d', 4)
print(d) # {'a': 1, 'c': 3}

d.clear()
print(d) # {}
```

★ RECUERDA

`keys()`, `values()` e `items()` devuelven vistas (no listas). Usa `list()` si necesitas una lista real.



3.2 RECORRER DICCIONARIOS

Solo claves	Solo valores	Claves y valores
<code>d = {'a': 1, 'b': 2}</code> <code>for clave in d:</code> <code>print(clave)</code> Salida: a b	<code>for valor in d.values():</code> <code>print(valor)</code> Salida: 1 2	<code>for clave, valor in d.items():</code> <code>print(clave, "->", valor)</code> Salida: a -> 1 b -> 2

3.3 COMPRENSIÓN DE DICCIONARIOS

Forma rápida de crear diccionarios.

Ejemplo 1: cuadrado de números

```
cuadrados = {x: x**2 for x in range(1, 6)}
print(cuadrados)
Salida: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Ejemplo 2: invertir claves y valores

```
d = {'a': 1, 'b': 2, 'c': 3}
invertido = {v: k for k, v in d.items()}
print(invertido)
Salida: {1: 'a', 2: 'b', 3: 'c'}
```

3.4 DICCIONARIOS ANIDADOS

Un diccionario puede contener otros diccionarios como valores.

```
usuario = {
    'nombre': 'Ana',
    'edad': 25,
    'contacto': {
        'email': 'ana@mail.com',
        'telefono': '1234-5678'
    }
}
print(usuario['contacto']['email'])
```



3.5 CONTAR FRECUENCIAS (EJEMPLO REAL)

Contar cuántas veces aparece cada palabra en un texto.

```
texto = "python es genial y python es util y python es facil"
palabras = texto.split()
frecuencias = {}
for p in palabras:
    frecuencias[p] = frecuencias.get(p, 0) + 1
print(frecuencias)
Salida: {'python': 3, 'es': 2, 'genial': 1, 'y': 2, 'util': 1, 'facil': 1}
```

RESUMEN RÁPIDO

¿Qué es? Colección de pares clave-valor. Claves únicas e inmutables.	Acceder <code>d['clave']</code> # valor Usa <code>.get()</code> para evitar errores.	Modificar <code>d['clave'] = valor</code> <code>.update()</code> , <code>.pop()</code> , <code>.popitem()</code> , <code>.clear()</code>	Recorrer <code>for clave in d</code> <code>for valor in d.values()</code> <code>for k, v in d.items()</code>	Crear rápido Comprehensions de diccionarios (<code>k: v for ...</code>)	Anidados Diccionarios dentro de otros diccionarios.
--	---	---	--	--	---



¡Los diccionarios son herramientas poderosas para organizar información!





DICCIONARIOS

Volumen 11

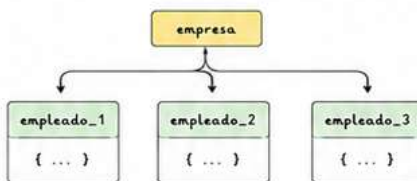
4. DICCIONARIOS ANIDADOS



Los diccionarios pueden contener **otros diccionarios** (o listas) como valores.
Esto permite representar **datos más complejos** y estructurados.

4.1 ¿QUÉ ES UN DICCIONARIO ANIDADO?

Un diccionario anidado es un diccionario donde uno o más valores son otros diccionarios.
Ideal para datos jerárquicos.



4.2 EJEMPLO BÁSICO

```
empresa = {  
    "nombre": "GuilleTech",  
    "ubicacion": "Buenos Aires",  
    "empleados": {  
        "e1": {"nombre": "Ana", "cargo": "Dev", "edad": 25},  
        "e2": {"nombre": "Luis", "cargo": "Data", "edad": 30}  
    }  
}
```

ACCESOS

```
print(empresa["nombre"]) # GuilleTech  
print(empresa["empleados"]["e1"]["cargo"]) # Dev  
print(empresa["empleados"]["e2"]["edad"]) # 30
```

4.3 ACCEDER, MODIFICAR Y AGREGAR

Acceder

```
cargo = empresa["empleados"]["e1"]["cargo"]  
print(cargo) # Dev
```

Modificar

```
empresa["empleados"]["e1"]["edad"] = 26  
  
print(empresa["empleados"]["e1"]["edad"])  
# 26
```

Agregar nuevo empleado

```
empresa["empleados"]["e3"] = {  
    "nombre": "Sofia",  
    "cargo": "QA",  
    "edad": 28  
}  
  
print(empresa["empleados"]["e3"])  
# {'nombre': 'Sofia', 'cargo': 'QA', 'edad': 28}
```

4.4 RECORRER DICCIONARIOS ANIDADOS

Recorrer empleados e imprimir sus datos.

```
for id_emp, datos in empresa["empleados"].items():  
    print(f"ID: {id_emp}")  
    for clave, valor in datos.items():  
        print(f"({clave}): {valor}")
```

Salida:

```
ID: e1 Ana cargo: 26 edad: 36  
ID: e2 Luis nombre: Sofia cargo: QA  
cargo: Data edad: 30 edad: 28
```

4.5 DICCIONARIO ANIDADO CON LISTAS

También podemos tener listas dentro de diccionarios anidados.

```
tienda = {  
    "producto": {  
        "nombre": "Laptop",  
        "precio": 1500,  
        "características": ["16GB RAM", "512GB SSD", "15.6\""],  
        "colores": ["negro", "gris"]  
    }  
}
```

Acceder

```
print(tienda["producto"]["características"][0]) # 16GB RAM  
print(tienda["producto"]["colores"][1]) # gris
```

4.6 COMPRESIÓN CON ANIDADOS

Crear un resumen de empleados (solo nombres y cargos).

```
resumen = {id_emp: {"nombre": datos["nombre"],  
                    "cargo": datos["cargo"]}  
for id_emp, datos in empresa["empleados"].items()}  
print(resumen)
```

Salida:

```
{  
  'e1': {'nombre': 'Ana', 'cargo': 'Dev'},  
  'e2': {'nombre': 'Luis', 'cargo': 'Data'},  
  'e3': {'nombre': 'Sofia', 'cargo': 'QA'}  
}
```

4.7 MÉTODOS ÚTILES EN ANIDADOS

Método	¿Qué hace?	Ejemplo
get()	Accede sin error si no existe.	empresa.get("empleados", {}).get("e1")
setdefault()	Devuelve valor o lo crea.	empresa.setdefault("activo", True)
update()	Actualiza un diccionario interno.	empresa["empleados"]["e1"].update({"edad": 27})
pop()	Elimina clave de un diccionario interno.	empresa["empleados"].pop("e2")

4.8 EJEMPLO PRÁCTICO COMPLETO

Inventario de una biblioteca.

```
biblioteca = {  
    "libros": {  
        "l1": {"titulo": "Python 101", "autor": "J. Doe", "stock": 5},  
        "l2": {"titulo": "Algoritmos", "autor": "Cormen", "stock": 2}  
    }  
}  
  
# Aumentar stock de un libro  
biblioteca["libros"]["l1"]["stock"] += 1  
print(biblioteca["libros"]["l1"]["stock"]) # 6
```



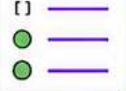
CLAVE: Los diccionarios anidados nos permiten **organizar información compleja** de forma clara y escalable.
¡Son **esenciales** en proyectos reales!





DICCIONARIOS

Volumen 11



5. RESUMEN GENERAL Y BUENAS PRÁCTICAS



Los diccionarios son una de las estructuras más **potentes** de Python. Permiten almacenar datos **relacionados**, acceder **rápidamente** por clave y trabajar con información compleja de forma **clara** y **eficiente**.

5.1 OPERACIONES MÁS COMUNES

Operación	Código	Ejemplo (d)	Resultado
Acceder	<code>d["clave"]</code>	<code>d["a"]</code>	1
Agregar / Modificar	<code>d["clave"] = valor</code>	<code>d["b"] = 2</code>	<code>{'a': 1, 'b': 2}</code>
Eliminar clave	<code>del d["clave"]</code>	<code>del d["a"]</code>	<code>{'b': 2}</code>
Ver todas las claves	<code>d.keys()</code>	<code>d.keys()</code>	<code>dict_keys(['b'])</code>
Ver todos los valores	<code>d.values()</code>	<code>d.values()</code>	<code>dict_values([2])</code>
Ver todos los items	<code>d.items()</code>	<code>d.items()</code>	<code>dict_items([('b', 2)])</code>
Copiar diccionario	<code>d.copy()</code>	<code>d.copy()</code>	<code>{'b': 2}</code>
Limpiar diccionario	<code>d.clear()</code>	<code>d.clear()</code>	<code>{}</code>

5.2 RECORRER DICCIONARIOS

Solo claves

```
for clave in d:
    print(clave) # a, b, c
```

Solo valores

```
for valor in d.values():
    print(valor) # 1, 2, 3
```

Claves y valores

```
for clave, valor in d.items():
    print(clave, "->", valor) # a -> 1
                                # b -> 2
                                # c -> 3
```

5.3 DICCIONARIOS ANIDADOS

```
usuario = {
    "nombre": "Ana",
    "direccion": {
        "calle": "Sempre 123",
        "ciudad": "Buenos Aires",
        "cp": 1428
    },
    "contactos": [
        {"tipo": "cel", "numero": "11-1234-5678"},
        {"tipo": "email", "valor": "ana@mail.com"}
    ]
}

print(usuario["direccion"]["ciudad"]) # Buenos Aires
print(usuario["contactos"][0]["numero"]) # 11-1234-5678
```

5.4 COMPRENSIÓN DE DICCIONARIOS

Crear diccionarios de forma elegante.

Ejemplo: cuadrado de números pares

```
cuadrados = {x: x**2 for x in range(1, 6)}
if x % 2 == 0:
    print(cuadrados)
# {2: 4, 4: 16}
```

Intercambiar claves y valores

```
d = {"x": 1, "y": 2, "z": 3}
invertido = {v: k for k, v in d.items()}
print(invertido)
# {1: 'x', 2: 'y', 3: 'z'}
```

5.5 MÉTODOS ÚTILES

<code>get(clave, defecto)</code>	# Accede sin error
<code>setdefault(clave, valor)</code>	# Asigna si no existe
<code>update(otro_dict)</code>	# Actualiza con otro dict
<code>pop(clave)</code>	# Elimina y devuelve valor
<code>popitem()</code>	# Elimina y devuelve un item
<code>clear()</code>	# Vacía el diccionario
<code>copy()</code>	# Copia superficial

★ Usa `.get()` cuando la clave puede no existir para evitar errores.

5.6 ERRORES COMUNES

- ❌ Acceder a una clave que no existe sin usar `.get()`.
`d["no_existe"]` # `KeyError`
- ❌ Usar listas como claves (no son inmutables).
`d[[1, 2, 3]] = "lista"` # `TypeError`
- ❌ Modificar el diccionario mientras se recorre.
`for k in d: del d[k]` # Evita esto
- ❌ Confundir `.items()` con `.values()` o `.keys()`.

BUENAS PRÁCTICAS

- ✓ Usa nombres descriptivos para claves.
- ✓ Mantén estructuras simples y claras.
- ✓ Usa `.get()` para accesos seguros.
- ✓ Documenta la estructura en proyectos.
- ✓ Evita diccionarios muy anidados si se puede simplificar.



Un buen diccionario hace tu código más legible, rápido y mantenible.

5.7 CASOS DE USO REALES

- Configuraciones de aplicaciones
`{ "tema": "oscuro", "idioma": "es" }`
- Catálogos de productos
`{ "id": 101, "nombre": "Laptop", "precio": 1500 }`
- Usuarios y permisos
`{ "usuario": "ana", "rol": "admin", "activo": True }`
- Conteo de frecuencias
`{ "python": 3, "java": 2, "sql": 1 }`
- Datos de APIs / JSON
`{ "status": "ok", "data": [...] }`

5.8 RESUMEN ULTRA RÁPIDO

Concepto	Descripción breve	Ejemplo	Salida
Diccionario	Colección de pares clave-valor.	<code>d = {"a": 1, "b": 2}</code>	<code>{'a': 1, 'b': 2}</code>
Acceder	Obtener valor por clave.	<code>d["a"]</code>	1
Agregar / Modificar	Asignar valor a una clave.	<code>d["c"] = 3</code>	<code>{'a': 1, 'b': 2, 'c': 3}</code>
Eliminar	Quitar clave y su valor.	<code>del d["b"]</code>	<code>{'a': 1, 'c': 3}</code>
Recorrer	Iterar claves, valores o items.	<code>for k, v in d.items(): ...</code>	<code>(a, 1) (c, 3)</code>
Anidado	Diccionario dentro de otro.	<code>d = {"user": {"id": 1}}</code>	<code>{'user': {'id': 1}}</code>
Comprensión	Crear diccionarios rápidamente.	<code>{x: x**2 for x in [1, 2]}</code>	<code>{1: 1, 2: 4}</code>
Métodos útiles	Herramientas para manipular.	<code>d.get("a"), d.copy(), d.update(...)</code>	Mejora seguridad y código

PALABRAS CLAVE

- clave
- valor
- par
- inmutable
- métodos
- anidado
- items
- seguridad
- estructura
- organización



DOMINA LOS DICCIONARIOS y podrás manejar información compleja como un PRO!





SETS

1. ¿QUÉ SON LOS SETS?



Los sets (conjuntos) son colecciones de elementos **únicos** e **inmutables**.

No permiten elementos repetidos y **no tienen un orden** específico.

Se usan para eliminar duplicados y para **operaciones matemáticas** de conjuntos.

CONCEPTO CLAVE

- ✓ Cada elemento es único (no hay duplicados).
- ✓ No tienen orden: los elementos pueden cambiar de posición.
- ✓ Son mutables: podemos agregar y eliminar elementos.
- ✓ Aceptan elementos de cualquier tipo inmutable (números, strings, tuplas, etc.).
- ✓ No se accede por índice o posición.

EJEMPLO VISUAL

```
lista_con_duplicados = [1, 2, 2, 3, 4, 4, 5, 1]
```

Convertimos a set

```
mi_set = {1, 2, 3, 4, 5}
```

- Se eliminaron los duplicados.
- El orden puede cambiar.

Representación

```
{1, 2, 3, 4, 5}
```

set

1.1 CREAR SETS

1. Con llaves {}

```
s1 = {1, 2, 3, 4}
s2 = {'a', 'b', 'c'}
s3 = {1, 'hola', 3.14}
```

Salida:

```
{1, 2, 3, 4}
{'a', 'b', 'c'}
{1, 'hola', 3.14}
```

2. Con set()

```
s4 = set([1, 2, 2, 3, 4])
s5 = set('abracadabra')
```

Salida:

```
{1, 2, 3, 4}
{'a', 'b', 'r', 'c', 'd'}
```

3. Set vacío

```
s6 = set() # Correcto
```

Salida:

```
set()
```

⚠ {} crea un diccionario vacío, no un set.

4. No permite duplicados

```
s7 = {1, 1, 2, 2, 3, 3}
```

Salida:

```
{1, 2, 3}
```

1.2 CARACTERÍSTICAS IMPORTANTES

123

Elementos únicos

No permite valores repetidos.



Sin orden

El orden de los elementos puede variar.



Mutable

Se pueden agregar y eliminar elementos.



Elementos inmutables

Los elementos del set deben ser inmutables (números, strings, tuplas, etc.).

¿Qué no se puede hacer?

- ✗ Acceder por índice: `s[0]` ✗
- ✗ Tener elementos duplicados: `{1,1}` ✗
- ✗ Usar listas o diccionarios como elementos: `{{1,2}}` ✗



TIP: Usa sets cuando necesites almacenar elementos **únicos** y no te importe el orden.

1.3 EJEMPLOS PRÁCTICOS

Ejemplo 1: Eliminar duplicados

```
numeros = [1, 2, 2, 3, 4, 4, 5, 1]
unicos = set(numeros)
print(unicos)
```

Salida:

```
{1, 2, 3, 4, 5}
```

Ejemplo 2: Verificar pertenencia

```
letras = {'a', 'e', 'i', 'o', 'u'}
print('a' in letras) # True
print('b' in letras) # False
```

Salida:

```
True
False
```

Ejemplo 3: Contar elementos únicos

```
palabra = 'programacion'
letras_unicas = set(palabra)
print(letras_unicas)
print(len(letras_unicas))
```

Salida:

```
('p', 'r', 'o', 'g', 'a', 'm', 'c', 'i', 'n')
9
```

Los sets son perfectos para trabajar con datos únicos y comparar colecciones.





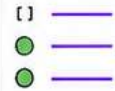
SETS

Volumen 12

2. OPERACIONES CON SETS



Las operaciones con sets son muy **potentes** y **eficientes**.
¡Ideales para resolver problemas reales!



2.1 UNIÓN (|)

Combina todos los elementos de ambos sets (sin duplicados).

```
a = {1, 2, 3}
b = {3, 4, 5}
union = a | b
# También: a.union(b)
```

Salida:
{1, 2, 3, 4, 5}

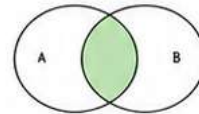
2.2 INTERSECCIÓN (&)

Devuelve los elementos comunes a ambos sets.

```
a = {1, 2, 3}
b = {2, 3, 4, 5}
inter = a & b
# También: a.intersection(b)
```

Salida:
{2, 3}

Representación:



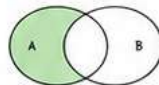
2.3 DIFERENCIA (-)

Elementos que están en el primer set y no en el segundo.

```
a = {1, 2, 3}
b = {2, 3, 4, 5}
dif = a - b
# También: a.difference(b)
```

Salida:
{1}

Representación:

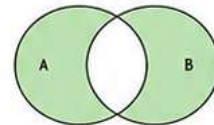


2.4 DIFERENCIA SIMÉTRICA (^)

Elementos que están en uno u otro set, pero no en ambos.

```
a = {1, 2, 3}
b = {2, 3, 4, 5}
sim = a ^ b
# También: a.symmetric_difference(b)
```

Representación:



2.5 SUBCONJUNTOS Y SUPERCONJUNTOS

Relación	Significado	Ejemplo	Resultado	Relación	Significado	Ejemplo	Resultado
Subconjunto (<=)	Todos los elementos de A están en B.	a = {1, 2} b = {1, 2, 3, 4} a <= b	True	Superconjunto (>=)	Todos los elementos de B están en A.	a = {1, 2, 3, 4} b = {1, 2} a >= b	True
Subconjunto propio (<)	A está en B y no son iguales.	a = {1, 2} b = {1, 2, 3} a < b	True	Superconjunto propio (>)	A contiene a B y no son iguales.	a = {1, 2, 3} b = {1, 2} a > b	True

★ También existen:

```
isdisjoint() # No tienen elementos en común
a = {1, 2}
b = {3, 4}
a.isdisjoint(b) # True
```

```
issubset() -> a.issubset(b) es lo mismo que a <= b
issuperset() -> a.issuperset(b) es lo mismo que a >= b
```

2.6 MÉTODOS ÚTILES CON SETS

Método	Descripción	Ejemplo rápido
add(x)	Agrega un elemento.	s = {1, 2, 3}
update(iterable)	Agrega varios elementos.	s.add(4) # {1, 2, 3, 4}
remove(x)	Elimina x (error si no existe).	s.update([5, 6]) # {1, 2, 3, 4, 5, 6}
discard(x)	Elimina x (sin error si no existe).	s.discard(2) # {1, 3, 4, 5, 6}
pop()	Elimina y devuelve un elemento aleatorio.	x = s.pop() # elimina uno (aleatorio)
clear()	Vacía el set.	s.clear() # set()
copy()	Devuelve una copia superficial.	



2.7 CASOS DE USO REALES



Eliminar duplicados de una lista
items = [1, 2, 2, 3, 4, 4, 5]
unicos = set(items)
{1, 2, 3, 4, 5}



Verificar permisos o accesos
permisos_usuario = {'leer', 'editar'}
permisos_requeridos = {'leer'}
permisos_requeridos <= permisos_usuario # True



Comparar datos
nuevos = {'a', 'b', 'c'}
existentes = {'b', 'c', 'd'}
nuevos ^ existentes # {'a', 'd'}
Diferencia simétrica



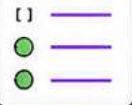
TIP PRO: Los sets son ideales para verificar pertenencia (in) rápidamente.
¡Mucho más rápidos que las listas para búsquedas!





SETS

Volumen 12



3. MÉTODOS Y FUNCIONES CON SETS



Los sets tienen métodos y funciones muy útiles para manipular y consultar conjuntos de datos.

3.1 MÉTODOS PRINCIPALES DE SETS

Método	Descripción	Ejemplo	Resultado
<code>add(x)</code>	Agrega un elemento al set.	<code>s = {1, 2, 3}</code> <code>s.add(4)</code>	<code>{1, 2, 3, 4}</code>
<code>update(iterable)</code>	Agrega varios elementos. Acepta cualquier iterable.	<code>s = {1, 2}</code> <code>s.update([3, 4, 5])</code>	<code>{1, 2, 3, 4, 5}</code>
<code>remove(x)</code>	Elimina el elemento <code>x</code> . Si no existe, da error.	<code>s = {1, 2, 3}</code> <code>s.remove(2)</code>	<code>{1, 3}</code>
<code>discard(x)</code>	Elimina el elemento <code>x</code> . Si no existe, no hace nada.	<code>s = {1, 2, 3}</code> <code>s.discard(10)</code>	<code>{1, 2, 3}</code>
<code>pop()</code>	Elimina y devuelve un elemento aleatorio del set.	<code>s = {1, 2, 3}</code> <code>e = s.pop()</code>	<code>e = 1</code> (o <code>2</code> o <code>3</code>) <code>s</code> queda con 2 elementos
<code>clear()</code>	Elimina todos los elementos del set.	<code>s = {1, 2, 3}</code> <code>s.clear()</code>	<code>set()</code>
<code>copy()</code>	Devuelve una copia superficial del set.	<code>s2 = s.copy()</code>	<code>s2</code> es una copia de <code>s</code>



IMPORTANTE

Los sets no tienen índice. No podemos acceder a elementos por posición.

El método `pop()` elimina un elemento aleatorio.

Para recorrer, usamos bucles `for`.



3.2 FUNCIONES ÚTILES DE SETS

Función	Descripción	Ejemplo	Resultado
<code>len(s)</code>	Devuelve la cantidad de elementos.	<code>s = {1, 2, 3, 3}</code> <code>len(s)</code>	<code>3</code>
<code>x in s</code>	Verifica si <code>x</code> está en el set.	<code>2 in {1, 2, 3}</code>	<code>True</code>
<code>x not in s</code>	Verifica si <code>x</code> NO está en el set.	<code>5 not in {1, 2, 3}</code>	<code>True</code>
<code>min(s)</code>	Devuelve el menor elemento (numérico o alfabético).	<code>min({4, 2, 7})</code>	<code>2</code>
<code>max(s)</code>	Devuelve el mayor elemento (numérico o alfabético).	<code>max({4, 2, 7})</code>	<code>7</code>

3.3 CONVERSIONES



- De lista a set (elimina duplicados):

```
lista = [1, 2, 2, 3, 4, 4]
s = set(lista)  # {1, 2, 3, 4}
```

- De set a lista (orden no garantizado):

```
s = {3, 1, 2}
l = list(s)  # [3, 1, 2] (puede variar)
```

- De set a tupla:

```
t = tuple(s)  # (3, 1, 2) (puede variar)
```

3.4 EJEMPLOS PRÁCTICOS

Ejemplo 1: Eliminar duplicados de una lista

```
numeros = [1, 2, 2, 3, 4, 4, 5]
sin_dup = set(numeros)
print(sin_dup)
```

Salida:

```
{1, 2, 3, 4, 5}
```



Ejemplo 2: Palabras únicas en un texto

```
texto = "python es facil y python es muy util"
palabras = texto.split()
unicas = set(palabras)
print(unicas)
```

Salida:

```
{'python', 'es', 'facil', 'y', 'muy', 'util'}
```



Ejemplo 3: Intersección de intereses entre dos grupos

```
grupo_a = {'futbol', 'musica', 'cine', 'viajes'}
grupo_b = {'musica', 'lectura', 'cine', 'arte'}
comunes = grupo_a & grupo_b
print(comunes)
```

Salida:

```
{'musica', 'cine'}
```



Los sets son muy útiles para:

- eliminar duplicados
- comprobar pertenencia rápida
- hacer operaciones de conjuntos
- resolver problemas de lógica y filtros



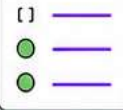
Volumen 12

• Sets



SETS

Volumen 12



4. RESUMEN GENERAL Y BUENAS PRÁCTICAS



Los sets son una herramienta **poderosa** para manejar colecciones de elementos **únicos**.
Úsalos cuando necesites **eficiencia**, **claridad** y operaciones de **conjuntos**.

4.1 RESUMEN RÁPIDO

+	Crear set(), {}	Crea sets. Usa {} solo para vacío.
U	Unión ()	Combina elementos de ambos sets.
∩	Intersección (&)	Elementos comunes.
-	Diferencia (-)	Elementos del primero que no están en el segundo.
Δ	Diferencia simétrica (^)	Elementos que están en uno u otro, pero no en ambos.
⊆	Subconjunto (≤)	Todos los elementos están contenidos.
⊇	Superconjunto (≥)	Contiene a otro set.
=	Igualdad (==)	Tienen los mismos elementos.
≠	Desigualdad (!=)	No tienen los mismos elementos.

4.2 CUÁNDO USAR SETS

- ✓ Eliminar duplicados de una lista.
- ✓ Comprobar pertenencia de forma rápida.
- ✓ Realizar operaciones entre conjuntos.
- ✓ Validar datos únicos (IDs, nombres, etc.).
- ✓ Resolver problemas de lógica y filtros.

4.3 COMPLEJIDAD APROXIMADA

Operación	Tiempo promedio	Notas
Agregar (add)	O(1)	Promedio
Eliminar (remove, discard)	O(1)	Promedio
Buscar (in)	O(1)	Promedio
Unión ()	O(n + m)	n y m = tamaños
Intersección (&)	O(min(n, m))	Más eficiente
Diferencia (-)	O(n)	n = tamaño del set
Recorrer (for)	O(n)	n = tamaño del set

4.4 COMPARACIÓN: LISTA vs SET

LISTA	SET
• Permite duplicados. • Mantiene el orden de inserción. • Acceso por índice. • Búsqueda: O(n) • Ideal para datos ordenados o con posiciones.	• No permite duplicados. • No garantiza orden. • No tiene índice. • Búsqueda: O(1) promedio • Ideal para datos únicos y operaciones de conjuntos.

4.5 BUENAS PRÁCTICAS

- ★ Usa nombres descriptivos para tus sets.
- ★ No conviertas innecesariamente de lista a set y viceversa.
- ★ Recuerda: los sets **no son indexables**.
- ★ Para convertir a lista (si necesitas orden): `list(mi_set)`
- ★ Evita modificar un set mientras lo recorres.
- ★ En problemas de pertenencia frecuente, usa set.

4.6 EJEMPLO INTEGRADOR

Objetivo:

Dada una lista de usuarios, eliminar duplicados, contar únicos y encontrar intereses comunes entre dos grupos.

Código:

```
usuarios = ['Ana', 'Luis', 'Ana', 'Sofia', 'Luis', 'Pedro']
usuarios_unicos = set(usuarios) # eliminar duplicados
print(usuarios_unicos)          # {'Ana', 'Sofia', 'Luis', 'Pedro'}
print(len(usuarios_unicos))     # 4 usuarios únicos

grupo_a = {'futbol', 'musica', 'cine', 'viajes'}
grupo_b = {'musica', 'lectura', 'cine', 'arte'}
comunes = grupo_a & grupo_b # intereses comunes
solo_a = grupo_a - grupo_b # solo en A
solo_b = grupo_b - grupo_a # solo en B
print(comunes)              # {'musica', 'cine'}
print(solo_a)                # {'futbol', 'viajes'}
print(solo_b)                # {'lectura', 'arte'}
```

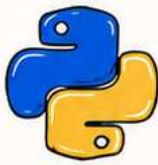
Salida esperada:

```
{'Ana', 'Sofia', 'Luis', 'Pedro'}
4 usuarios únicos
{'musica', 'cine'}
{'futbol', 'viajes'}
{'lectura', 'arte'}
```



Con sets resolvemos tareas complejas de forma simple, rápida y elegante.





STRINGS

1. ¿QUÉ SON LAS STRINGS?



Las strings (cadenas de texto) son secuencias de caracteres que se escriben entre comillas simples (' ') o dobles (" "). Pueden contener letras, números, espacios y símbolos. Son inmutables: una vez creadas, no se pueden modificar. Se usan para almacenar y manipular texto.

CONCEPTO CLAVE

- ✓ Son inmutables.
- ✓ Pueden estar entre comillas simples o dobles.
- ✓ Pueden contener cualquier carácter.
- ✓ Permiten muchas operaciones útiles.
- ✓ Muy usadas en la vida real.



EJEMPLO VISUAL

```
mensaje = "Hola, mundo!"
```

↑
variable

↑
string (texto)

- Entre comillas → es un string.
- Todo lo que esté entre comillas se toma como texto.

1.1 CREAR STRINGS

1. Con comillas simples

```
texto1 = 'Hola Python'  
texto2 = '1234'
```

2. Con comillas dobles

```
texto3 = "Aprendiendo Python"  
texto4 = "GuilleGiu Notes" ♥
```



Puedes usar comillas simples o dobles indistintamente, pero sé consistente en tu código.



1.2 EJEMPLOS PRÁCTICOS

Ejemplo 1: Mensaje simple

```
saludo = "¡Hola, bienvenido!"  
print(saludo)
```

Salida:

¡Hola, bienvenido!

Ejemplo 2: Con números

```
dato = "Tengo 21 años"  
print(dato)
```

Salida:

Tengo 21 años

Ejemplo 3: Con espacios y símbolos

```
frase = "Python es genial :)"  
print(frase)
```

Salida:

Python es genial :)

Ejemplo 4: Combinación

```
nombre = "GuilleGiu"  
curso = "Python"  
print("Estoy aprendiendo " +  
      curso + " con " + nombre)
```

Salida:

Estoy aprendiendo Python con GuilleGiu

Las strings son la base para trabajar con texto en Python.





STRINGS

Volumen 13

2. CREAR Y ACCEDER A STRINGS

2.1 CREAR STRINGS

Puedes crear strings usando comillas **simples**, **dobles** o **triples**.

```
s1 = 'Hola'           # comillas simples
s2 = "Mundo"          # comillas dobles
s3 = '''Hola Python''' # triples comillas
```



Usa triples comillas para textos largos o de varias líneas.



Ejemplo multilinea

```
mensaje = """
Hola Guille,
esta es una string
de varias líneas.
Aprendamos Python juntos!
"""
```



2.2 ACCEDER A CARACTERES (INDEXING)

Cada carácter en un string tiene una posición (índice).
Los índices empiezan en 0.

```
fruta = "Python"
```

índices:

0	1	2	3	4	5
P	y	t	h	o	n
-6	-5	-4	-3	-2	-1

```
fruta[0] # 'P' → primer carácter
```

```
fruta[3] # 'h'
```

```
fruta[-1] # 'n' → último carácter
```

```
fruta[-2] # 'o'
```

TIP



Usa índices negativos para contar desde el final.



2.3 OBTENER SUBSTRINGS (SLICING)

Podemos obtener partes de un string usando [inicio:fin].
Incluye inicio, excluye fin.

```
texto = "Aprender Python es genial"
```

```
texto[0:8]   # 'Aprender'
texto[9:15]  # 'Python'
texto[16:18] # 'es'
texto[19:]   # 'genial' (desde 19 al final)
texto[:8]    # 'Aprender' (desde el inicio)
texto[:]     # copia completa del string
```

Recordatorio rápido

- inicio: desde dónde empezar.
- fin: hasta dónde llegar (sin incluir).
- Si omites inicio, empieza en 0.
- Si omites fin, llega hasta el final.



2.4 STRING IMMUTABLES



Una vez creada, una string **NO** se puede cambiar.



```
saludo = "Hola"
saludo[0] = "h" ❌
# TypeError: 'str' object does not
# support item assignment
```



Si necesitas modificar un texto, crea uno nuevo:

```
nuevo = saludo.replace("H", "h")
# 'hola'
```



Volumen 13 • Strings

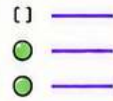
➤ Designed & Summarized by [GuilleGiu](#) ⇐

DEDICADO Y HECHO CON AMOR ❤️



STRINGS

Volumen 13



3. OPERACIONES BÁSICAS CON STRINGS



Las strings son **muy poderosas** y permiten muchas operaciones útiles. Aquí veremos las más comunes que usarás **todo el tiempo**.



3.1 CONCATENACIÓN

Unimos dos o más strings con el operador +.

```
nombre = "Guille"
curso = "Python"
mensaje = "Hola, " + nombre + ", estás
          aprendiendo " + curso + "!"
print(mensaje)
```

Salida:

Hola, Guille, estás aprendiendo Python!



3.2 REPETICIÓN

Repetimos un string varias veces con *.

```
risa = "ja" * 4
guiones = "-" * 10
print(risa)
print(guiones)
```

Salida:

jajajaja



3.3 LONGITUD

Con la función `len()` obtenemos la cantidad de caracteres.

```
texto = "Aprender Python es genial"
largo = len(texto)
print(largo)
```

Salida:

24



TIP



`len()` cuenta todos los caracteres:

- Letras
- Números
- Espacios
- Símbolos

¡Todo cuenta!



3.4 PERTENENCIA (in / not in)

Verificamos si un carácter o subcadena está dentro de una string.

```
frase = "Python es increíble"
"Python" in frase # True
"increíble" in frase # True
"Java" in frase # False
```

```
"Python" not in frase # False
"Java" not in frase # True
" " in frase # True(hay espacios)
"!" in frase # False
```

Recuerda

`in` y `not in` devuelven `True` o `False`. Muy útiles en condicionales.



3.5 COMPARACIÓN

Podemos comparar strings usando operadores de comparación.

```
a = "Python"
b = "python"
c = "Python"
a == b # False (mayúsculas ≠ minúsculas)
a == c # True
a != b # True
a < b # Comparación alfabética
```



La comparación alfabética se hace letra por letra según el orden del diccionario. Las mayúsculas y minúsculas son diferentes.

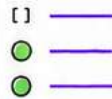




STRINGS

Volumen 13

4. MÉTODOS DE STRINGS



Los strings tienen muchos métodos que nos permiten hacer tareas útiles *sin tener que escribir mucho código*.

Un *método* es una función que pertenece a un *string*.

No modifican el string original, sino que *devuelven uno nuevo*.



4.1 MÉTODOS MÁS USADOS

Aquí tienes algunos métodos muy útiles:

`.upper()`

Convierte todas las letras a mayúsculas.

Ejemplo:

```
texto = "hola mundo"
print(texto.upper()) # HOLA MUNDO
```

`.lower()`

Convierte todas las letras a minúsculas.

Ejemplo:

```
texto = "HOLA MUNDO"
print(texto.lower()) # hola mundo
```

`.capitalize()`

Convierte la primera letra a mayúscula y el resto a minúsculas.

Ejemplo:

```
texto = "hola mundo"
print(texto.capitalize()) # Hola mundo
```

`.strip()`

Elimina espacios en blanco al inicio y al final del string.

Ejemplo:

```
texto = " hola mundo "
print(texto.strip()) # hola mundo
```

`.replace( viejo, nuevo )`

Reemplaza una parte del string.

Ejemplo:

```
texto = "Python es genial"
nuevo = texto.replace(" genial", " increíble")
print(nuevo) # Python es increíble
```

`.split()`

Divide el string en una lista usando un separador (por defecto, espacios).

Ejemplo:

```
texto = "Python es fácil"
print(texto.split()) # ['Python', 'es', 'fácil']
```



- Los métodos siempre devuelven un *nuevo string* o lista, no cambian el original.
- Puedes encadenar métodos (usarlos uno después de otro).
- Son muy útiles para limpiar y transformar texto en nuestros programas.



EJERCICIO RÁPIDO

Prueba estos métodos en tu código:



- Convierte "hola mundo" a mayúsculas.
- Elimina espacios de " Aprender Python ".
- Reemplaza "Python" por "Python es genial".



4.2 EJEMPLOS PRÁCTICOS

`# upper()`

```
texto = "hola"
print(texto.upper())
# HOLA
```

`# lower()`

```
texto = "HOLA"
print(texto.lower())
# hola
```

`# strip()`

```
texto = " hola "
print(texto.strip())
# hola
```

`# replace()`

```
texto = "Python es genial"
print(texto.replace(" genial",
    " increíble"))
# Python es increíble
```



RECUERDA

Practica estos métodos en diferentes frases y observa cómo transforman tus strings. ¡Así se aprende mejor!



TIP EXTRA

Combina métodos:

```
texto = " hola mundo "
print(texto.strip().capitalize()) # Hola mundo
```

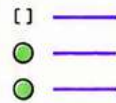




STRINGS

Volumen 13

5. EXTRA: ESCAPES Y F-STRINGS



Hay caracteres especiales que no podemos escribir directamente en un string.

Para eso usamos **secuencias de escape**.

También veremos las **f-strings**, que nos permiten insertar variables dentro de un string de forma simple y elegante.



5.1 SECUENCIAS DE ESCAPE COMUNES

Secuencia	Significado	Ejemplo
<code>\n</code>	Nueva línea	"Hola\nMundo"
<code>\t</code>	Tabulación (tab)	"Hola\tPython"
<code>\\</code>	Barra inversa	"C:\\Users\\Guille"
<code>\"</code>	Comillas dobles	"Él dijo: \"Hola\""
<code>\'</code>	Comillas simples	'It\'s Python'
<code>\r</code>	Retorno de carro	"Hola\rMundo"
<code>\b</code>	Retroceso (backspace)	"Python\b!"
<code>\f</code>	Salto de página	"Página 1\fPágina 2"

EJEMPLO VISUAL

```
texto = "Primera línea\nSegunda línea"
print(texto)
```

Salida:

Primera línea
Segunda línea

★ Los escapes hacen que el string se muestre como queremos, agregando saltos de línea, tabulaciones, comillas, etc.

5.2 F-STRINGS (CADENAS FORMATEADAS)

Las **f-strings** (formatted strings) permiten insertar variables o expresiones directamente dentro del string usando llaves `{}`.

```
nombre = "Guille"
lenguaje = "Python"
nivel = 13

mensaje = f"Hola {nombre}, estás aprendiendo {lenguaje}."
mensaje += f" ¡Vas por el volumen {nivel}! 🐍"
print(mensaje)
```

Salida:

Hola Guille,
estás aprendiendo Python.
¡Vas por el volumen 13! 🐍

★ Dentro de las llaves `{}` puedes poner variables o incluso operaciones:
`f"El doble de 5 es {5*2}"` → "El doble de 5 es 10"

5.3 RAW STRINGS (R-STRINGS)

Si no quieres que Python interprete los escapes, usa una **r** antes de las comillas.

```
ruta = r"C:\Users\Guille\Documentos\proyecto"
print(ruta)
```

Salida:

C:\Users\Guille\Documentos\proyecto

5.4 STRINGS MULTILÍNEA

Puedes usar triple comillas para crear strings de varias líneas fácilmente.

```
poema = """Rosas son rojas,
Python es genial,
y este camino lo vamos a lograr."""
print(poema)
```

Salida:

Rosas son rojas,
Python es genial,
y este camino lo vamos a lograr.

★ Las strings son tu herramienta para comunicarte con el programa y con los usuarios. Domínalas y podrás hacer programas más claros, limpios y profesionales.





STRING METHODS

1. ¿QUÉ SON LOS MÉTODOS DE STRING?



Los métodos de string son funciones que pertenecen a un string y nos permiten hacer **muchas operaciones útiles**.

No modifican el string original (**inmutables**), sino que **devuelven un nuevo string**.

¡Son muy importantes en Python!



CONCEPTO CLAVE

- ✓ Se usan con la sintaxis: `string.metodo()`
- ✓ Siempre devuelven un nuevo string.
- ✓ No cambian el string original.
- ✓ Son muy legibles y fáciles de encadenar.
- ✓ Hay métodos para transformar, buscar, reemplazar, validar y más.



EJEMPLO VISUAL

```
texto = "Aprender Python es genial"
      ↓
nuevo = texto.upper()
      ↓
print(nuevo)
```

Salida:

APRENDER PYTHON ES GENIAL



1.1 CATEGORÍAS DE MÉTODOS

Podemos agrupar los métodos de string en varias categorías:

T Transformación

- `upper()`
- `lower()`
- `capitalize()`
- `title()`
- `swapcase()`

Cambian el caso del texto.

🔍 Búsqueda

- `find()`
- `index()`
- `count()`
- `startswith()`
- `endswith()`

Buscan posiciones o cuentan apariciones.

✎ Modificación

- `replace()`
- `strip()`
- `lstrip()`
- `rstrip()`

Reemplazan o eliminan caracteres.

🛡️ Verificación

- `isalpha()`
- `isdigit()`
- `isalnum()`
- `isspace()`
- `istitle()`

Verifican el contenido del string.

☰ Alineación

- `center()`
- `ljust()`
- `rjust()`
- `zfill()`

Alinean o rellenan el texto.

1.2 EJEMPLOS RÁPIDOS

upper() y lower()

```
texto = "Python"
texto.upper() # "PYTHON"
texto.lower() # "python"
```



replace()

```
frase = "Hola mundo"
frase.replace("mundo", "Python")
# "Hola Python"
```



strip()

```
s = " hola "
s.strip()
# "hola"
```



startswith()

```
s = "Aprender Python"
s.startswith("Apr")
# True
```

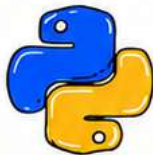


Puedes encadenar métodos:

```
texto = " aprender PYTHON "
texto.strip().lower().replace("python", "Python")
# "Aprender Python"
```

Los métodos de string te ahorran mucho código y hacen que tus programas sean más limpios y profesionales.





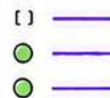
STRING METHODS

Volumen 14

2. TRANSFORMAR TEXTO



Los métodos de transformación nos permiten cambiar el texto a **mayúsculas**, **minúsculas**, quitar espacios y más.
¡Ideales para limpiar y dar formato a nuestras strings!



2.1 MAYÚSCULAS Y MINÚSCULAS

.upper()

Convierte todo el string a mayúsculas.

```
texto = "hola mundo"
print(texto.upper())
# HOLA MUNDO
```

**.lower()**

Convierte todo el string a minúsculas.

```
texto = "HOLA MUNDO"
print(texto.lower())
# hola mundo
```

**.capitalize()**

Convierte la primera letra a mayúscula y el resto a minúsculas.

```
texto = "hola mundo"
print(texto.capitalize())
# Hola mundo
```

2.2 QUITAR ESPACIOS

.strip()

Elimina espacios al inicio y al final del string.

```
texto = " hola mundo "
print(texto.strip())
# 'hola mundo'
```

.lstrip()

Elimina espacios solo al inicio.

```
texto = " hola mundo"
print(texto.lstrip())
# 'hola mundo'
```

**.rstrip()**

Elimina espacios solo al final.

```
texto = "hola mundo "
```

```
print(texto.rstrip())
# 'hola mundo'
```

TIP



Muy útil cuando trabajamos con datos de usuarios o archivos.



2.3 REEMPLAZAR, CONTAR Y BUSCAR

.replace(viejo, nuevo)

Reemplaza una parte del string.

```
texto = "Python es genial"
nuevo = texto.replace("genial", "increíble")
print(nuevo)
# Python es increíble
```

**.count(sub)**

Cuenta cuántas veces aparece una subcadena.

```
texto = "banana"
print(texto.count("an"))
# 2
```

**.find(sub)**

Devuelve el índice de la primera aparición. Si no existe, devuelve -1.

```
texto = "Hola Python"
print(texto.find("Python"))
# 5
```



Usa find() si necesitas saber la posición. Ideal para validaciones.



2.4 DIVIDIR Y UNIR STRINGS

.split(separador)

Divide el string en una lista usando el separador.

```
texto = "uno,dos,tres"
lista = texto.split(",")
print(lista)
# ['uno', 'dos', 'tres']
```

.join(iterable)

Une los elementos de un iterable en un string usando el separador.

```
lista = ['Python', 'es', 'genial']
texto = "-".join(lista)
print(texto)
# Python-es-genial
```



Primero dividimos, luego podemos modificar y finalmente unir de nuevo.
¡Muy poderoso! 🧡



2.5 MÉTODOS DE VERIFICACIÓN

.isalnum()

Solo letras y números.

```
"abc123".isalnum()
# True
```

.isalpha()

Solo letras.

```
"python".isalpha()
# True
```

.isdigit()

Solo números.

```
"12345".isdigit()
# True
```

.isspace()

Solo espacios.

```
" ".isspace()
# True
```

.istitle()

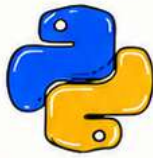
Cada palabra inicia con mayúscula.

```
"Hola Mundo".istitle()
# True
```



Perfectos para validar entradas o limpiar datos.





STRING METHODS

Volumen 14

3. BUSCAR Y VERIFICAR



Muchos métodos nos ayudan a **buscar** partes de texto o **verificar** si algo existe dentro de una string.
¡Muy útiles para **validar** datos y tomar **decisiones**!



3.1 BUSCAR POSICIONES

.find(sub)

Devuelve el índice de la primera aparición de sub.
Si no existe, devuelve -1.

```
texto = "Hola Python"
pos = texto.find("Python")
print(pos) # 5
no = texto.find("Java")
print(no) # -1
```



.index(sub)

Igual que find(), pero si no existe, lanza un error.

```
texto = "Hola Python"
pos = texto.index("Python")
print(pos) # 5
# texto.index("Java")
# ValueError: substring not found
```



.rfind(sub)

Busca desde el final del string hacia el principio.

```
texto = "ana banana anana"
pos = texto.rfind("ana")
print(pos) # 10
```

```
0          10          15
|-----|-----|
  ↑         ↑
  |         |
```



3.2 VERIFICAR CONTENIDO

.count(sub)

Cuenta cuántas veces aparece sub.

```
texto = "banana"
c = texto.count("an")
print(c) # 2
```



.in / .not in

Verifica si una subcadena existe en el string.

```
texto = "Hola Mundo"
"Hola" in texto # True
"Java" not in texto # True
```



.startswith(sub)

Verifica si el string empieza con sub.

```
texto = "Hola Python"
print(texto.startswith("Hola"))
# True
```



.endswith(sub)

Verifica si el string termina con sub.

```
texto = "archivo.txt"
print(texto.endswith(".txt"))
# True
```



3.3 EJEMPLO INTEGRADOR

```
texto = " Aprender Python es genial y Python es poder. "

# Buscar
print("Índice de 'Python':", texto.find("Python")) # 11
print("¿Existe 'Java'?", "Java" in texto) # False
print("Cantidad de 'Python':", texto.count("Python")) # 2

# Verificar
print("Empieza con espacios?", texto.startswith(" ")) # True
print("Termina con 'poder.'?", texto.strip().endswith("poder. ")) # True
```

TIP



Usa **.strip()** antes de hacer validaciones si puede haber espacios al inicio o al final. Así evitas errores invisibles.



3.4 COMPARACIONES ÚTILES

.isalnum()

Verifica si todos los caracteres son letras o números.

```
print("abc123".isalnum()) # True
print("abc 123".isalnum()) # False
```



.isalpha()

Verifica si todos los caracteres son letras.

```
print("python".isalpha()) # True
print("python123".isalpha()) # False
```



.isdigit()

Verifica si todos los caracteres son números.

```
print("12345".isdigit()) # True
print("123a".isdigit()) # False
```



Practica estos métodos y verás cómo manipular texto en Python se vuelve más fácil y poderoso.

¡Tú puedes! 🍀





STRING METHODS

Volumen 14

4. REEMPLAZAR Y MODIFICAR



Además de transformar, también podemos **reemplazar** partes del texto o **modificarlo** completamente.

¡Perfectos para **personalizar** y **limpiar** nuestros datos!



4.1 REEMPLAZAR (REPLACE)

Reemplaza todas las apariciones de un texto por otro.

```
texto = "Python es increíble"
nuevo = texto.replace("increíble", "genial")
print(nuevo) # Python es genial
```

TIP



Puedes limitar la cantidad de reemplazos con el tercer parámetro:

```
texto.replace("a", "o", 2)
```

→ Reemplaza solo las primeras 2 apariciones.



4.2 DIVIDIR (SPLIT)

Divide el string en una lista usando un separador.

```
frase = "Python es genial y poderoso"
palabras = frase.split()
print(palabras) # ['Python', 'es', 'genial', 'y', 'poderoso']
```



Sin separador, divide por espacios.
Con separador, divide donde indiquemos.



4.3 UNIR (JOIN)

Une los elementos de una lista en un solo string usando un separador.

```
palabras = ['Python', 'es', 'genial', 'y', 'poderoso']
frase = " ".join(palabras)
print(frase) # Python es genial y poderoso
```



El separador es el string que está antes de `.join()`.
Puede ser vacío "" para unir sin nada entre medio.



4.4 ELIMINAR ESPACIOS (STRIP, LSTRIP, RSTRIP)

`.strip()`

Elimina espacios al inicio y al final.

```
texto = " hola mundo "
print(texto.strip()) # "hola mundo"
```

¡Muy útil para limpiar datos!



`.lstrip()`

Elimina espacios solo al inicio.

```
texto = " hola mundo "
print(texto.lstrip()) # "hola mundo "
```



`.rstrip()`

Elimina espacios solo al final.

```
texto = " hola mundo "
print(texto.rstrip()) # " hola mundo"
```



4.5 CAMBIAR MAYÚSCULAS Y MINÚSCULAS

`.upper()` → todo a mayúsculas

```
texto = "hola Python"
print(texto.upper()) # HOLA PYTHON
```

`.lower()` → todo a minúsculas

```
texto = "HOLA PYTHON"
print(texto.lower()) # hola python
```

`.title()` → primera letra de cada palabra en mayúscula

```
texto = "python es genial"
print(texto.title()) # Python Es Genial
```

`.swapcase()` → invierte mayúsculas por minúsculas y viceversa

```
texto = "PyThOn Es GeNiAl"
print(texto.swapcase()) # pYtHoN eS gEnIaL
```



4.6 MODIFICACIONES AVANZADAS

`.center(ancho, relleno)`

Centra el texto en un ancho dado usando un relleno.

```
texto = "Python"
print(texto.center(20, "-")) # -----Python-----
```

`.ljust(ancho, relleno)`

Alinea a la izquierda y rellena el resto.

```
texto = "Python"
print(texto.ljust(15, ".")) # Python.....
```

`.rjust(ancho, relleno)`

Alinea a la derecha y rellena el resto.

```
texto = "Python"
print(texto.rjust(15, ".")) # .....Python
```



RECUERDA

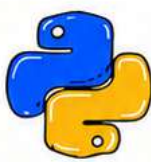
Los métodos de strings te dan superpoderes para manipular texto de forma rápida y eficiente. Prácticos y te harán la vida mucho más fácil.



Pequeños métodos, grandes resultados.

¡Sigue así, vas genial! 🌟





STRING METHODS

Volumen 14

5. EJEMPLOS PRÁCTICOS



La mejor forma de aprender es practicando con ejemplos reales.

¡Explora, prueba y juega con los métodos!

Cuanto más los uses, más natural te resultarán 🐍



5.1 EJEMPLOS EN CONTEXTO

TÍTULO LIMPIO



Limpiamos y formateamos un título de un artículo.

```
titulo = " python es  
GENIAL!!  
limpio = titulo.strip().  
title()  
print(limpio)
```

Salida:

Python Es Genial

EMAIL VÁLIDO



Validamos un email de usuario.

```
email = " MiCorreo@  
GMAIL.com "  
email = email.strip().  
lower()  
print(email)
```

Salida:

micorreo@gmail.com

NOMBRE DE ARCHIVO



Creamos un nombre de archivo seguro y consistente.

```
nombre = " reporte final "  
seguro = nombre.strip().  
replace(" ", "_").lower()  
print(seguro + ".txt")
```

Salida:

reporte_final.txt

MENSAJE BONITO



Damos formato y alineamos un mensaje.

```
msg = " python "  
bonito = msg.center(20, "*")  
print(bonito.upper())
```

Salida:

***** PYTHON *****

5.2 CONSEJOS Y BUENAS PRÁCTICAS

- ✓ Usa `.strip()` siempre que recibas datos del usuario.
- ✓ Normaliza texto con `.lower()` para comparaciones.
- ✓ Reemplaza espacios con guiones bajos en nombres de archivos o URLs.
- ✓ Usa `.title()` para mostrar títulos legibles.
- ✓ Evita modificar el string original: los métodos devuelven uno nuevo.
- ✓ Combina métodos para obtener el resultado exacto.



5.3 ENCADENAR MÉTODOS (CHAINING)

Podemos encadenar métodos para transformar un string en una sola línea.

```
texto = " Aprender PYTHON es GENIAL!! "  
resultado = (texto.strip().  
lower().  
replace(" ", "_").  
title())  
print(resultado)
```

Salida:

Aprender_Python_Es_Genial!!

TIP



El orden de los métodos importa. Piensa siempre en el resultado que quieres obtener.



5.4 RESUMEN GENERAL DE MÉTODOS

Categoría	Método	¿Qué hace?	Ejemplo	Resultado
Transformación	<code>upper()</code>	Convierte a mayúsculas	<code>"hola".upper()</code>	<code>"HOLA"</code>
	<code>lower()</code>	Convierte a minúsculas	<code>"HOLA".lower()</code>	<code>"hola"</code>
	<code>capitalize()</code>	Primera mayúscula	<code>"pythen es genial".capitalize()</code>	<code>"Python es genial"</code>
	<code>title()</code>	Cada palabra en mayúscula	<code>"python es genial".title()</code>	<code>"Python Es Genial"</code>
	<code>swapcase()</code>	Intercambia mayúsculas/minúsculas	<code>"PyThOn".swapcase()</code>	<code>"pYtHoN"</code>
Búsqueda	<code>find(sub)</code>	Índice de la primera aparición	<code>"hola mundo".find("m")</code>	<code>5</code>
	<code>index(sub)</code>	Índice o error si no existe	<code>"hola".index("x")</code>	<code>ValueError</code>
	<code>count(sub)</code>	Cuenta cuántas veces aparece	<code>"banana".count("a")</code>	<code>3</code>
	<code>startswith(sub)</code>	¿Empieza con sub?	<code>"python".startswith("py")</code>	<code>True</code>
	<code>endswith(sub)</code>	¿Termina con sub?	<code>"archivo.txt".endswith(".txt")</code>	<code>True</code>
Modificación	<code>replace(viejo, nuevo)</code>	Reemplaza partes del texto	<code>"uno dos".replace(" ", "_")</code>	<code>"uno_dos"</code>
	<code>strip()</code>	Quita espacios al inicio y final	<code>" hola ".strip()</code>	<code>"hola"</code>
	<code>lstrip()</code>	Quita espacios solo al inicio	<code>" hola".lstrip()</code>	<code>"hola"</code>
	<code>rstrip()</code>	Quita espacios solo al final	<code>"hola ".rstrip()</code>	<code>"hola"</code>
	<code>center(ancho, relleno)</code>	Centra el texto	<code>"py".center(10, "-")</code>	<code>"---py---"</code>
Alineación	<code>ljust(ancho, relleno)</code>	Alinea a la izquierda	<code>"py".ljust(8, ".")</code>	<code>"py....."</code>
	<code>rjust(ancho, relleno)</code>	Alinea a la derecha	<code>"py".rjust(8, ".")</code>	<code>"......py"</code>
	<code>zfill(ancho)</code>	Rellena con ceros a la izquierda	<code>"42".zfill(5)</code>	<code>"00042"</code>



Cada método que aprendes es una nueva herramienta en tu caja de herramientas.

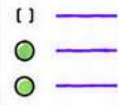
¡Sigue practicando y verás cómo cada día entiendes más y mejor. 🐍





BOOLEANOS

Volumen 15



1. ¿QUÉ SON LOS BOOLEANOS?



Los booleanos representan solo dos valores posibles: **Verdadero** o **Falso**.

En Python se escriben como **True** y **False** (siempre con mayúscula inicial).

Son muy importantes para tomar decisiones en nuestros programas.



CONCEPTO CLAVE

- ✓ Solo existen dos valores: True y False.
- ✓ Se usan en comparaciones y condiciones.
- ✓ Devuelven el resultado de una expresión lógica.
- ✓ No son cadenas de texto ("True" ≠ True).
- ✓ Son la base de estructuras como if, while, etc.



EJEMPLO VISUAL

```
edad = 18
es_mayor = edad >= 18 # ¿Es mayor de edad?
↓
print(es_mayor) # True
```

Salida:

True



1.1 VALORES BOOLEANOS

Valor	Representación	Tipo	Descripción	
True	True	bool	Verdadero. Algo que se cumple.	✓
False	False	bool	Falso. Algo que no se cumple.	✗

TIP



True y False son valores especiales, no necesitan comillas. Si usas "True" es una cadena de texto, no un booleano.



1.2 COMPARACIONES (DEVUELVEN BOOLEANOS)

Las comparaciones siempre devuelven True o False.

```
5 > 2 # True
7 == 7 # True
3 != 4 # True
10 < 5 # False
"a" == "a" True
"python" != "Python" # True
```

Operador	Significado	Ejemplo	Resultado
==	Igual a	5 == 5	True
!=	Distinto de	5 != 3	True
>	Mayor que	8 > 2	True
<	Menor que	2 < 1	False
>=	Mayor o igual que	5 >= 5	True
<=	Menor o igual que	3 <= 2	False



Estas expresiones son la base para tomar decisiones en nuestro código.



1.3 OPERADORES LÓGICOS

and (Y)



Devuelve True solo si AMBAS condiciones son True.

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

```
ej: 5 > 2 and 3 < 10
# True
```

or (O)



Devuelve True si AL MENOS una condición es True.

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

```
ej: 5 > 2 or 3 > 10
# True
```

not (NO)



Invierte el valor booleano.

A	not A
True	False
False	True

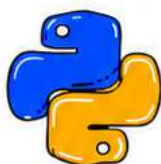
```
ej: not (5 > 2)
# False
```

TIP



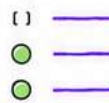
Los operadores lógicos se usan mucho en condiciones complejas. ¡Prácticalos para entender mejor!





BOOLEANOS

Volumen 15



3. OPERACIONES Y USOS PRÁCTICOS



Los booleanos se combinan con operadores para construir expresiones lógicas más complejas y tomar decisiones inteligentes.



3.1 OPERACIONES CON BOOLEANOS

Operador	Nombre	Descripción	Ejemplo	Resultado
and	Y lógico	True solo si AMBOS son True	True and False	False
or	O lógico	True si AL MENOS UNO es True	True or False	True
not	NO lógico	Invierte el valor True ↔ False	not True	False

EJEMPLO EN PYTHON

```
# Usando operadores lógicos
a = True
b = False

resultado1 = a and b      # False
resultado2 = a or b       # True
resultado3 = not a        # False

print(resultado1)
print(resultado2)
print(resultado3)
```

Salida:

```
False
True
False
```



3.2 TABLA DE VERDAD - and, or, not

AND (y)			OR (o)			NOT (no)	
A	B	A and B	A	B	A or B	A	not A
True	True	True	True	True	True	True	False
True	False	False	True	False	True	False	True
False	True	False	False	True	True	False	True
False	False	False	False	False	False	False	True

TIP



Memoriza estas tablas.
Son la base para entender condiciones complejas.
¡Te van a salvar siempre!



3.3 EJEMPLOS PRÁCTICOS

1. Verificar edad para entrar

```
edad = 18
tiene_entrada = True

puede_entrar = edad >= 18 and tiene_entrada
print(puede_entrar)      # True
```

¿Qué hace?

Comprueba que la persona sea mayor de edad Y tenga entrada.

Salida:

```
True
```



2. Descuento en una tienda

```
es_cliente = True
es_estado = False

descuento = es_cliente or es_estado
print(descuento)        # True
```

¿Qué hace?

Da descuento si el cliente es VIP es estudiante.

Salida:

```
True
```



3. Validar formulario

```
nombre_ok = True
email_ok = True
edad_ok = False

formulario_valido = nombre_ok and email_ok and edad_ok
print(formulario_valido) # False
```

¿Qué hace?

El formulario es válido solo si todos los campos están correctos.

Salida:

```
False
```



3.4 ERRORES COMUNES

- ✗ Usar = en lugar de == para comparar.
- ✗ Olvidar que and tiene prioridad sobre or.
- ✗ No usar paréntesis en expresiones complejas.
- ✗ Comparar booleanos con números.

```
if edad = 18:      # ✗ Error
if edad == 18:    # ✔ Correcto

a or b and c      # Es a or (b and c)
(a or b) and c    # Aclara tu intención

if x == 1:        # Evitar
if x is True:     # Mejor
```

RECUERDA



Escribe siempre claro.
Usa paréntesis cuando tengas dudas y el código será más fácil de leer y mantener.



Los booleanos son pequeños, pero su poder es gigante.

Domina la lógica y tus programas tomarán decisiones como un PRO.



Volumen 15 • Booleanos

Designed & Summarized by GuilleGiu

APRENDIENDO HOY, CREANDO MI MEJOR MAÑANA



BOOLEANOS

Volumen 15

4. DECISIONES CON BOOLEANOS



Los booleanos son la base de las **decisiones** en Python.

Gracias a ellos, nuestro código puede elegir qué hacer en cada situación.



4.1 IF, ELIF, ELSE

Usamos booleanos (**True** o **False**) para decidir qué bloque de código ejecutar.

```
edad = 20
if edad >= 18:           # True
    print("Eres mayor de edad")
elif edad >= 13:         # False
    print("Eres adolescente")
else:                    # False
    print("Eres menor de edad")
```

Salida:

Eres mayor de edad



TIP



Python revisa las condiciones de arriba hacia abajo.

Ejecuta el primer bloque que resulte **True** y salta los demás.



4.2 OPERADORES DE COMPARACIÓN (RECORDATORIO)

Operador	Significado	Ejemplo	Resultado
<code>==</code>	Igual a	<code>5 == 5</code>	True
<code>!=</code>	Distinto de	<code>5 != 3</code>	True
<code>></code>	Mayor que	<code>7 > 4</code>	True
<code><</code>	Menor que	<code>2 < 10</code>	True
<code>>=</code>	Mayor o igual que	<code>6 >= 6</code>	True
<code><=</code>	Menor o igual que	<code>3 <= 2</code>	False

4.3 COMBINANDO CON `and`, `or`, `not`

Operador	Significado	Ejemplo	Resultado
<code>and</code>	Ambas condiciones son True	<code>True and False</code>	False
<code>or</code>	Al menos una condición es True	<code>True or False</code>	True
<code>not</code>	Invierte el valor	<code>not True</code>	False

4.4 EJEMPLOS REALES

1. Acceso a una app
Solo permite entrar si el usuario y la contraseña son correctos.

```
usuario = "guille"
clave = "1234"
login = True
if usuario == "guille" and clave == "1234":
    login = True
else:
    login = False
print(login) # True
```



2. Descuento en tienda
Hay descuento si el cliente es VIP o gasta más de \$1000.

```
es_vip = False
total_compra = 1200
tiene_descuento = es_vip or total_compra > 1000
print(tiene_descuento) # True
```



3. Validar edad para evento
Entrada permitida si tiene entre 18 y 65 años (inclusive).

```
edad = 30
entrada_valida = edad >= 18 and edad <= 65
print(entrada_valida) # True
```

4.5 TABLAS DE VERDAD

AND (y)

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

OR (o)

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

NOT (no)

A	not A
True	False
False	True

TIP



Si alguna condición no te queda clara, usa `print()` para ver el resultado.



EN LA PRÁCTICA



- ✓ Define la condición que quieres evaluar.
- ✓ Usa operadores para combinar varias condiciones.
- ✓ Estructura tu código con `if`, `elif` y `else`.
- ✓ Prueba con diferentes valores.



RECUERDA

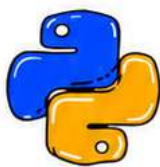
Los booleanos pueden parecer simples, pero son la base de toda la lógica de programación.



Volumen 15 • Booleanos

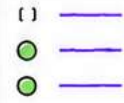
Designed & Summarized by [GuilleGiu](#)

APRENDIENDO HOY, CREANDO SIEJØR MAÑANA



BOOLEANOS

Volumen 15



4. RESUMEN GENERAL DE BOOLEANOS



Los booleanos son la base de la lógica en Python.

Nos permiten tomar decisiones, comparar valores y controlar el flujo de nuestros programas.

Domina los booleanos y podrás construir código más inteligente y eficiente. 🧠



1. VALORES Y REPRESENTACIÓN

Valor	Representación	Tipo	Descripción
True	True	bool	Verdadero. Algo que se cumple. ✓
False	False	bool	Falso. Algo que no se cumple. ✗



En Python, True y False siempre se escriben con la primera letra en mayúscula.

2. OPERADORES LÓGICOS

Operador	Significado	Ejemplo	Resultado
and (y)	Ambas condiciones son True	A and B	True solo si A y B son True
or (o)	Al menos una condición es True	A or B	True si al menos una es True
not (no)	Invierte el valor booleano	not A	Cambia True a False y False a True



Estos operadores se usan mucho en if, while, validaciones y combinaciones de condiciones.

3. COMPARACIONES (DEVUELVEN BOOLEANOS)

Las comparaciones siempre devuelven True o False.

```
5 > 2           # True
7 == 7          # True
3 != 4          # True
10 < 5          # False
"a" == "a"      True
"python" != "Python" # True
```

Operador	Significado	Ejemplo	Resultado
==	Igual a	5 == 5	True
!=	Distinto de	5 != 3	True
>	Mayor que	8 > 2	True
<	Menor que	2 < 1	False
>=	Mayor o igual que	5 >= 5	True
<=	Menor o igual que	3 <= 2	False

TIP



Las comparaciones son la base para tomar decisiones en nuestro código.



4. USOS PRINCIPALES

EN CONDICIONES (if, elif, else)

```
edad = 20
if edad >= 18:
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
```

✓ Decidimos qué bloque de código ejecutar.

EN BUCLES (while)

```
contador = 0
while contador < 5:
    print(contador)
    contador += 1
```

✓ El bucle se repite mientras la condición sea True.

EN VALIDACIONES

```
usuario = "guille"
clave = "1234"
if usuario == "guille" and clave == "1234":
    print("Acceso permitido")
else:
    print("Acceso denegado")
```

✓ Combinamos condiciones para validar datos.

TIP



Piensa en los booleanos como interruptores: encendido (True) o apagado (False). Tu programa toma decisiones según el estado del interruptor.



5. TABLA DE VERDAD (OPERADORES LÓGICOS)

AND (y)

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

OR (o)

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

NOT (no)

A	not A
True	False
False	True

TIP



Memoriza estas tablas. Te ayudarán a entender cómo se combinan las condiciones.



EN RESUMEN

- ✓ Booleanos = True (verdadero) o False (falso).
- ✓ Se usan en comparaciones y decisiones.
- ✓ Operadores lógicos combinan condiciones.
- ✓ Son esenciales para controlar el flujo del programa.

RECUERDA



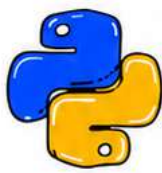
Los booleanos parecen simples, pero son el corazón de la lógica de programación. 🧠



Volumen 15 • Booleanos

Designed & Summarized by GuilleGiu

APRENDIENDO HOY, CRECIENDO SIEMPRE



RANGE + ITERADORES

1. ¿QUÉ ES RANGE?



`range()` es una función incorporada que genera una secuencia de números.

Es muy útil para recorrer rangos con bucles `for`.



1.1 SINTAXIS

`range()` puede usarse de 1, 2 o 3 formas:

`range(stop)` → 0 hasta `stop-1`

`range(start, stop)` → `start` hasta `stop-1`

`range(start, stop, step)` → con un paso (`step`)



`stop` NO se incluye en la secuencia.
El valor por defecto de `step` es 1.



1.2 EJEMPLOS BÁSICOS

Código	Descripción	Salida
<code>list(range(5))</code>	Números del 0 al 4	<code>[0, 1, 2, 3, 4]</code>
<code>list(range(2, 6))</code>	Números del 2 al 5	<code>[2, 3, 4, 5]</code>
<code>list(range(1, 10, 2))</code>	Del 1 al 9 de 2 en 2	<code>[1, 3, 5, 7, 9]</code>
<code>list(range(10, 0, -2))</code>	Del 10 al 2 de -2 en -2	<code>[10, 8, 6, 4, 2]</code>

TIP



Si el `step` es positivo, `start` debe ser menor que `stop`.
Si el `step` es negativo, `start` debe ser mayor que `stop`.



1.3 USO EN BUCLES FOR

`range()` se usa comúnmente para repetir acciones.

```
for i in range(5):  
    print(i)          # Imprime 0, 1, 2, 3, 4  
  
for i in range(1, 6):  
    print(f"Número: {i}")  
  
# Imprime del 1 al 5
```



1.4 RANGOS COMUNES EN PYTHON

Rango	Equivalencia	Uso típico
<code>range(n)</code>	0 a <code>n-1</code>	Contar desde 0
<code>range(1, n+1)</code>	1 a <code>n</code>	Contar desde 1
<code>range(0, n, 2)</code>	0, 2, 4, ...	Números pares
<code>range(1, n, 2)</code>	1, 3, 5, ...	Números impares
<code>range(n, 0, -1)</code>	<code>n</code> hasta 1	Contar hacia atrás



¡Recuerda!
`stop` nunca
se incluye. 😊

1.5 ¿QUÉ DEVUELVE RANGE()?

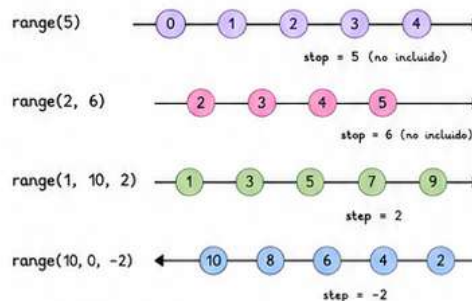
`range()` devuelve un objeto de tipo `range` (no una lista).

Es un objeto iterable, lo que significa que puede recorrerse con un bucle `for`.

```
r = range(5)  
type(r)  # <class 'range'>  
list(r)  # [0, 1, 2, 3, 4]
```



1.6 DIAGRAMA VISUAL



Piensa en...

`range()` es como una escalera: tú eliges desde dónde empiezas, hasta dónde llegas y de cuánto en cuánto subes (o bajas).



RESUMEN RÁPIDO



- ✓ `range()` genera una secuencia de números.
- ✓ Tiene 3 formas: `range(stop)`, `range(start, stop)`, `range(start, stop, step)`.
- ✓ `stop` no se incluye.

- ✓ Se usa principalmente con bucles `for`.
- ✓ Devuelve un objeto iterable (no una lista).
- ✓ Muy útil para contar, iterar y generar secuencias.





RANGE + ITERADORES

Volumen 16

2. ITERADORES EN PYTHON



Un iterador es un objeto que permite recorrer elementos uno por uno.

En Python, muchos objetos son iterables (pueden recorrerse) y crean iteradores cuando los usamos en un bucle `for`.



2.1 ITERABLE vs ITERADOR

ITERABLE

- ✓ Es un objeto del que podemos obtener un iterador.
- ✓ Ejemplos: list, tuple, str, range, dict, set.
- ✓ Se usa en bucles `for`.

ITERADOR

- ✓ Es un objeto que produce valores uno por uno.
- ✓ Tiene los métodos `__iter__()` y `__next__()`.
- ✓ Se usa internamente por `for`.

TIP

- ★ Todo iterador es iterable, pero no todo iterable es iterador. Cuando usas `for`, Python obtiene un iterador automáticamente.

2.2 FUNCIONES ITER() Y NEXT()

`iter(obj)` devuelve un iterador del objeto iterable.

`next(iterador)` devuelve el siguiente elemento.

```
it = iter([10, 20, 30]) # crea un iterador
print(next(it))         # 10
print(next(it))         # 20
print(next(it))         # 30
# print(next(it))       # StopIteration (no hay más)
```

Quando no quedan más elementos, `next()` lanza `StopIteration`.

TIP

- ★ No es común usar `iter()` y `next()` directamente, pero entenderlos te ayudará a entender `for`.

2.3 RANGOS NEGATIVOS Y DESCENDENTES

Podemos usar `range()` para contar hacia atrás.

Código	Descripción	Salida
<code>list(range(5, 0, -1))</code>	De 5 a 1 (sin incluir 0)	[5, 4, 3, 2, 1]
<code>list(range(10, -1, -2))</code>	De 10 a 0 (-2 en -2)	[10, 8, 6, 4, 2, 0]
<code>list(range(-1, -6, -1))</code>	De -1 a -5	[-1, -2, -3, -4, -5]

- ★ El step negativo siempre cuenta hacia atrás. start debe ser mayor que stop.

2.4 USO DE RANGE() CON LETRAS (USANDO ord/chr)

Podemos combinar `range()` con `ord()` y `chr()` para recorrer letras.

```
for c in range(ord('a'), ord('f')):
    print(chr(c), end=' ')
# Salida: a b c d e
```

`ord()` convierte carácter a número.
`chr()` convierte número a carácter.

- ★ Muy útil para generar secuencias de letras, códigos ASCII, etc.

2.5 OBJETOS ITERABLES COMUNES

LISTA

```
frutas = ['manzana', 'pera', 'uva']
for f in frutas:
    print(f)
# manzana
# pera
# uva
```

TUPLA

```
coordenadas = (10, 20, 30)
for c in coordenadas:
    print(c)
# 10
# 20
# 30
```

CADENA (str)

```
texto = "Hola"
for letra in texto:
    print(letra)
# H
# o
# l
# a
```

DICCIONARIO (dict)

```
edades = {'Ana': 20, 'Luis': 30}
for clave in edades:
    print(clave, edades[clave])
# Ana 20
# Luis 30
```

SET (conjunto)

```
nums = {1, 2, 3}
for n in nums:
    print(n)
# 1
# 2
# 3 (orden no garantizado)
```

2.6 ENUMERATE()

Devuelve pares índice + valor mientras itera.

```
nombres = ['Ana', 'Luis', 'Mia']
for i, nombre in enumerate(nombres):
    print(i, nombre)
# 0 Ana
# 1 Luis
# 2 Mia
```

2.7 ZIP()

Itera sobre varios iterables en paralelo.

```
nombres = ['Ana', 'Luis', 'Mia']
edades = [20, 30, 25]
for n, e in zip(nombres, edades):
    print(n, 'tiene', e, 'años')
# Ana tiene 20 años
# Luis tiene 30 años
# Mia tiene 25 años
```

2.8 COMPRENSIÓN CON RANGE()

Crear listas de forma rápida.

```
cuadrados = [x*x for x in range(6)]
print(cuadrados)
# [0, 1, 4, 9, 16, 25]
pares = [x for x in range(20) if x % 2 == 0]
print(pares)
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

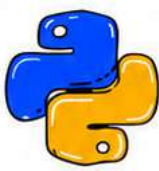


EN RESUMEN

- ✓ `range()` genera secuencias de números.
- ✓ Los iteradores permiten recorrer elementos uno por uno.
- ✓ `for` usa iteradores de forma automática.

- ✓ `iter()` y `next()` son la base de los bucles.
- ✓ Muchos objetos en Python son iterables.
- ✓ Con `enumerate()`, `zip()` y comprensions escribimos código más limpio y poderoso.





RANGE + ITERADORES

Volumen 16

3. RECORRIDOS AVANZADOS Y HERRAMIENTAS ÚTILES



Los iteradores y `range()` son herramientas poderosas para escribir código limpio, eficiente y *Pythonic*.

Existen muchas formas de recorrer y transformar datos con ellos.



3.1 ENUMERATE() CON ÍNDICE Y VALOR

Devuelve tuplas (índice, valor) mientras itera.

```
frutas = ["manzana", "pera", "uva"]
for i, fruta in enumerate(frutas):
    print(i, fruta)
```

0 manzana
1 pera
2 uva

TIP

★ Puedes especificar desde qué índice empezar:
for i, fruta in enumerate(frutas, start=1):
empieza en 1



3.2 REVERSE() PARA RECORRER AL REVÉS

Dos formas de usarlo:

```
Con range():
for i in range(5, 0, -1):
    print(i)
```

5, 4, 3, 2, 1

Con listas u otros iterables:

```
nums = [10, 20, 30, 40]
for n in reversed(nums):
    print(n)
```

40
30
20
10

reverse() no modifica la lista original, solo la recorre al revés. 😊

3.3 LIST COMPREHENSION CON RANGE

Ideal para generar listas de forma compacta.

```
cuadrados = [x**2 for x in range(6)]
# [0, 1, 4, 9, 16, 25]

pares = [x for x in range(20) if x % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

tabla_3 = [3 * x for x in range(1, 11)]
# [3, 6, 9, 12, 15, 18, 21, 24, 27, 30]
```

TIP

★ Las comprensiones son más legibles y rápidas que los bucles tradicionales en muchos casos.



3.4 ZIP() PARA RECORRER EN PARALELO

Combina elementos de varios iterables.

```
nombres = ["Ana", "Luis", "Mia"]
edades = [20, 25, 30]

for n, e in zip(nombres, edades):
    print(n, "tiene", e, "años")
```

Ana tiene 20 años
Luis tiene 25 años
Mia tiene 30 años

TIP

★ zip() se detiene en el iterable más corto.

¡Perfecto para combinar datos!



3.5 FILTER() CON RANGO

Filtra elementos que cumplen una condición.

```
pares = list(filter(lambda x: x % 2 == 0, range(20)))
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

mayores_10 = list(filter(lambda x: x > 10, range(20)))
# [11, 12, 13, 14, 15, 16, 17, 18, 19]
```

Más claro con función normal:

```
def es_par(x):
    return x % 2 == 0

pares = list(filter(es_par, range(20)))
```



3.6 ANY() Y ALL() CON RANGO

Verifican condiciones en todos o al menos uno de los elementos.

```
todos_positivos = all(x > 0 for x in range(1, 10))
alguno_negativo = any(x < 0 for x in range(-5, 5))
```

True
True



Útiles para validar condiciones rápidamente.



3.7 EJEMPLOS PRÁCTICOS COMBINADOS

Suma de números del 1 al 100

```
total = sum(range(1, 101))
print(total)
```

5050



Números divisibles por 3 y 5

```
multiplos = [x for x in range(1, 100)
              if x % 3 == 0 and x % 5 == 0]
print(multiplos)
```

[15, 30, 45, 60, 75, 90]

3 y 5

Tabla de multiplicar con comprensión

```
tabla = {n: [n * x for x in range(1, 11)]
        for n in range(1, 6)}

for n, fila in tabla.items():
    print(f"{n}: {fila}")
```



RESUMEN GENERAL

- ✓ `range()` genera secuencias de números con gran flexibilidad.
- ✓ Los iteradores permiten recorrer elementos uno por uno.
- ✓ Funciones como `iter()`, `next()`, `enumerate()`, `reversed()`, `zip()`, `filter()`, `any()`, `all()` hacen tu código más poderoso.

- ✓ Las comprensiones y herramientas de alto nivel mejoran la legibilidad y eficiencia.
- ✓ Practica combinarlas para resolver problemas de forma elegante y *Pythonic*.





RANGE + ITERADORES

Volumen 16

4. RESUMEN GENERAL



`range()` y los iteradores son herramientas fundamentales para recorrer secuencias de forma eficiente y elegante en `Python`.

Los iteradores nos permiten trabajar con datos de gran tamaño sin cargar todo en memoria.



1. RANGE() EN RESUMEN

Forma	Descripción	Ejemplo	Resultado
<code>range(stop)</code>	0 hasta <code>stop-1</code>	<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(start, stop)</code>	<code>start</code> hasta <code>stop-1</code>	<code>range(2, 6)</code>	2, 3, 4, 5
<code>range(start, stop, step)</code>	con paso (<code>step</code>)	<code>range(1, 10, 2)</code>	1, 3, 5, 7, 9
step negativo	cuenta hacia atrás	<code>range(10, 0, -2)</code>	10, 8, 6, 4, 2

★ `stop` **NO** se incluye en la secuencia.

2. ITERADORES EN RESUMEN

- ✓ Un iterador devuelve elementos uno por uno.
- ✓ Se obtiene con `iter()`.
- ✓ Se avanza con `next()`.
- ✓ Cuando no hay más elementos, `next()` lanza `StopIteration`.
- ✓ No podemos volver atrás (son de una sola dirección).

```
it = iter([10, 20, 30])
next(it)      # 10
next(it)      # 20
next(it)      # 30
next(it)      # StopIteration
```

📌 Ideal para recorrer archivos línea a línea, lectura de streams y datos grandes.



3. USOS COMUNES CON RANGE()

- ✓ Bucle `for` básico

```
for i in range(5):
    print(i)      # 0 1 2 3 4
```
- ✓ Contar desde un número

```
for i in range(1, 6):
    print(i)      # 1 2 3 4 5
```
- ✓ Contar de `n` en `n`

```
for i in range(0, 10, 2):
    print(i)      # 0 2 4 6 8
```
- ✓ Contar hacia atrás

```
for i in range(5, -1, -1):
    print(i)      # 5 4 3 2 1 0
```



4. FUNCIONES Y HERRAMIENTAS RELACIONADAS

Herramienta	Descripción	Ejemplo
<code>enumerate(iterable)</code>	Devuelve índice y valor.	<code>for i, v in enumerate(lista):</code>
<code>zip(*iterables)</code>	Combina elementos en paralelo.	<code>for a, b in zip(lista1, lista2):</code>
<code>reversed(iterable)</code>	Itera al revés.	<code>for x in reversed(lista):</code>
<code>sorted(iterable)</code>	Devuelve lista ordenada.	<code>sorted(lista, reverse=True)</code>
<code>filter(func, iterable)</code>	Filtra elementos que cumplen condición.	<code>list(filter(lambda x: x > 10, lista))</code>
<code>any(iterable)</code>	True si al menos uno es True.	<code>any(x > 0 for x in nums)</code>
<code>all(iterable)</code>	True si todos son True.	<code>all(x % 2 == 0 for x in nums)</code>

5. CUÁNDO USAR CADA UNO

USA `RANGE()` CUANDO:

- ✓ Necesitas una secuencia de números.
- ✓ Quieres repetir acciones un número determinado de veces.
- ✓ Vas a controlar índices en listas u otras estructuras.



USA ITERADORES CUANDO:

- ✓ Trabajas con datos grandes o que no quieres cargar completos en memoria.
- ✓ Lees archivos, streams o redes.
- ✓ Necesitas generar valores bajo demanda.



6. EJEMPLO INTEGRADOR

Recorremos un archivo línea por línea usando un iterador:

```
with open("datos.txt", "r", encoding="utf-8") as f:
    it = iter(f)
    for i, linea in enumerate(it, start=1):
        print(f"{i}; {linea.strip()}") # procesa línea a línea
```

TIP

★ Los iteradores permiten trabajar con archivos enormes sin cargar todo en memoria.



EN RESUMEN



RANGE()

- Genera secuencias numéricas.
- Flexible con `start`, `stop` y `step`.
- Muy usado en bucles `for`.
- `stop` nunca se incluye.

ITERADORES

- Devuelven elementos uno a uno.
- Se obtienen con `iter()`.
- Se avanza con `next()`.
- No permiten volver atrás.

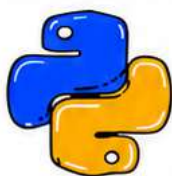
HERRAMIENTAS CLAVE

- `enumerate()`, `zip()`, `reversed()`, `filter()`, `any()`, `all()`, `sorted()`.
- Hacen tu código más limpio y poderoso.

BUENAS PRÁCTICAS

- Prefiere iteradores para datos grandes.
- Usa `range()` para bucles y secuencias numéricas.
- Mantén el código simple y legible.





MÓDULOS

Volumen 17

1. ¿QUÉ SON LOS MÓDULOS?



Un **módulo** es un archivo Python (.py) que contiene código **reutilizable**: funciones, variables, clases, etc.

Nos permiten organizar nuestro código y usarlo en otros programas.

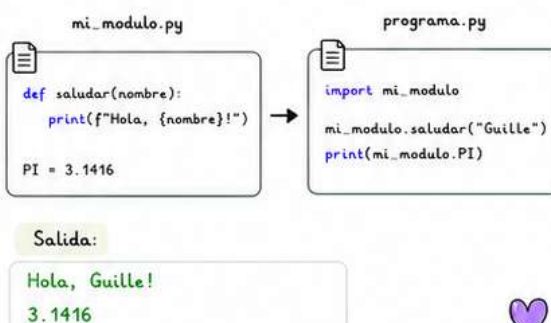


CONCEPTO CLAVE

- ✓ Un módulo es como una "caja de herramientas".
- ✓ Evita repetir código.
- ✓ Mejora la organización y el mantenimiento.
- ✓ Python ya incluye muchos módulos (built-in).
- ✓ También podemos crear nuestros propios módulos.



EJEMPLO VISUAL



1.1 TIPOS DE MÓDULOS

Tipo	Descripción	Ejemplo
Módulos integrados (built-in)	Vienen con Python. No necesitan instalación.	math, random, os, sys, datetime
Módulos externos	Se instalan con pip. Amplían la funcionalidad.	requests, pandas, numpy, matplotlib
Módulos propios	Los creamos nosotros en archivos .py	mi_modulo.py, utils.py, constantes.py

TIP



Usar módulos hace nuestro código más limpio, ordenado y profesional.



1.2 VENTAJAS DE USAR MÓDULOS



Reutilización: escribis una vez y usás muchas veces.



Organización: separás responsabilidades en archivos distintos.



Mantenimiento: más fácil actualizar y corregir código.



Colaboración: otros desarrolladores pueden entenderlo y usarlo.



1.3 EJEMPLO RÁPIDO CON UN MÓDULO BUILT-IN

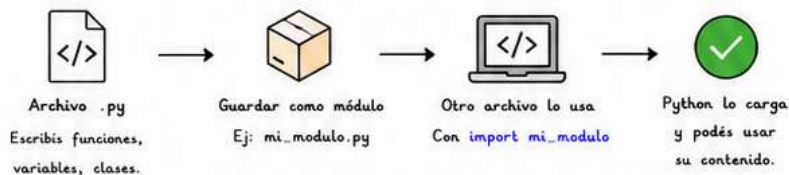
```
import math  # Importamos el módulo math
r = 5        # Radio del círculo
area = math.pi * r**2  # Usamos math.pi
print(area)  # 78.53981633974483
```



math.pi es una constante del módulo math que ya viene con Python.



1.4 CÓMO FUNCIONA UN MÓDULO (IDEA GENERAL)



Los módulos son la base de cualquier proyecto Python bien estructurado. 😊



Volumen 17 • Módulos

Designed & Summarized by [GuilleGiu](#)

APRENDIENDO HOY, CRECIENDO SIEMPRE





MÓDULOS

Volumen 17

2. IMPORTAR MÓDULOS



Para usar el contenido de un módulo en nuestro programa, debemos **importarlo**.
Existen varias formas de hacerlo.



2.1 IMPORT BÁSICO

Importa todo el módulo.

```
import math  
r = 5  
area = math.pi * r**2  
print(area) # 78.53981633974483
```

Accedemos a sus funciones usando el nombre del módulo.



2.2 ALIAS (APODO)

Le damos un nombre corto al módulo.

```
import numpy as np  
arr = np.array([1, 2, 3, 4])  
print(np.mean(arr)) # 2.5
```

Útil para módulos con nombres largos.



2.3 IMPORTAR ALGO ESPECÍFICO

Importa solo lo que necesitamos.

```
from math import sqrt, pi  
r = 9  
area = pi * r**2  
raiz = sqrt(r)  
print(area) # 254.46900494077223  
print(raiz) # 3.0
```

Solo importamos funciones específicas.



2.4 ALIAS EN IMPORT ESPECÍFICO

Podemos usar alias incluso en funciones específicas.

```
from datetime import datetime as dt  
hoy = dt.now()  
print(hoy)  
# 2025-05-22 14:30:45.123456
```

Ideal para nombres que usamos mucho.



TIP



Usa alias para escribir menos y hacer el código más limpio y legible.



from ... import ... es más eficiente cuando solo necesitas partes del módulo.



2.5 IMPORT RELATIVO (ENTRE MÓDULOS PROPIOS)

Permite importar módulos dentro del mismo paquete (carpeta).

Estructura de carpetas

```
mi_paquete/  
|-- __init__.py  
|-- utils.py  
|-- constantes.py  
|-- main.py
```

main.py

```
from . import utils  
from .constantes import PI  
  
print(utils.saludar("Guille"))  
print(PI)
```



El punto (.) significa "desde la misma carpeta".
Dos puntos (..) significa "desde la carpeta padre".



2.6 ¿CUÁNDO USAR CADA FORMA?

Forma	Cuándo usar
import modulo	Cuando usas muchas funciones del módulo.
import modulo as m	Cuando el nombre es largo o lo usas mucho.
from modulo import algo	Cuando solo necesitas partes específicas.
from modulo import algo as a	Para combinar claridad + código corto.
Import relativo (.)	Cuando trabajas con módulos propios dentro de un paquete.

2.7 EJEMPLOS PRÁCTICOS

Generar número aleatorio

```
import random  
n = random.randint(1, 10)  
print(n)  
  
# Ej: 7
```



Fechas y horas actuales

```
from datetime import datetime  
ahora = datetime.now()  
print(ahora)  
  
# 2025-05-22 14:30:45.123456
```



Cálculos matemáticos

```
from math import sqrt, pow  
print(sqrt(16)) # 4.0  
print(pow(2, 3)) # 8.0
```



Errores comunes

- ✗ Olvidar importar el módulo.
- ✗ Escribir mal el nombre del módulo.
- ✗ No tener instalado el módulo externo (pip install nombre_módulo).
- ✗ Usar el módulo sin el prefijo cuando usaste import.



EN RESUMEN



- ✓ Los módulos nos ayudan a reutilizar código.
- ✓ Hay varias formas de importar módulos.
- ✓ Usa alias para escribir menos y mejor.

- ✓ Importa solo lo que necesitas para mantener el código limpio.
- ✓ Los imports relativos ayudan a organizar proyectos grandes.



Volumen 17 • Módulos



MÓDULOS

Volumen 17

3. CREAR MÓDULOS PROPIOS



Podemos crear nuestros propios módulos para reutilizar funciones, clases o constantes.
Solo necesitamos un archivo `.py` y luego importarlo.
¡Así mantenemos nuestro código organizado y profesional!



3.1 ESTRUCTURA BÁSICA DE UN MÓDULO

Creemos un archivo llamado `operaciones.py`

```
# operaciones.py
PI = 3.1416

def sumar(a, b):
    return a + b

def restar(a, b):
    return a - b

def area_circulo(r):
    return PI * r**2
```



Todo lo que esté definido en el módulo (funciones, variables, clases) podrá ser usado desde otros archivos.



3.2 USAR NUESTRO MÓDULO

En otro archivo podemos importarlo y usarlo.

```
# programa.py
import operaciones

print(operaciones.PI)           # 3.1416
print(operaciones.sumar(3, 4))  # 7
print(operaciones.area_circulo(5)) # 78.54
```

Salida:

```
3.1416
7
78.54
```

TIP



Usamos el nombre del módulo seguido del punto (.) para acceder a lo que contiene.



3.3 IMPORTAR SOLO LO NECESARIO

Podemos importar solo funciones o variables específicas.

```
from operaciones import sumar, PI

print(sumar(10, 5))  # 15
print(PI)            # 3.1416
```

TIP



Más eficiente y claro cuando usamos pocas cosas del módulo.



3.4 IMPORTAR TODO (NO RECOMENDADO)

Importa todo el contenido del módulo.

```
from operaciones import *

print(sumar(2, 3))  # 5
print(PI)           # 3.1416
```

¡CUIDADO!



Puede generar conflictos de nombres y hacer el código menos legible. ☹️

3.5 CONSTANTES EN MÓDULOS

Las constantes se escriben en MAYÚSCULAS por convención.

```
# config.py
API_URL = "https://api.ejemplo.com"
TIMEOUT = 30
VERSION = "1.0.0"
```

TIP



Usa constantes para valores que no deben cambiar.



3.6 MÓDULOS DENTRO DE PAQUETES

Podemos organizar módulos en carpetas (paquetes).

```
mi_paquete/
├── __init__.py (indica que es un paquete)
├── operaciones.py
├── utils.py
└── constantes.py
```

TIP



Permite proyectos más grandes y mejor organizados.



3.7 EJEMPLO COMPLETO

1. Crear módulo

```
operaciones.py
├── sumar()
├── restar()
└── PI
```

2. Importar módulo

```
import operaciones

Traemos el módulo a nuestro programa.
```

3. Usar funciones

```
resultado = operaciones.sumar(8, 2)
print(resultado)  # 10

area = operaciones.area_circulo(3)
print(area)      # 28.27
```

4. Ver resultado

```
10
28.27
```

EN RESUMEN

- ✓ Los módulos organizan y reutilizan código.
- ✓ Importa módulos con `import`.
- ✓ Usa `from ... import ...` para traer solo lo necesario.

- ✓ Crea tus propios módulos (`.py`).
- ✓ Usa constantes en MAYÚSCULAS.
- ✓ Evita `import *`.

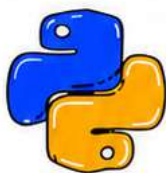
- ✓ Organiza módulos en paquetes.
- ✓ Los módulos hacen tu código más limpio, escalable y profesional.



Volumen 17 • Módulos

Designed & Summarized by [GuilleGiu](#)

APRENDIENDO HOY, CRECIENDO SIEMPRE



MÓDULOS

Volumen 17

4. RESUMEN GENERAL



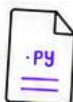
Los módulos en Python nos permiten **organizar**, **reutilizar** y **compartir** código.

Hacen nuestro código más **limpio**, **mantenible** y **profesional**.



1. ¿QUÉ SON LOS MÓDULOS?

- Son archivos **.py** que contienen código reutilizable.
- Pueden incluir funciones, variables, clases o constantes.
- Permiten dividir programas grandes en partes más pequeñas y manejables.
- Pueden ser built-in, externos o creados por nosotros.



2. FORMAS DE IMPORTAR

Forma	Ejemplo	Uso
<code>import módulo</code>	<code>import math</code>	Importa todo el módulo.
<code>import módulo as alias</code>	<code>import numpy as np</code>	Usa un alias corto.
<code>from módulo import algo</code>	<code>from math import sqrt</code>	Importa algo específico.
<code>from módulo import algo as alias</code>	<code>from datetime import datetime as dt</code>	Alias en import específico.
<code>from módulo import *</code>	<code>from operaciones import *</code>	Importa todo (no recomendado).

3. TIPOS DE MÓDULOS



Integrados (built-in)

Vienen con Python. No necesitas instalar nada.
Ej: `math`, `random`, `os`, `sys`, `datetime`



Externos (de terceros)

Se instalan con `pip`.
Ej: `requests`, `numpy`, `pandas`, `matplotlib`



Propios (creados por nosotros)

Archivos **.py** que escribimos para reutilizar.
Ej: `mi_módulo.py`, `utils.py`

4. VENTAJAS PRINCIPALES

- ✓ Reutilización de código.
- ✓ Mejora la organización del proyecto.
- ✓ Facilita el mantenimiento y la actualización.
- ✓ Permite trabajar en equipo más fácilmente.
- ✓ Hace el código más legible y profesional.

TIP



Usa módulos siempre que repitas código o quieras mantener tu proyecto escalable.



5. BUENAS PRÁCTICAS

- ✓ Usa nombres descriptivos para tus módulos.
- ✓ Cada módulo debe tener una única responsabilidad.
- ✓ Evita `import *`.
- ✓ Importa solo lo que realmente necesitas.
- ✓ Organiza tus módulos en paquetes cuando el proyecto crezca.



RECUERDA

Un buen módulo es fácil de entender, fácil de usar y fácil de mantener.



6. EJEMPLOS RÁPIDOS DE REPASO

```
# Módulo built-in
import random
print(random.randint(1, 10)) # 6

# Módulo externo (pip install requests)
import requests
r = requests.get("https://api.ejemplo.com")
print(r.status_code) # 200

# Módulo propio
from operaciones import sumar
print(sumar(10, 5)) # 15
```



CONSEJO

Prueba importar módulos diferentes y explora su documentación.



7. FLUJO DE TRABAJO CON MÓDULOS



1. Crear módulo
Escribes funciones, clases, variables o constantes en un archivo **.py**



2. Guardar e importar
Guardas el archivo e importas el módulo donde lo necesitas.



3. Usar su contenido
Usas sus funciones, clases o variables como herramientas.



4. Reutilizar y mantener
Reutilizas el módulo en distintos proyectos y lo mejoras con el tiempo.



5. Código más sólido
Tu código queda limpio, organizado, reutilizable y profesional.

EN RESUMEN



- ✓ Los módulos nos ayudan a organizar y reutilizar código.
- ✓ Podemos importar módulos de varias formas.
- ✓ Existen módulos integrados, externos y propios.

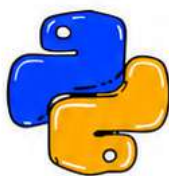
- ✓ Seguir buenas prácticas mejora la calidad del código.
- ✓ Usar módulos es clave para proyectos grandes y profesionales.



Volumen 17 • Módulos

Designed & Summarized by **GuilleGiu**

APRENDIENDO HOY, CRECIENDO SIEMPRE



FECHAS: datetime

Volumen 18

1. ¿QUÉ ES datetime?



El módulo datetime nos permite trabajar con fechas y horas en Python.

Incluye clases para manejar fechas, tiempos, combinaciones y zonas horarias.

Muy útil para reportes, logs, programación de tareas y más.



1.1 IMPORTAR datetime

```
from datetime import datetime, date, time, timedelta
```



datetime es la clase principal para fechas y horas completas.

date solo fecha, time solo hora,

timedelta diferencias de tiempo.



1.2 CLASES PRINCIPALES

Clase	Descripción	Ejemplo
datetime	Fecha y hora completas	2025-05-22 15:30:00
date	Solo fechas	2025-05-22
time	Solo horas	15:30:00
timedelta	Duración / diferencia	2 días, 3 horas

1.3 OBTENER FECHA Y HORA ACTUAL

```
from datetime import datetime
ahora = datetime.now() # Fecha y hora actual
solo_fecha = date.today() # Solo fecha
solo_hora = datetime.now().time() # Solo hora
print(ahora) # 2025-05-22 15:30:45.123456
print(solo_fecha) # 2025-05-22
print(solo_hora) # 15:30:45.123456
```

TIP



datetime.now() obtiene la fecha y hora del sistema actual.

date.today() es equivalente a datetime.now().date()



1.4 CREAR FECHAS Y HORAS

```
from datetime import datetime, date, time
f = date(2025, 5, 22)
h = time(15, 30, 0)
fh = datetime(2025, 5, 22, 15, 30, 0)
print(f) # 2025-05-22
print(h) # 15:30:00
print(fh) # 2025-05-22 15:30:00
```



El formato es:

date(año, mes, día) time(hora, minuto, segundo)
datetime(año, mes, día, hora, minuto, segundo)



1.5 FORMATEAR FECHAS

```
ahora = datetime.now()
print(ahora.strftime("%Y-%m-%d")) # 2025-05-22
print(ahora.strftime("%d/%m/%Y")) # 22/05/2025
print(ahora.strftime("%H:%M:%S")) # 15:30:45
print(ahora.strftime("%d %B %Y")) # 22 May 2025
```



Códigos comunes de strftime()

Código	Significado	Código	Significado
%Y	Año (4 dígitos)	%M	Minuto (00-59)
%m	Mes (01-12)	%S	Segundo (00-59)
%d	Día (01-31)	%B	Nombre del mes
%H	Hora (00-23)	%a	Día de la semana (abreviado)

1.6 PARSEAR (CONVERTIR TEXTO A FECHA)

```
texto = "22/05/2025 15:30"
f = datetime.strptime(texto, "%d/%m/%Y %H:%M")
print(f) # 2025-05-22 15:30:00
```



strptime() convierte un string al formato datetime según el patrón indicado.



Ejemplo de patrones

Patrón	Ejemplo
"%Y-%m-%d"	2025-05-22
"%d/%m/%Y"	22/05/2025
"%d/%m/%Y %H:%M"	22/05/2025 15:30

EN RESUMEN

¿QUÉ ES?



- datetime maneja fechas y horas.
- Incluye varias clases especializadas.

CLASES PRINCIPALES

- datetime (fecha y hora)
- date (solo fecha)
- time (solo hora)
- timedelta (diferencia)



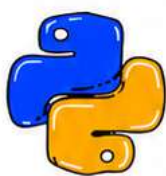
FUNCIONES CLAVE

- now(), today()
- strftime() → formato
- strptime() → parsear
- timedelta para diferencias

USOS COMUNES

- Reportes y logs
- Programar tareas
- Cálculos de duración
- Validaciones





FECHAS: datetime

Volumen 18

2. MANIPULAR FECHAS Y HORAS



Con `datetime` podemos sumar, restar y comparar fechas y horas.
Usamos objetos `timedelta` para representar duraciones.



2.1 SUMAR Y RESTAR TIEMPO (timedelta)

```
from datetime import datetime, timedelta
```

```
ahora = datetime.now()
una_semana = timedelta(weeks=1)
tres_horas = timedelta(hours=3)
```

Sumar

```
en_una_semana = ahora + una_semana
en_tres_horas = ahora + tres_horas
```

Restar

```
ayer = ahora - timedelta(days=1)
```

```
print(en_una_semana)
print(en_tres_horas)
print(ayer)
```

`timedelta` acepta:

- days
- seconds
- minutes
- hours
- weeks



Salida (ejemplo):

```
2025-05-29 15:30:45.123456
2025-05-22 18:30:45.123456
2025-05-21 15:30:45.123456
```

2.2 DIFERENCIA ENTRE FECHAS

```
from datetime import datetime
```

```
f1 = datetime(2025, 5, 22, 15, 30, 0)
f2 = datetime(2025, 5, 25, 10, 15, 0)
```

```
diferencia = f2 - f1
```

```
print(diferencia)          # Timedelta
print(diferencia.days)     # Dias
print(diferencia.seconds)  # Segundos
print(diferencia.total_seconds()) # Total seg.
```

Salida (ejemplo):

```
2 days, 18:45:00
2
67500
67500.0
```

TIP



La resta de dos fechas devuelve un objeto `timedelta`.



2.3 COMPARAR FECHAS

```
from datetime import datetime
```

```
f1 = datetime(2025, 5, 22)
f2 = datetime(2025, 5, 25)
```

```
print(f1 < f2)      # True
print(f1 == f2)     # False
print(f1 != f2)     # True
print(f1 <= f2)     # True
print(f1 >= f2)     # False
```



`datetime` soporta todos los operadores de comparación: `<`, `<=`, `>`, `>=`, `==`, `!=`



2.4 MIN, MAX Y ORDENAR

```
from datetime import datetime
```

```
fechas = [
    datetime(2025, 5, 22),
    datetime(2025, 5, 25),
    datetime(2025, 5, 20)
]
```

```
print(min(fechas))      # Mínima
print(max(fechas))      # Máxima
fechas.sort()           # Orden ascendente
print(fechas)
```

Salida (ejemplo):

```
2025-05-20 00:00:00
2025-05-25 00:00:00
[datetime.datetime(2025, 5, 20, 0, 0),
 datetime.datetime(2025, 5, 22, 0, 0),
 datetime.datetime(2025, 5, 25, 0, 0)]
```



`datetime` se puede ordenar porque es comparable.



2.5 CAMBIAR PARTES DE UNA FECHA (replace)

```
from datetime import datetime
```

```
hoy = datetime(2025, 5, 22, 15, 30, 0)
```

```
nueva_fecha = hoy.replace(year=2026)
nueva_hora = hoy.replace(hour=8, minute=0, second=0)
nueva_fecha_hora = hoy.replace(day=1, month=1,
                                hour=0, minute=0, second=0)
```

```
print(nueva_fecha)
print(nueva_hora)
print(nueva_fecha_hora)
```

Salida (ejemplo):

```
2026-05-22 15:30:00
2025-05-22 08:00:00
2025-01-01 00:00:00
```

TIP



`replace()` devuelve una nueva fecha con los cambios aplicados.



2.6 FORMATEAR FECHAS (strftime)

```
from datetime import datetime
```

```
hoy = datetime(2025, 5, 22, 15, 30, 45)
```

```
print(hoy.strftime("%Y-%m-%d"))      # 2025-05-22
print(hoy.strftime("%d/%m/%Y"))      # 22/05/2025
print(hoy.strftime("%H:%M:%S"))      # 15:30:45
print(hoy.strftime("%A, %d de %B de %Y"))
# Jueves, 22 de Mayo de 2025
```

Códigos útiles de `strftime`

Código	Significado
%Y	Año con 4 dígitos (2025)
%y	Año con 2 dígitos (25)
%m	Mes (01-12)
%d	Día (01-31)
%H	Hora (00-23)
%M	Minuto (00-59)
%S	Segundo (00-59)
%A	Día de la semana
%B	Nombre del mes

2.7 PARSEAR FECHAS (strptime)

```
from datetime import datetime
```

```
texto = "22/05/2025 15:30:45"
formato = "%d/%m/%Y %H:%M:%S"
fecha = datetime.strptime(texto, formato)
```

```
print(fecha)
print(type(fecha))
```

Salida (ejemplo):
2025-05-22 15:30:45
<class 'datetime.datetime'>



`strptime()` convierte texto en un objeto `datetime` según el formato indicado.



EN RESUMEN



MANIPULAR

- Sumar y restar con `timedelta`.
- Diferencias entre fechas.



COMPARAR Y ORDENAR

- Comparaciones con operadores.
- `min()`, `max()` y `sort()`.



MODIFICAR

- Cambiar partes de una fecha con `replace()`.



FORMATEAR Y PARSEAR

- `strftime()` para mostrar.
- `strptime()` para leer texto como fecha.



Volumen 18 • Fechas: datetime

Designed & Summarized by GuilleGiu

APRENDIENDO HOY, CRECIENDO SIEMPRE



FECHAS: datetime

Volumen 18

3. DURACIONES Y ZONAS HORARIAS



Con `datetime` también podemos trabajar con duraciones de tiempo (`timedelta`) y zonas horarias (`timezone`).
Ideal para programas que dependen del tiempo y de diferentes lugares.



3.1 `timedelta`: DURACIONES

`timedelta` representa una duración o diferencia de tiempo.

```
from datetime import timedelta
duracion = timedelta(days=2, hours=3, minutes=15)
print(duracion) # 2 days, 3:15:00
print(duracion.total_seconds()) # segundos totales
print(duracion.days) # 2
print(duracion.seconds) # 11700 (3*3600 + 15*60)
```

TIP

★ `.total_seconds()` devuelve toda la duración convertida a segundos.



3.2 OPERACIONES CON `timedelta`

Podemos sumar y restar duraciones a fechas.

```
from datetime import datetime, timedelta
ahora = datetime(2025, 5, 22, 15, 30)
duracion = timedelta(days=3, hours=2)

futuro = ahora + duracion # Sumar duración
pasado = ahora - duracion # Restar duración

print(futuro) # 2025-05-25 17:30:00
print(pasado) # 2025-05-19 13:30:00
```



Sumar = avanzar en el tiempo
Restar = retroceder en el tiempo



3.3 COMBINAR FECHAS Y HORAS: `datetime.combine()`

Une una fecha (`date`) y una hora (`time`) en un `datetime`.

```
from datetime import date, time, datetime

d = date(2025, 5, 22)
t = time(9, 45, 30)

fecha_hora = datetime.combine(d, t)
print(fecha_hora) # 2025-05-22 09:45:30
```



3.4 EXTRAER PARTE DE UNA FECHA

Usamos atributos de `datetime` para obtener partes.

```
ahora = datetime(2025, 5, 22, 15, 30, 45)
print(ahora.year) # 2025
print(ahora.month) # 5
print(ahora.day) # 22
print(ahora.hour) # 15
print(ahora.minute) # 30
print(ahora.second) # 45
print(ahora.weekday()) # 3 (Lunes=0, Domingo=6)
```

TIP



`weekday()` es útil para saber el día de la semana.



3.5 ZONAS HORARIAS con `timezone` (UTC)

`timezone` permite trabajar con zonas horarias fijas.

```
from datetime import datetime, timezone, timedelta

utc = timezone.utc
ahora_utc = datetime(2025, 5, 22, 15, 30, tzinfo=utc)
print(ahora_utc) # 2025-05-22 15:30:00+00:00
```



UTC (Tiempo Universal Coordinado) es la referencia de tiempo para todo el mundo.



3.6 ZONAS HORARIAS con desplazamientos

Podemos crear zonas con desplazamientos (ej: GMT-3).

```
from datetime import datetime, timezone, timedelta

gmt_menos3 = timezone(timedelta(hours=-3))
ahora_arg = datetime(2025, 5, 22, 12, 30, tzinfo=gmt_menos3)
print(ahora_arg) # 2025-05-22 12:30:00-03:00
```



El valor negativo (-3) indica que está 3 horas detrás de UTC.



3.7 EJEMPLOS PRÁCTICOS

Objetivo	Código	Salida (ejemplo)	Descripción
Fecha actual	<code>datetime.now()</code>	2025-05-22 15:30:45.123456	Obtiene fecha y hora actual.
Restar 5 días	<code>datetime.now() - timedelta(days=5)</code>	2025-05-17 15:30:45.123456	Retrocede 5 días.
Sumar 1 hora	<code>datetime.now() + timedelta(hours=1)</code>	2025-05-22 16:30:45.123456	Avanza 1 hora.
Solo fecha	<code>date.today()</code>	2025-05-22	Obtiene solo la fecha actual.
Solo hora	<code>datetime.now().time()</code>	15:30:45.123456	Obtiene solo la hora actual.
Convertir a string	<code>datetime.now().strftime("%d/%m/%Y %H:%M")</code>	22/05/2025 15:30	Formatea a texto.

EN RESUMEN



DURACIONES

- Usa `timedelta`.
- Suma, resta y compara.



FECHAS Y HORAS

- `datetime` combina todo.
- Extrae partes fácilmente.



ZONAS HORARIAS

- Usa `timezone`.
- Trabaja con UTC y desplazamientos.



APLICACIONES

- Reportes y logs
- Programación de tareas
- Validaciones y más





FECHAS: datetime

Volumen 18

4. TRABAJAR CON FECHAS: EJEMPLOS AVANZADOS



Con `datetime` podemos formatear, calcular diferencias, iterar por fechas y realizar operaciones útiles en aplicaciones reales.

¡Dominar las fechas te hará más **productivo**!



4.1 FORMATEAR FECHAS (`strftime`)

Convierte una fecha a un string con el formato que necesites.

```
from datetime import datetime
ahora = datetime(2025, 5, 22, 15, 30, 45)

print(ahora.strftime("%Y-%m-%d")) # 2025-05-22
print(ahora.strftime("%d/%m/%Y")) # 22/05/2025
print(ahora.strftime("%H:%M:%S")) # 15:30:45
print(ahora.strftime("%A, %d de %B de %Y"))
# Jueves, 22 de Mayo de 2025
```

TIP

★ Usamos `strftime()` para personalizar cómo queremos mostrar una fecha.

Códigos comunes

%Y	Año (2025)
%y	Año (25)
%m	Mes (01-12)
%d	Día (01-31)
%H	Hora (00-23)
%M	Minuto (00-59)
%S	Segundo (00-59)
%A	Día semana (Lunes)
%B	Mes completo (Mayo)
%a	Día semana corto (Lun)

4.2 PARSEAR FECHAS (`strptime`)

Convierte un string en un objeto `datetime` según un formato.

```
from datetime import datetime
texto = "22/05/2025 15:30"
formato = "%d/%m/%Y %H:%M"
fecha = datetime.strptime(texto, formato)

print(fecha) # 2025-05-22 15:30:00
print(type(fecha)) # <class 'datetime.datetime'>
```

TIP

★ `strptime()` interpreta el texto usando el formato indicado y devuelve un `datetime`.

4.3 DIFERENCIA ENTRE FECHAS (`timedelta`)

Calcula la diferencia entre dos fechas.

```
from datetime import datetime
f1 = datetime(2025, 5, 22, 10, 0, 0)
f2 = datetime(2025, 5, 29, 14, 30, 0)
diff = f2 - f1

print(diff) # 7 days, 4:30:00
print(diff.days) # 7
print(diff.seconds) # 16200 (4*3600+30*60)
print(diff.total_seconds()) # 637200.0
```



El resultado siempre es un objeto `timedelta`. Usa `.days`, `.seconds` o `.total_seconds()` según lo que necesites.

4.4 FECHAS RELATIVAS (sumar / restar)

Suma o resta días, horas, minutos, etc.

```
from datetime import datetime, timedelta
ahora = datetime.now()
mas_5 = ahora + timedelta(days=5)
menos_2h = ahora - timedelta(hours=2)

print(mas_5) # ahora + 5 días
print(menos_2h) # ahora - 2 horas
```

★ `timedelta` acepta:

- days
- hours
- minutes
- seconds
- weeks

4.5 ITERAR ENTRE FECHAS

Recorre todas las fechas entre inicio y fin.

```
from datetime import datetime, timedelta
inicio = datetime(2025, 5, 20)
fin = datetime(2025, 5, 25)
delta = timedelta(days=1)
actual = inicio

while actual <= fin:
    print(actual.strftime("%Y-%m-%d"))
    actual += delta
```

TIP

★ Ideal para generar reportes diarios, calendarios o recordatorios.

4.6 INICIO Y FIN DEL DÍA / MES / AÑO

Función útil para obtener límites de tiempo.

```
from datetime import datetime, timedelta
ahora = datetime.now()
inicio_dia = datetime(ahora.year, ahora.month, ahora.day)
fin_dia = inicio_dia + timedelta(days=1) - timedelta(seconds=1)
inicio_mes = datetime(ahora.year, ahora.month, 1)
if ahora.month == 12:
    inicio_sig_mes = datetime(ahora.year + 1, 1, 1)
else:
    inicio_sig_mes = datetime(ahora.year, ahora.month + 1, 1)
fin_mes = inicio_sig_mes - timedelta(seconds=1)

print("Inicio día:", inicio_dia)
print("Fin día:", fin_dia)
print("Inicio mes:", inicio_mes)
print("Fin mes:", fin_mes)
```



Útil para filtros de reportes y consultas en bases de datos.

4.7 EJEMPLOS PRÁCTICOS

A) Edad exacta de una persona

```
from datetime import date
nacimiento = date(1995, 8, 15)
hoy = date.today()
edad = hoy.year - nacimiento.year - ((hoy.month, hoy.day) < (nacimiento.month, nacimiento.day))

print("Edad:", edad, "años")
```



B) Días restantes para un evento

```
from datetime import date
evento = date(2025, 12, 25)
hoy = date.today()
restantes = (evento - hoy).days

print("Faltan", restantes, "días")
```



C) Próximo lunes

```
from datetime import date, timedelta
hoy = date.today()
dias_para_lunes = (7 - hoy.weekday()) % 7
proximo_lunes = hoy + timedelta(days=dias_para_lunes)

print("Próximo lunes:", proximo_lunes)
```



EN RESUMEN



MANIPULAR

- Sumar, restar y comparar fechas.
- `timedelta` para duraciones.



FORMATEAR / PARSEAR

- `strftime()` para mostrar.
- `strptime()` para convertir texto.



ZONAS HORARIAS

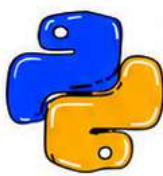
- `timezone.utc`
- Otras zonas con desplazamientos (GMT).



APLICACIONES

- Reportes, logs
- Calendarios
- Automatizaciones





MATH + RANDOM

Volumen 19

1. ¿QUÉ SON?



Python tiene **módulos** que nos permiten trabajar con matemáticas y aleatoriedad.

- **math** → funciones matemáticas avanzadas.
- **random** → generar números y elecciones al azar.



1.1 MÓDULO MATH

Proporciona funciones matemáticas comunes.

```
import math
```

Importante:

math no necesita instalarse, viene incluido en Python.



Funciones comunes de math

Función	Descripción
<code>math.sqrt(x)</code>	Raíz cuadrada
<code>math.pow(x, y)</code>	x elevado a la y
<code>math.fabs(x)</code>	Valor absoluto (float)
<code>math.ceil(x)</code>	Redondeo hacia arriba
<code>math.floor(x)</code>	Redondeo hacia abajo
<code>math.pi</code>	Valor de π (pi)
<code>math.e</code>	Valor de e (número de Euler)
<code>math.sin(x)</code>	Seno de x (en radianes)
<code>math.cos(x)</code>	Coseno de x (en radianes)
<code>math.log(x)</code>	Logaritmo natural (ln x)
<code>math.log10(x)</code>	Logaritmo en base 10

1.2 EJEMPLOS COMUNES CON MATH

Ejemplos con el módulo math

```
import math
```

```
print(math.sqrt(16))      # 4.0
print(math.pow(2, 3))     # 8.0
print(math.fabs(-7))      # 7.0
print(math.ceil(4.2))     # 5
print(math.floor(4.8))    # 4
print(math.pi)            # 3.141592653589793
print(math.e)             # 2.718281828459045
print(math.sin(math.pi/2)) # 1.0 (seno 90°)
print(math.log(1))        # 0.0
print(math.log10(1000))   # 3.0
```



TIP



Los ángulos en las funciones trigonométricas (sin, cos, tan) se trabajan en radianes, no en grados.



1.3 MÓDULO RANDOM

Permite generar números aleatorios y realizar elecciones al azar.

```
import random
```



Funciones comunes de random

Función	Descripción
<code>random.random()</code>	Número decimal aleatorio entre 0.0 y 1.0
<code>random.randint(a, b)</code>	Entero aleatorio entre a y b (incluidos)
<code>random.uniform(a, b)</code>	Decimal aleatorio entre a y b
<code>random.choice(lista)</code>	Elige un elemento aleatorio de una lista
<code>random.shuffle(lista)</code>	Mezcla los elementos de una lista
<code>random.sample(lista, k)</code>	Devuelve k elementos aleatorios sin repetir

1.4 EJEMPLOS CON RANDOM

Ejemplos con el módulo random

```
import random
```

```
print(random.random())      # 0.37482913...
print(random.randint(1, 10)) # 7
print(random.uniform(1.5, 2.5)) # 1.83745...
```

```
opciones = ['piedra', 'papel', 'tijera']
print(random.choice(opciones)) # papel
```

```
lista = [1, 2, 3, 4, 5]
random.shuffle(lista)
print(lista)                # [3, 1, 5, 2, 4]
print(random.sample(lista, 3)) # [2, 5, 1]
```

```
random.seed(n)
```

Fija la semilla del generador aleatorio para obtener siempre los mismos resultados.



Ejemplo:

```
random.seed(42)
print(random.random()) # 0.639426798...
print(random.random()) # 0.025010755...
```



EN RESUMEN



MATH

- Funciones matemáticas avanzadas.
- Cálculos precisos y constantes útiles.
- Ideal para cálculos científicos.



RANDOM

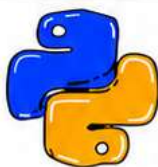
- Genera números y elecciones aleatorias.
- Útil para juegos, simulaciones y pruebas.
- ¡La aleatoriedad hace todo más divertido!



Volumen 19 • Math + Random

Designed & Summarized by GuilleGiu

APRENDIENDO HOY, CRECIENDO SIEMPRE



MATH + RANDOM

Volumen 19

2. USOS PRÁCTICOS Y ÚTILES



Combinando `math` y `random` podemos crear simulaciones, juegos, cálculos útiles y mucho más. ¡La práctica es la clave!



2.1 REDONDEO Y PRECISIÓN

Controla decimales y redondeos.

```
import math
x = 3.1415926

print(round(x, 2))      # 3.14
print(math.floor(x))    # 3 (siempre abajo)
print(math.ceil(x))     # 4 (siempre arriba)
print(format(x, ".3f")) # 3.142 (formateo)
```



★ `round(x, n)` → redondea a `n` decimales
`floor()` → hacia abajo | `ceil()` → hacia arriba



2.2 GENERAR NÚMEROS ALEATORIOS EN RANGOS

Con `random.randint(a, b)` incluimos ambos extremos.

```
import random

# Enteros aleatorios
print(random.randint(1, 10))    # 1 a 10 (incluye 10)
print(random.randint(100, 200)) # 100 a 200

# Flotantes aleatorios
print(random.uniform(1.5, 9.5)) # decimal entre 1.5 y 9.5
```



★ `randint(a, b)` → entero entre `a` y `b` (incluidos)
`uniform(a, b)` → decimal entre `a` y `b`



2.3 ELEGIR ELEMENTOS ALEATORIOS

Ideal para listas, sorteos o juegos.

```
import random

lista = ['rojo', 'azul', 'verde', 'amarillo']

print(random.choice(lista))    # elige uno
print(random.choices(lista, k=2)) # elige 2 (pueden repetirse)
random.shuffle(lista)          # mezcla la lista
print(lista)                   # ['verde', 'azul', ..., ...]
```



Funciones clave	
<code>choice(lista)</code>	Elige un elemento aleatorio.
<code>choices(lista, k)</code>	Elige <code>k</code> elementos (pueden repetirse).
<code>shuffle(lista)</code>	Mezcla el orden de la lista (en el lugar).



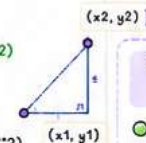
2.4 CÁLCULOS AVANZADOS CON MATH

```
import math

# Distancia entre 2 puntos (x1,y1) y (x2,y2)
x1, y1 = 2, 3
x2, y2 = 6, 8

distancia = math.sqrt((x2-x1)**2 + (y2-y1)**2)
print(distancia)    # 7.0710678118654755

# Conversión de grados a radianes
grados = 60
radianes = math.radians(grados)
print(radianes)    # 1.0471975511965976
```



Otras funciones útiles:

- `math.exp(x)`
- `math.log(x)`
- `math.factorial(n)`
- `math.gcd(a, b)`
- `math.hypot(x, y)`



2.5 SIMULACIÓN: DADO VIRTUAL

Creamos un dado de 6 caras.

```
import random

def tirar_dado():
    return random.randint(1, 6)

for i in range(5):
    print(f"Tirada #{i+1}: {tirar_dado()}")
```



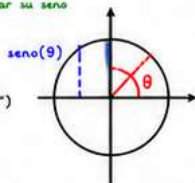
Salida (ejemplo):

Tirada #1: 4
Tirada #2: 2
Tirada #3: 6
Tirada #4: 1
Tirada #5: 5



2.7 COMBINANDO MATH + RANDOM

```
# Generar un ángulo aleatorio (0 a 360) y calcular su seno
import math, random
angulo = random.uniform(0, 360)
seno = math.sin(math.radians(angulo))
print(f"Ángulo: {angulo:.2f} → seno: {seno:.4f}")
```



★ Útil en simulaciones físicas, gráficos, trigonometría y más.



2.6 JUEGO RÁPIDO: ADIVINA EL NÚMERO

```
import random
secreto = random.randint(1, 100)
intentos = 0

print("Adivina un número entre 1 y 100")
while True:
    n = int(input("Tu número: "))
    intentos += 1
    if n == secreto:
        print(f"¡Correcto! Lo lograste en {intentos} intentos.")
        break
    elif n < secreto:
        print("Muy bajo")
    else:
        print("Muy alto")
```



2.8 CASOS PRÁCTICOS RÁPIDOS

A) Moneda: Cara o Cruz

```
import random
moneda = random.choice(["Cara", "Cruz"])
print(moneda)
```



B) Nota aleatoria (ej. examen)

```
import random
nota = random.randint(1, 10)
print(f"Tu nota: {nota}/10")
```

10/10

EN RESUMEN

MATH



- Operaciones avanzadas.
- Trigonometría.
- Redondeos y logaritmos.

RANDOM



- Genera números.
- Elige elementos.
- Mezcla listas.

COMBINACIONES



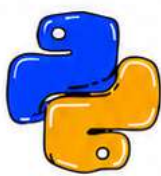
- Juegos y simulaciones.
- Cálculos útiles.
- Más creatividad.

PRÁCTICA



- Experimenta.
- Prueba.
- ¡Diviértete!





MATH + RANDOM

Volumen 19

4. USOS PRÁCTICOS Y COMBINACIONES



Combinando `math` y `random` podemos resolver problemas reales y crear simulaciones, juegos y herramientas útiles.



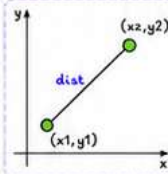
¡La práctica transforma el conocimiento en habilidad!



4.1 DISTANCIA ENTRE DOS PUNTOS

Usamos `math.sqrt()` para aplicar el teorema de Pitágoras.

```
import math
x1, y1 = 2, 3
x2, y2 = 7, 8
dist = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
print(f"Distancia: {dist:.2f}") # 7.07
```



TIP



Ideal para gráficos, simulaciones o cálculos de posición en juegos y robótica.



4.2 CONVERSIÓN DE GRADOS A RADIANES Y VICEVERSA

De grados a radianes

```
import math
grados = 180
rad = math.radians(grados)
print(rad) # 3.141592653589793
```

De radianes a grados

```
import math
rad = math.pi / 2
grados = math.degrees(rad)
print(grados) # 90.0
```

TIP



Esencial al trabajar con trigonometría y gráficos.



4.3 SIMULAR LANZAMIENTO DE DADOS

Con `random.randint()` simulamos resultados reales.

```
import random
# Dado de 6 caras
dado = random.randint(1, 6)
print(f"Resultado del dado: {dado}")
```



Ejemplo de posibles salidas:
1, 2, 3, 4, 5 o 6

4.4 ELEGIR ELEMENTO ALEATORIO DE UNA LISTA

Con `random.choice()` seleccionamos un elemento al azar.

```
import random
opciones = ["piedra", "papel", "tijera"]
eleccion = random.choice(opciones)
print(f"Elegiste: {eleccion}")
```

piedra?
papel?
tijera?



Ejemplo de posibles salidas:
piedra / papel / tijera

4.5 GENERAR NÚMEROS ALEATORIOS CON DECIMALES

Con `random.uniform(a, b)` obtenemos números decimales.

```
import random
valor = random.uniform(0, 1) # entre 0 y 1
print(valor) # 0.37482913...
otro = random.uniform(5.5, 10.5) # entre 5.5 y 10.5
print(otro) # 8.237465...
```

TIP



Útil para porcentajes, probabilidades y simulaciones.



4.6 REDONDEO Y PRECISIÓN EN CÁLCULOS

Combinamos redondeo con constantes para precisión.

```
import math
pi_aprox = math.pi
area = pi_aprox * (5 ** 2)
print(f"Área círculo radio 5: {area}")
print(f"Redondeado (2 dec): {round(area, 2)}")
```

Salida:
78.53981633974483
Redondeado (2 dec):
78.54



TIP



Redondear mejora la presentación y evita errores de precisión en comparaciones.



4.7 MINI PROYECTO: JUEGO ADIVINA EL NÚMERO

La computadora elige un número secreto y el usuario intenta adivinarlo.

- Número entre 1 y 100
- El juego da pistas si es mayor o menor
- Cuenta los intentos



```
import random
secreto = random.randint(1, 100)
intentos = 0
print("🎯 Adivina el número entre 1 y 100")
while True:
    intento = int(input("Tu número: "))
    intentos += 1
    if intento < secreto:
        print("👉 Es mayor")
    elif intento == secreto:
        print("🎉 ¡Correcto!")
    else:
        print("👈 Es menor")
    if intento == secreto:
        print(f"¡Correcto! Era {secreto}")
        print(f"Lo lograste en {intentos} intentos")
        break
```

Ejemplo de ejecución:

🎯 Adivina el número entre 1 y 100
Tu número: 50
Es mayor
Tu número: 75
Es menor
Tu número: 62
¡Correcto! Era 62
Lo lograste en 3 intentos 😊

Ideas para mejorar:

- Limitar intentos
- Niveles de dificultad
- Guardar récords
- Interfaz gráfica



EN RESUMEN



MATH

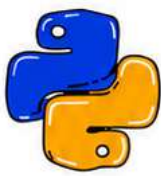
- Módulo para funciones matemáticas avanzadas.
- Cálculos precisos y constantes útiles.
- Ideal para ciencias, ingeniería y análisis.



RANDOM

- Genera números y elecciones al azar.
- Perfecto para juegos, simulaciones y pruebas.
- ¡La aleatoriedad hace todo más interesante!





{ JSON }

Módulo 20

Índice del módulo

- Qué es JSON
- Módulo json
- Lectura y escritura
- Ejemplos prácticos

1. ¿QUÉ ES JSON?



JSON (JavaScript Object Notation) es un formato ligero para intercambiar y almacenar datos.

- ✓ Fácil de leer para humanos.
- ✓ Fácil de generar y parsear en Python.
- ✓ Basado en pares clave-valor y estructuras.



1.1 ESTRUCTURA DE JSON

Utiliza 2 estructuras principales:

- **Objeto** → { clave: valor, ... }
Similar a un diccionario en Python.
- **Arreglo (lista)** → [valor, valor, ...]
Similar a una lista en Python.

Valores permitidos:

- string → "texto"
- number → 123, 45.6
- boolean → true, false
- null → null
- object → { ... }
- array → [...]



1.2 EJEMPLO DE JSON

Ejemplo de un objeto JSON:

```
{
  "nombre": "Guille",
  "edad": 32,
  "programador": true,
  "lenguajes": ["Python", "SQL", "JavaScript"],
  "direccion": {
    "ciudad": "Buenos Aires",
    "pais": "Argentina"
  },
  "proyectos": null
}
```

★ JSON es independiente del lenguaje, por eso se usa en APIs, archivos de configuración, bases de datos, etc. ♡

1.3 MÓDULO json EN PYTHON

Python tiene el módulo integrado **json** para trabajar con este formato.

```
import json
```

No es necesario instalar nada, viene incluido en Python.

★ Funciones principales:

- **json.dumps()** → convierte un objeto Python a JSON (string).
- **json.loads()** → convierte un JSON (string) a objeto Python.
- **json.dump()** → guarda un objeto Python en un archivo JSON.
- **json.load()** → lee un archivo JSON y lo convierte a Python.

1.4 DE PYTHON A JSON (SERIALIZAR)

Convertimos objetos Python a JSON (string)

```
import json
datos = {
  "nombre": "Guille",
  "edad": 32,
  "lenguajes": ["Python", "SQL"]
}
texto_json = json.dumps(datos, indent=4)
print(texto_json) # indent=4 → formato legible
```

Salida:

```
{
  "nombre": "Guille",
  "edad": 32,
  "lenguajes": [
    "Python",
    "SQL"
  ]
}
```

Ahora es un string JSON listo para enviar o guardar.

1.5 DE JSON A PYTHON (DESERIALIZAR)

Convertimos un string JSON a objetos Python.

```
import json
texto_json = '{"nombre": "Guille", "edad": 32}'
datos = json.loads(texto_json)
print(type(datos))
print(datos["nombre"]) # Guille
print(datos["edad"] + 1) # 33
```

TIP

★ Después de **json.loads()**, trabajas con un diccionario (objeto) o lista (array) de Python normal. 😊

EN RESUMEN



JSON

- Formato de texto ligero y universal.
- Usa objetos {} y arreglos [].
- Ideal para intercambio de datos.



PYTHON + json

- dumps / loads → con strings.
- dump / load → con archivos.
- Fácil, rápido y muy útil. ♡





{ JSON }

2. LECTURA Y ESCRITURA DE ARCHIVOS JSON



Podemos guardar datos en **archivos JSON** para luego leerlos y usarlos en nuestros programas.

- ✓ Útil para guardar configuraciones.
- ✓ Intercambiar datos entre sistemas.
- ✓ Persistencia ligera y legible.



2.1 ESCRIBIR EN UN ARCHIVO JSON

Usamos `json.dump()` para escribir datos en un archivo.

```
import json
datos = {
    "nombre": "Guille",
    "edad": 32,
    "lenguajes": ["Python", "SQL", "JavaScript"]
}

# Abrimos el archivo en modo escritura
with open("datos.json", "w", encoding="utf-8") as f:
    json.dump(datos, f, indent=4)

print("Archivo datos_json creado ✅")
```

Salida:

Archivo datos_json creado ✅



TIP

- ★ El parámetro `indent=4` hace que el archivo JSON quede formateado y legible.



2.2 LEER UN ARCHIVO JSON

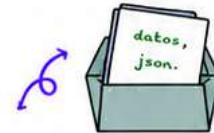
Usamos `json.load()` para leer el contenido del archivo.

```
import json
with open("datos.json", "r", encoding="utf-8") as f:
    datos = json.load(f)

print(type(datos))      # <class 'dict'>
print(datos["nombre"])  # Guille
print(datos["lenguajes"][1]) # SQL
```

Salida:

```
<class 'dict'>
Guille
SQL
```



`json.load()` devuelve un objeto de Python:
dict si el JSON es un objeto,
list si el JSON es un arreglo.



2.3 MODOS DE APERTURA MÁS COMUNES

Modo	Descripción	Ejemplo
"r"	Leer un archivo (debe existir).	<code>open("archivo.json", "r")</code>
"w"	Escribir (sobrescribe si existe).	<code>open("archivo.json", "w")</code>
"a"	Agregar al final del archivo (útil para logs).	<code>open("archivo.json", "a")</code>



NOTA

Siempre usa `encoding="utf-8"` para evitar problemas con acentos y caracteres especiales. 😊

2.4 EJEMPLOS PRÁCTICOS

GUARDAR UNA LISTA DE USUARIOS

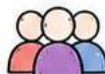
```
import json
usuarios = [
    {"id": 1, "nombre": "Ana"},
    {"id": 2, "nombre": "Luis"},
    {"id": 3, "nombre": "Marta"}
]

with open("usuarios.json", "w", encoding="utf-8") as f:
    json.dump(usuarios, f, indent=4)

print("Usuarios guardados ✅")
```

Salida:

Usuarios guardados ✅



LEER Y RECORRER UNA LISTA JSON

```
import json
with open("usuarios.json", "r", encoding="utf-8") as f:
    usuarios = json.load(f)

for u in usuarios:
    print(f"ID: {u['id']} - Nombre: {u['nombre']}")
```

Salida:

```
ID: 1 - Nombre: Ana
ID: 2 - Nombre: Luis
ID: 3 - Nombre: Marta
```



ACTUALIZAR Y VOLVER A GUARDAR

```
import json
with open("usuarios.json", "r", encoding="utf-8") as f:
    usuarios = json.load(f)

# Agregamos un nuevo usuario
usuarios.append({"id": 4, "nombre": "Leo"})

with open("usuarios.json", "w", encoding="utf-8") as f:
    json.dump(usuarios, f, indent=4)

print("Usuario agregado ✅")
```

Salida:

Usuario agregados ✅



EN RESUMEN



- JSON es un formato ligero y legible.
- En Python trabajamos con el módulo `json`.
- Podemos convertir datos Python ↔ JSON.



- `dump(s)` → Python a JSON (string o archivo).
- `load(s)` → JSON a Python (string o archivo).
- Ideal para guardar, compartir y configurar datos.





{ REGEX }

EXPRESIONES REGULARES

1. ¿QUÉ ES REGEX?

Página 01

Volumen 21

Índice del módulo
• ¿Qué es Regex?
• Sintaxis básica
• Caracteres especiales
• Ejemplos prácticos



Las expresiones regulares (Regex) son patrones que permiten buscar, validar y manipular texto de forma poderosa y flexible.

- ✓ Validar emails, teléfonos, URLs, contraseñas, etc.
- ✓ Extraer información de textos.
- ✓ Reemplazar o limpiar datos.



1.1 ¿DÓNDE SE USA?

En Python usamos el módulo `re` para trabajar con Regex.

```
import re
# buscar coincidencias
re.search(patron, texto)
# encontrar todas las coincidencias
re.findall(patron, texto)
# reemplazar texto
re.sub(patron, reemplazo, texto)
# dividir texto
re.split(patron, texto)
# validar patrón completo
re.fullmatch(patron, texto)
```



1.2 EJEMPLO RÁPIDO

Buscamos todos los números en una cadena.

```
import re
texto = "Hoy tengo 2 tareas y 5 pendientes."
numeros = re.findall(r"\d+", texto)
print(numeros) # ['2', '5']
```

TIP

Los patrones se escriben entre comillas normales o crudas (r"..."). Usar r"..." evita escapar barras \.

2. SINTAXIS BÁSICA

Símbolo	Significado	Ejemplo	Coincide con
.	Cualquier carácter (excepto salto de línea)	a,c	abc, a!c, a~c
^	Inicio de la cadena	^Hola	Hola mundo, Hola123
\$	Fin de la cadena	mundo\$	Hola mundo
*	0 o más repeticiones	ab*c	ac, abc, abb!c
+	1 o más repeticiones	ab+c	abc, abbc (no ac)
?	0 o 1 repetición (opcional)	colou?r	color, colour
{n}	Exactamente n repeticiones	a{3}	aaa
{n,}	n o más repeticiones	a{2,}	aa, aaa, aaaa
{n,m}	Entre n y m repeticiones	a{2,4}	aa, aaa, aaaa
\	Escapar caracteres especiales	\.	coincide con el punto (.) literal

NOTA

Los patrones son sensibles a mayúsculas y minúsculas.

3. CLASES DE CARACTERES

Patrón	Significado	Ejemplo	Coincide con
\d	Dígito (0-9)	\d+	123, 45
\D	No dígito	\D+	abc, %, -
\w	Palabra (letras, dígitos, guion bajo)	\w+	python_123, hola
\W	No palabra	\W+	!@#, ., -
\s	Espacio en blanco	\s+	, \t, \n
\S	No espacio en blanco	\S+	a, 1, @
[abc]	Cualquier carácter entre corchetes	[abc]+	a, b, c
[^abc]	Cualquier carácter excepto los del set	[^abc]+	d, 1, @
range	Rango de caracteres	[a-z]	letras minúsculas

4. EJEMPLOS PRÁCTICOS

✓ Validar email básico

```
patron = r"^[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]$"
email = "guille@gmail.com"
re.fullmatch(patron, email) # Match
```

✓ Extraer fechas (dd/mm/yyyy)

```
texto = "Reunión el 21/05/2025."
fechas = re.findall(r"\d{2}/\d{2}/\d{4}", texto)
print(fechas) # ['21/05/2025']
```

✓ Reemplazar múltiples espacios por uno solo

```
texto = "Hola mundo de Python"
limpio = re.sub(r"\s+", " ", texto)
print(limpio) # Hola mundo de Python
```

TIP

Usa r"..." para evitar escribir \\. Ejemplo: r"\d+" es mejor que "\\d+"

EN RESUMEN



- Regex = patrones para trabajar con texto.
- Potente para buscar, validar y transformar.
- En Python usamos el módulo `re`.

Empieza simple y ve combinando caracteres especiales según lo que necesites.





{ REGEX }

EXPRESIONES REGULARES

2. PATRONES AVANZADOS

Página 02

Volumen 21

Índice del módulo

- ¿Qué es Regex?
- Sintaxis básica
- Caracteres especiales
- Ejemplos prácticos



Regex se vuelve muy **potente** cuando combinamos símbolos, clases y grupos para resolver problemas reales.

- ✓ Buscar con precisión.
- ✓ Capturar partes específicas.
- ✓ Validar formatos complejos.
- ✓ Reemplazar y transformar texto.



2.1 CLASES DE CARACTERES (Continuación)

Patrón	Significado	Ejemplo	Coincide con
[aeiou]	Vocal minúscula	[aeiou]	a, e, i, o, u
[A-Z]	Cualquier mayúscula	python[A-Z]	pythonC
[0-9]	Cualquier dígito	orden[0-9]+	orden123
[^abc]	Cualquier carácter excepto a, b o c	[^abc]+	def, xyz, 123
[^0-9]	Cualquier NO dígito	[^\d]+	abc, !@#, -

TIP

- ★ El símbolo ^ dentro de [] niega el conjunto.
- Fuera de [] indica inicio de cadena.

2.2 GRUPOS Y CAPTURAS

Los grupos () permiten capturar partes del texto.

```
import re
texto = "Nombre: Ana Edad: 28"
patron = r"Nombre:\s*(\w+)\s+Edad:\s*(\d+)"
match = re.search(patron, texto)
if match:
    nombre = match.group(1)
    edad = match.group(2)
    print(nombre) # Ana
    print(edad) # 28
```

Métodos útiles

- group(0) → todo el match
- group(1) → primer grupo
- groups() → todos los grupos
- groupdict() → con nombres (ver 2.4)

TIP

- ★ Los grupos capturan por posición: (1), (2), (3)...

2.3 CUANTIFICADORES (Repeticiones)

Definen cuántas veces puede repetirse un patrón.

Patrón	Significado	Ejemplo	Coincide con
*	0 o más	ab*c	ac, abc, abbcc
+	1 o más	ab+c	abc, abbcc
?	0 o 1 (opcional)	colou?r	color, colour
{n}	Exactamente n	a{3}	aaa
{n,}	n o más	a{2,}	aa, aaa, aaaa
{n,m}	Entre n y m	a{2,4}	aa, aaa, aaaa
? ??	No codiciosos (lazy)	a.?b	coincidencia mínima

TIP

- ★ Por defecto, los cuantificadores son codiciosos (greedy) y toman lo máximo posible.

2.4 GRUPOS CON NOMBRE

Permiten identificar grupos por nombre (más legible).

```
import re
texto = "Email: guille@gmail.com"
patron = r"^(?P<usuario>\w+)@(?P<dominio>\w+\.\w+)$"
match = re.search(patron, texto)
if match:
    print(match.group('usuario')) # guille
    print(match.group('dominio')) # gmail.com
    print(match.groupdict())
    # {'usuario': 'guille', 'dominio': 'gmail.com'}
```

TIP

- ★ Usamos (?P<nombre>patron) para nombrar el grupo.

2.5 EJEMPLOS PRÁCTICOS REALES

VALIDAR TELÉFONO ARGENTINO (10 dígitos)

```
import re
telefono = "11 1234-5678"
patron = r"^\d{2}[\s-]?\d{4}[\s-]?\d{4}$"
print(bool(re.match(patron, telefono)))
# True
```

Acepta formatos:
1123456788, 11-1234-5678, 11 1234 5678



EXTRAER URLS DE UN TEXTO

```
import re
texto = "Visita https://python.org o http://github.com para más info."
patron = r"https?://[\w.-]+(?:/[\w.-]+)?"
urls = re.findall(patron, texto)
print(urls)
# ['https://python.org', 'http://github.com']
```



VALIDAR CONTRASEÑA SEGURA

- ✓ Al menos 8 caracteres
- ✓ Al menos 1 mayúscula
- ✓ Al menos 1 número
- ✓ Al menos 1 carácter especial

```
import re
password = "Guille2025!"
patron = r"^(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*])(?=.{8,})$"
print(bool(re.match(patron, password)))
# True
```



EN RESUMEN



- Regex = patrones para buscar y manipular texto.
- Muchos símbolos, clases y cuantificadores.
- En Python usamos el módulo **re**.



- Extremadamente útil en el mundo real.
- Ahorra tiempo y evita errores.
- ¡Practicar es la clave para dominarlo!



Volumen 21 • REGEX

Designed & Summarized by GuilleGiu

APRENDIENDO HOY, CRECIENDO SIEMPRE



{ TRY / EXCEPT }

MANEJO DE ERRORES

Volumen 22

Índice del módulo

- ¿Qué es Try/Except?
- Sintaxis básica
- Cláusulas opcionales
- Tipos de errores comunes
- Ejemplos prácticos

1. ¿QUÉ ES TRY / EXCEPT?



Try / Except nos permite manejar **errores** (excepciones) en tiempo de ejecución para que nuestro programa no se detenga inesperadamente.

- ✓ Evita que el programa crashee.
- ✓ Permite controlar qué hacer ante un error.
- ✓ Mejora la experiencia del usuario.
- ✓ Puedes registrar, limpiar o recuperarte del error.



1.1 ¿QUÉ ES UNA EXCEPCIÓN?

Es un error que ocurre durante la ejecución del programa.

Ejemplos:

- División por cero.
- Convertir texto a número inválido.
- Abrir un archivo que no existe.
- Acceder a un índice fuera de rango.



TIP



No todos los errores se pueden predecir, pero podemos prepararnos para los más comunes.



1.2 SINTAXIS BÁSICA

Estructura mínima:

```
try:
    # Código que puede generar un error
    ...
except:
    # Código que se ejecuta si ocurre un error
```



El bloque **try** se ejecuta primero.

Si ocurre un error, el bloque **except** se ejecuta.

El programa continúa después del bloque **except**.



1.3 EJEMPLO BÁSICO

Manejo de división por cero:

```
try:
    num = 10
    div = num / 0
    print(div)
except:
    print("No se puede dividir por cero.")
print("El programa continúa...")
```



Salida:

No se puede dividir por cero.
El programa continúa...



TIP



Usar try/except evita que el programa se detenga bruscamente y permite mostrar mensajes amigables al usuario.



1.4 CAPTURAR TIPO ESPECÍFICO DE ERROR

Es recomendable capturar solo los errores que esperamos.

```
try:
    num = int(input("Ingresa un número: "))
    print(num * 2)
except ValueError:
    print("Eso no es un número válido.")
```



Errores comunes:

- **ValueError** → valor inválido
- **ZeroDivisionError** → división por cero
- **FileNotFoundException** → archivo no encontrado
- **IndexError** → índice fuera de rango

1.5 MÚLTIPLES EXCEPT

Podemos manejar diferentes errores.

```
try:
    num = int(input("Número: "))
    resultado = 10 / num
    print(resultado)
except ValueError:
    print("Ingresa solo números.")
except ZeroDivisionError:
    print("No se puede dividir por cero.")
```

TIP

El orden importa: Python ejecuta el primer **except** que coincide.



1.6 ELSE Y FINALLY

Cláusulas opcionales muy útiles.

```
try:
    archivo = open("datos.txt", "r")
    contenido = archivo.read()
except FileNotFoundError:
    print("El archivo no existe.")
else:
    print("Archivo leído correctamente.")
finally:
    print("Este código siempre se ejecuta.")
```

else: se ejecuta si NO hubo errores.

finally: se ejecuta SIEMPRE, haya o no error.



EN RESUMEN



- **try** → código que puede fallar.
- **except** → maneja el error.
- **else** → se ejecuta si no hubo errores.
- **finally** → se ejecuta siempre.



- Mejora la robustez de tus programas.
- Evita cierres inesperados.
- Permite dar mensajes claros al usuario.





{ ARCHIVOS }

FILE HANDLING

Volumen 23

Índice del módulo

- 1. ¿Qué son los archivos?
- 2. Abrir y cerrar archivos
- 3. Modos de apertura
- 4. Leer archivos
- 5. Escribir archivos
- 6. Ejemplos prácticos

1. ¿QUÉ SON LOS ARCHIVOS?



Los archivos permiten guardar información de forma **permanente** en el disco. Python nos brinda herramientas simples para crear, leer, escribir y modificar archivos.

- ✓ Guardar datos para usarlos luego.
- ✓ Trabajar con archivos de texto o binarios.
- ✓ Leer, escribir, agregar o modificar contenido.
- ✓ Manejar datos grandes sin usar mucha memoria.



1.1 ABRIR Y CERRAR UN ARCHIVO

Usamos la función `open()` para abrir un archivo.

```
# Abrir un archivo en modo lectura
archivo = open("ejemplo.txt", "r")
# ... realizar operaciones ...
archivo.close() # Cerrar el archivo
```



SIEMPRE
cierra el archivo
después de
usarlo.



1.2 SINTAXIS RECOMENDADA (WITH)

La forma más segura y práctica:

```
with open("ejemplo.txt", "r") as archivo:
    contenido = archivo.read()
    print(contenido)
```



El archivo se cierra automáticamente al salir del bloque `with`.



1.3 MODOS DE APERTURA MÁS COMUNES

Modo	Significado	¿Existe el archivo?	Descripción
"r"	Read (Leer)	Debe existir	Abre para leer. Error si no existe.
"w"	Write (Escribir)	No es obligatorio	Crea o sobrescribe el archivo.
"a"	Append (Agregar)	No es obligatorio	Crea el archivo o agrega al final.
"r+"	Read + Write	Debe existir	Leer y escribir desde el inicio.
"w+"	Write + Read	No es obligatorio	Escribir y leer. Sobrescribe el archivo.
"a+"	Append + Read	No es obligatorio	Agregar al final y también leer.

TIP

★ Puedes agregar `"b"` al final para trabajar con archivos binarios.

Ejemplo:

`"rb", "wb", "ab"`



1.4 LEER UN ARCHIVO

Métodos más usados para leer:

- `read()` → lee todo el contenido.
- `readline()` → lee una línea.
- `readlines()` → lee todas las líneas y las devuelve como lista.

```
with open("ejemplo.txt", "r") as archivo:
    todo = archivo.read() # todo el contenido
    linea = archivo.readline() # una línea
    lineas = archivo.readlines() # lista de líneas
```

Salida ejemplo:

Hola mundo
Python es genial
Sigue aprendiendo

→ `read()` → "Hola mundo\nPython es genial\nSigue aprendiendo\n"
→ `readline()` → "Hola mundo\n"
→ `readlines()` → ["Hola mundo\n", "Python es genial\n", "Sigue aprendiendo\n"]

1.5 ESCRIBIR EN UN ARCHIVO

Modos `"w"` (sobrescribe) y `"a"` (agrega):

```
# Sobrescribir
with open("datos.txt", "w") as f:
    f.write("Hola archivo\n")
    f.write("Nueva línea\n")

# Agregar al final
with open("datos.txt", "a") as f:
    f.write("Más contenido\n")
```

TIP

★ `write()` no agrega salto de línea. Usa `"\n"` para saltar de línea.



1.6 EJEMPLO COMPLETO

Leer, modificar y guardar.

```
# Leer contenido
with open("notas.txt", "r") as f:
    contenido = f.read()

# Modificar
contenido = contenido.upper()

# Guardar cambios
with open("notas.txt", "w") as f:
    f.write(contenido)
```



★ Primero lee todo, luego modifica y por último escribe el resultado.

EN RESUMEN



- Los archivos guardan datos de forma permanente.
- Usa `open()` para abrir y `close()` o `with` para cerrar.
- Elige el modo correcto según lo que necesites hacer.
- Lee con `read()`, `readline()`, `readlines()`.
- Escribe con `write()` y agrega con `"a"`.



- Organiza tus datos en archivos.
- Trabaja con grandes cantidades de información.
- Es esencial para proyectos reales.





{ ARCHIVOS }

FILE HANDLING

2. TRABAJANDO CON ARCHIVOS

Índice del módulo

- Modos de apertura
- Leer archivos
- Escribir y agregar
- Context manager (with)
- Ejemplos prácticos



Elegir el modo correcto de apertura es clave para trabajar con archivos de forma segura y eficiente.

- ✓ Usar el modo adecuado para cada tarea.
- ✓ Cerrar siempre los archivos (o usar with).
- ✓ Evita sobrescribir sin querer.
- ✓ Maneja excepciones al trabajar con archivos.



2.1 MODOS DE APERTURA (RESUMEN)

Modo	Descripción	¿Existe?	Posición inicial
"r"	Leer (read)	Debe existir	Inicio
"w"	Escribir (write)	No es obligatorio (Sobrescribe)	Inicio
"a"	Agregar (append)	No es obligatorio (Crea si no existe)	Final
"r+"	Leer y escribir	Debe existir	Inicio
"w+"	Leer y escribir (Sobrescribe)	No es obligatorio	Inicio
"a+"	Leer y agregar	No es obligatorio (Crea si no existe)	Final

TIP

★ Usa 'r' para leer, 'w' para escribir (sobrescribe), y 'a' para agregar al final.

2.2 LEER UN ARCHIVO (MÉTODOS)

- `read()` → lee todo el contenido.

```
with open("archivo.txt", "r") as f:  
    todo = f.read() # str con todo el contenido
```
- `readline()` → lee una línea.

```
with open("archivo.txt", "r") as f:  
    linea = f.readline() # str con una línea
```
- `readlines()` → lee todas las líneas y devuelve una lista.

```
with open("archivo.txt", "r") as f:  
    lineas = f.readlines() # list[str]
```

Ejemplo de archivo (archivo.txt):

Hola mundo
Python es genial
Sigue aprendiendo



2.3 ESCRIBIR EN UN ARCHIVO

- `write()` → escribe una cadena. (Sobrescribe si existe)

```
with open("notas.txt", "w") as f:  
    f.write("Hola GuilleGiu!\n")  
    f.write("Nueva línea\n")
```

- `writelines()` → escribe una lista de líneas.

```
lineas = ["Línea 1\n", "Línea 2\n", "Línea 3\n"]  
with open("notas.txt", "w") as f:  
    f.writelines(lineas)
```

TIP

★ El modo 'w' borra el contenido anterior del archivo si ya existía.

2.4 AGREGAR AL FINAL

- `append()` con el modo 'a' agrega sin borrar.

```
with open("notas.txt", "a") as f:  
    f.write("Nueva nota al final\n")
```

Contenido final de notas.txt:

Hola GuilleGiu!
Nueva línea
Nueva nota al final

TIP

★ El modo 'a' mueve el cursor automáticamente al final.

2.5 COMBINAR LECTURA Y ESCRITURA

Usamos "r+", "w+", "a+" según lo que necesitemos.

```
with open("datos.txt", "r+") as f:  
    contenido = f.read() # leer todo  
    f.write("\nOtra línea") # escribir al final  
    f.seek(0) # volver al inicio  
    print(f.read()) # leer nuevamente
```

`seek(pos)` → mueve el cursor a la posición indicada.
`tell()` → devuelve la posición actual del cursor.



2.6 USAR CONTEXT MANAGER (WITH) - RECOMENDADO

El bloque `with` se asegura de cerrar el archivo automáticamente, incluso si ocurre un error.

- ✓ Correcto (usa `with`)

```
with open("ejemplo.txt", "r") as f:  
    datos = f.read()  
# El archivo se cierra solo ☑
```

- ✗ Menos seguro (sin `with`)

```
f = open("ejemplo.txt", "r")  
datos = f.read()  
f.close() # si olvido esto, el  
# archivo puede quedar abierto
```

Ejemplo completo:

```
with open("log.txt", "a") as log:  
    log.write("Inicio del programa\n")  
    log.write("Todo salió bien 😊\n")
```



EN RESUMEN

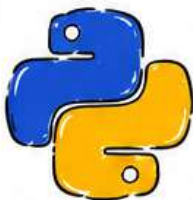


- Los archivos guardan información de forma permanente.
- Usa `open()` con el modo adecuado.
- Leer: `read()`, `readline()`, `readlines()`.
- Escribir: `write()`, `writelines()`, `append` con 'a'.
- Usa `with` para cerrar archivos automáticamente.



- Elige el modo correcto.
- Maneja archivos con cuidado.
- Usa `with` siempre que puedas.
- Practica para dominarlo!





{ POO BÁSICO }

PROGRAMACIÓN ORIENTADA A OBJETOS

1. ¿QUÉ ES POO?

Página 01

Volumen 24

Índice del módulo

- ¿Qué es POO?
- Clases y Objetos
- `__init__` self
- Crear y usar objetos
- Ejemplo práctico



POO (Programación Orientada a Objetos) es un paradigma que permite organizar el código en objetos que representan cosas del mundo real.

- ✓ Organiza mejor tu código.
- ✓ Es más fácil de entender y mantener.
- ✓ Permite reutilizar código (clases).
- ✓ Se parece a cómo pensamos en la vida real.



1.1 ¿QUÉ ES UNA CLASE?

Una clase es como un "molde" o plantilla para crear objetos.

Ejemplo: Clase Perro

```
class Perro:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
```



TIP



Piensa en la clase como el plano de una casa. Define qué tiene y cómo es.



1.2 ¿QUÉ ES UN OBJETO?

Un objeto es una instancia de una clase. Es una "copia" creada a partir del molde.

Ejemplo:

```
mi_perro = Perro("Firulais", 3)
otro_perro = Perro("Max", 5)
```



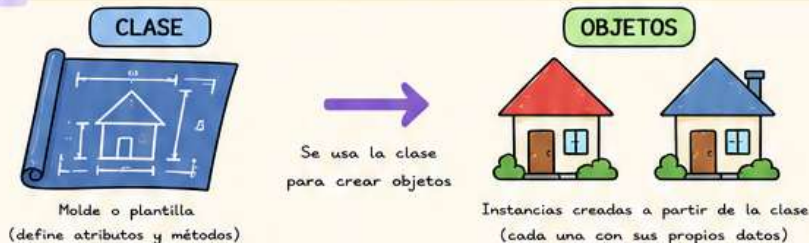
TIP



Piensa en el objeto como la casa construida a partir del plano.



1.3 DIAGRAMA VISUAL



1.4 `__init__` Y self (CONSTRUCTOR)

- `__init__` es un método especial que se ejecuta automáticamente al crear un objeto.
- `self` hace referencia al objeto actual.

Ejemplo:

```
class Perro:
    def __init__(self, nombre, edad):
        self.nombre = nombre # Atributo de instancia
        self.edad = edad # Atributo de instancia
```

TIP



`self.NO` es una palabra reservada. Por convención siempre se usa "self".



1.5 CREAR Y USAR OBJETOS

Crear objetos:

```
mi_perro = Perro("Firulais", 3)
otro_perro = Perro("Max", 5)
```

Acceder a atributos:

```
print(mi_perro.nombre) # Firulais
print(mi_perro.edad) # 3
print(otro_perro.nombre) # Max
print(otro_perro.edad) # 5
```

TIP



Cada objeto guarda sus propios datos. Cambiar uno NO cambia el otro.



EN RESUMEN

- ✓ Una CLASE es el molde.
- ✓ Un OBJETO es una copia creada de la clase.
- ✓ `__init__` inicializa los datos del objeto.
- ✓ `self` representa al objeto actual.
- ✓ Podemos crear muchos objetos a partir de una clase.



Volumen 24 • POO Básico

Designed & Summarized by GuilleGiu

APRENDIENDO HOY, CRECIENDO SIEMPRE



{ POO BÁSICO }

PROGRAMACIÓN ORIENTADA A OBJETOS

2. ATRIBUTOS Y MÉTODOS

Página 02

Volumen 24

Índice del módulo

- Atributos
- Métodos
- Atributos de instancia
- Ejemplo completo
- Diferencia clave



Los **atributos** guardan datos.
Los **métodos** son funciones que realizan acciones.

- ✓ Los objetos tienen sus propios datos.
- ✓ Podemos acceder y modificar esos datos.
- ✓ Los métodos usan los datos del objeto.



2.1 ¿QUÉ ES UN ATRIBUTO?

Es una variable que pertenece a un objeto.
Cada objeto puede tener valores diferentes.

Ejemplo:

```
mi_perro = Perro("Firulais", 3)
print(mi_perro.nombre) # Firulais
print(mi_perro.edad)   # 3
```

Cada perro tiene su propio nombre y edad.

TIP



Los atributos son datos del objeto.
Ejemplos: nombre, edad, color, tamaño.



2.2 ¿QUÉ ES UN MÉTODO?

Es una función que pertenece a la clase.
Los métodos realizan acciones usando los datos del objeto.

Ejemplo:

```
class Perro:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
    def ladrar(self):
        print(f"{self.nombre} dice: ¡Guau guau! 🐶")
```

El método ladrar() usa el nombre del perro.

TIP



Los métodos son acciones del objeto.
Ejemplos: ladrar(), comer(), correr().



2.3 EJEMPLO COMPLETO

```
class Perro:
    def __init__(self, nombre, edad):
        self.nombre = nombre # atributo
        self.edad = edad     # atributo
    def ladrar(self):
        print(f"{self.nombre} dice: ¡Guau guau!")
    def presentarse(self):
        print(f"Hola, soy {self.nombre} y tengo {self.edad} años.")

# Crear objetos
mi_perro = Perro("Firulais", 3)
otro_perro = Perro("Max", 5)

# Usar atributos
print(mi_perro.nombre) # Firulais
print(otro_perro.edad) # 5

# Usar métodos
mi_perro.ladrar() # Firulais dice: ¡Guau guau!
otro_perro.presentarse() # Hola, soy Max y tengo 5 años.
```



Creamos la clase **Perro** con:

- 2 atributos: nombre y edad
- 2 métodos: **ladrar()** y **presentarse()**



Creamos 2 objetos:

- **mi_perro** → Firulais, 3 años
- **otro_perro** → Max, 5 años



Accedemos a atributos:

- **mi_perro.nombre** → Firulais
- **otro_perro.edad** → 5



Usamos métodos:

- **mi_perro.ladrar()**
- **otro_perro.presentarse()**

2.4 ATRIBUTO vs MÉTODO

	ATRIBUTO	MÉTODO
❓ ¿Qué es?	Dato que pertenece al objeto.	Función que pertenece al objeto.
</> Ejemplo	mi_perro.nombre	mi_perro.ladrar()
() ¿Tiene ()?	No	Sí
🔄 Devuelve algo	Sí, el valor del dato.	Puede mostrar, calcular o modificar algo.
🏷️ Tipo	Variable	Función

TIP



Piensa en un objeto como una persona:

- Atributos → datos (nombre, edad, altura)
- Métodos → acciones (caminar, hablar, correr)



EN RESUMEN

- ✓ Los atributos guardan información del objeto.
- ✓ Los métodos realizan acciones usando esos datos.
- ✓ Cada objeto tiene sus propios valores.



- ✓ Accedemos con punto: objeto.atributo
- ✓ Ejecutamos métodos con (): objeto.metodo()
- ✓ POO nos ayuda a organizar y reutilizar código.



Volumen 24 • POO Básico

Designed & Summarized by [GuilleGiu](#)

APRENDIENDO HOY, CRECIENDO SIEMPRE



{ POO BÁSICO }

PROGRAMACIÓN ORIENTADA A OBJETOS

3. RESUMEN Y EJEMPLO COMPLETO

Página 03

Volumen 24

Índice del módulo

- Resumen general
- Vista previa
- Ejemplo completo
- Diagrama de flujo
- Puntos clave



Con lo aprendido en este volumen ya puedes crear **clases**, **objetos**, **atributos** y **métodos** en Python.

- ✓ Creamos moldes (clases) para nuestros objetos.
- ✓ Cada objeto tiene sus propios datos.
- ✓ Los métodos nos permiten realizar acciones.
- ✓ Todo esto nos ayuda a escribir código organizado y reutilizable.



3.1 EJEMPLO COMPLETO: CLASE PERSONA

```
class Persona:
    def __init__(self, nombre, edad):
        # Atributos de instancia
        self.nombre = nombre
        self.edad = edad

    def presentarse(self):
        # Método que muestra información
        print(f"Hola, soy {self.nombre} y tengo {self.edad} años.")

    def cumplir_anios(self):
        # Método que modifica un atributo
        self.edad += 1
        print(f"¡{self.nombre} ahora tiene {self.edad} años! 🎉")
```

```
# --- USANDO LA CLASE---
# Crear objetos (instancias)
juana = Persona("Juan", 20)
maria = Persona("María", 25)

# Usar métodos
juana.presentarse()
maria.presentarse()

# Modificar datos con métodos
juana.cumplir_anios()
juana.presentarse()
```

Cada objeto tiene sus propios datos. Cambiar uno NO cambia el otro.

Salida en consola:

```
Hola, soy Juan y tengo 20 años.
Hola, soy María y tengo 25 años.
¡Juan ahora tiene 21 años! 🎉
Hola, soy Juan y tengo 21 años.
```



3.2 FLUJO DE CONCEPTOS

CLASE

Es el molde o plantilla. Define qué datos (atributos) y qué acciones (métodos) tendrá cada objeto.



OBJETO

Es una instancia creada a partir de la clase. Tiene sus propios valores.



ATRIBUTOS

Guardan información del objeto. Cada objeto tiene los suyos.



MÉTODOS

Son funciones que trabajan con los datos del objeto. Permiten realizar acciones.



TODO JUNTO

POO organiza el código, lo hace reutilizable y más fácil de mantener.



3.3 LO QUE VIENE DESPUÉS



- ★ Herencia: una clase puede heredar de otra.
- ★ Encapsulación: proteger los datos del objeto.
- ★ Polimorfismo: diferentes clases, mismo comportamiento.
- ★ Methods especiales: `__str__`, `__repr__` y más.
- ★ Class Methods y Static Methods.
- ★ Propiedades (properties) para controlar accesos.

POO es solo el comienzo. Con estas bases podrás construir programas más grandes y mejores.



EN RESUMEN

- ✓ Una CLASE es el molde.
- ✓ Un OBJETO es una copia del molde.
- ✓ Los ATRIBUTOS guardan datos.
- ✓ Los MÉTODOS realizan acciones.
- ✓ Cada objeto tiene sus propios datos.
- ✓ POO = código organizado y reutilizable.



Volumen 24 • POO Básico

Designed & Summarized by GuilleGiu

APRENDIENDO HOY, CRECIENDO SIEMPRE